UNIVERSITY OF COPENHAGEN
DEPARTMENT OF ECONOMICS



Master's thesis

Hans Andrias Gregoriussen

Enter the Autoencoder

Putting the Conditional Autoencoder Asset Pricing Model under Scrutiny

Date: 09-11-2024

Advisor: Rasmus Søndergaard Pedersen

Department	Department of Economics
Authors:	Hans Andrias Gregoriussen
Title and subtitle:	Enter the Autoencoder: Putting the Conditional Autoencoder Asset Pricing Model under Scrutiny
Advisor:	Rasmus Søndergaard Pedersen
Date:	09-11-2024

Abstract: This thesis paper investigates properties of the Conditional Autoencoder (CA) model, which is a latent factor model based on neural networks, proposed by Gu et al. 2020. The CA model allows for non-linearities in its factor loadings through neural networks while maintaining linear latent risk factors. This allows it to capture non-linear relationships between asset characteristics and expected returns. The CA model assumes a no-arbitrage condition, ensuring that all excess returns are fully compensated for risk. The properties examined are **i)** The predictive accuracy at the individual stock-level and at the portfolio-level. **ii)** Portfolio Forecast Performance with respect to Mean-Variance Efficiency and economic performance. **iii)** Stock characteristic selection by identifying which stock characteristics are most influential. **iv)** Risk Compensation versus capturing anomalies by determining whether the CA model captures true risk premiums or merely exploits market anomalies. **v)** Testing the models' Robustness analysis by introducing noise characteristics to the dataset. **vi)** The finite-sample performance is evaluated by a Monte Carlo Simulation study with 100 repetitions of simulated datasets to evaluate the models' prediction- and mean-variance portfolio performance. CA model variations with zero to three hidden layers in is factor loading network are compared against latent factor benchmark models: Instrumented Principal Components Analysis (IPCA), Principal Components Regression (PCR) Partial Least Squares Regression (PLS) and of traditional Fama& French models with observable risk factors are additionally assessed against the CA model. The number of latent factors. For all models, one to six factors are estimated to ensure comparability. The data consists of monthly stock returns, listed on major US stock exchanges NYSE, AMEX, and NASDAQ along along with 94 stock characteristics from the CRSP and Compustat databases, covering the period from 1980 to 2020. A walk-forward time series validation scheme is used to ensure robust out-of-sample predictions. Lasso regularization is employed in the CA model via a grid search to optimize the model performance. The out-of-sample predictions are evaluated using Total R^2 and Predictive R^2 metrics to assess how well the models explain contemporaneous realized factor returns and individual stock returns through systematic risk factors respectively. For the portfolio forecast performance, equal-weighted and a value-weighed long-short portfolios were created. The mean-variance efficiency performance was evaluated using the Sharpe Ratio the Economic performance was evaluated using average returns, Value-at-Risk and portfolio turnover rates. The Characteristic-importance was analyzed using a permutation method that iteratively shuffles each stock characteristic, while measuring the decrease in Total-R^2 performance, holding the others constant. The Robustness of the CA models was tested by introducing simulated noise characteristics into the dataset. With introduced noise characteristics, the prediction performance was evaluated at the individual- and at the portfolio-level along with a characteristics importance analysis to assess the stability of the selection of predictors. Finally, a Monte Carlo Simulation study was conducted with 100 repetitions to evaluate predictive performance under linear and non-linear data-generating scenarios. The Monte Carlo Simulation Study evaluated the predictive performance at the individual stock and portfolio levels, as well as the mean-variance efficiency of portfolios to assess the models' finite-sample performance. The findings of this paper showed that the CA model consistently outperformed the benchmark models in terms of Total-R^2 of and Predictive R^2 at the individual stock and at the portfolio level. The CA model performed significantly better in forecasting

portfolios, than the benchmark models in terms Sharpe Ratios and achieved higher average risk-adjusted returns than the benchmark models. Additionally, the CA model consistently identified key stock characteristics in the characteristic-importance analysis, even with noise characteristics to a smaller degree, indicating some degree of robustness compared to the benchmark models. When noise characteristics were introduced to the model, the CA model maintained better performance than the benchmark models both at the individual stock and at the portfolio level. Finally, the CA models demonstrated better predictive performance relative to the benchmark models at the individual stock-level and at the portfolio level. However, the IPCA model performed at least as well as the CA model in terms of risk-adjusted returns.Finally, the CA models demonstrated better predictive performance relative to the benchmark models in Monte Carlo Simulations, both at the individual stock-level and at the portfolio level. However, the IPCA model performed at least as well as the CA model in terms of risk-adjusted returns in the Monte Carlo Simulations. The superior performance of the CA model variations may be attributed to their ability to capture non-linear relationships and the absence of explicit regularization in the loss functions of benchmark models. These results suggest the at the Conditional Autoencoder model is a robust and effective tool for asset pricing, offering significant improvements over traditional models.

Table of content

1 Introduction **7**

2 Methodology **11**

2.1 Splitting of the Dataset for predictions . . . 11

2.2 Estimation methods . . . 13

2.2.1 Conditional Autoencoder . . . 14

2.2.2 Instrumented Principal Components Analysis (IPCA) . . . 17

2.2.3 Principal Components - Analysis (PCA) and Regression (PCR) . . . 19

2.2.4 Partial Least Squares . . . 21

2.2.5 Observable Factor Models . . . 22

2.3 Performance Evaluation Metrics: . . . 23

3 Empirical analysis **26**

3.1 data . . . 26

3.2 Out-of-sample performance evaluation . . . 26

3.3 Portfolio forecast performance . . . 31

3.4 Risk compensation versus anomalies . . . 34

3.5 Characteristic importance . . . 39

3.6 Robustness Check . . . 42

3.7 Simulation Study . . . 47

3.7.1 Monte Carlo Performance Evaluation of r_{t+1} . . . 49

3.7.2 Monte Carlo Performance Evaluation of x_{t+1} . . . 51

3.7.3 Monte Carlo Performance Evaluation of Long-Short portfolios . . . 53

4 Discussion **54**

5 Conclusion **56**

A Appendix **59**

A.1 Derivation of the Expected return from the Euler Equation . . . 59

A.2 Feed-Forward Neural Networks (FFNs) . . . 60

A.3 B: Backpropagation . . . 62

A.4 The conventional Autoencoder model . . . 66

A.5 description of the diagram of the Conditional Autoencoder model . . . 67

A.6 IPCA estimation of the unrestricted model, $\Gamma_\alpha \neq 0$ . . . 69

A.7 Derivation of the First Order Condition for the IPCA estimator w.r.t. $\boldsymbol{f_{t+1}}$ . . . 70

A.8 Derivation of the First Order Condition for the IPCA estimator w.r.t. $\boldsymbol{\Gamma_\alpha}$ . . . 70

A.9 Alternating Least Squares (ALS) algorithm . . . 71

A.10 IPCA assumptions for consistency . . . 72
A.11 Principal Components Analysis in details . . . 74
A.12 Singular Value Decomposition (SVD) and Diagonalization . . . 75
A.13 Detailed Explanation of Partial Least Squares and Its Equivalence to the Three-Pass Regression Filter . . . 77
A.13.1 Partial Least Squares (PLS) Regression . . . 77
A.13.2 The Three-Pass Regression Filter (3PRF) . . . 78
A.13.3 Equivalence of PLS and 3PRF . . . 80
A.13.4 Mathematical Proof of Equivalence . . . 80
A.14 Walk-Forward validation scheme . . . 82
A.15 Predictive R^2 by Gu et al. 2020 . . . 82
A.16 L: Calculation of Portfolio Weights . . . 83
A.17 Law of Large Numbers (LLN) and Central Limit Theorem (CLT) . . . 84
A.17.1 Law of Large Numbers (LLN) . . . 84
A.17.2 Central Limit Theorem (CLT) . . . 84

1 Introduction

The discipline of asset pricing is a fundamental aspect of financial economics, which plays a crucial role in understanding the dynamics of financial markets. Asset pricing is a cornerstone of financial economics, essential for understanding the valuation of securities, managing risk, and optimizing portfolios. Its primary goal is to estimate expected excess returns[1], which represent the compensation investors receive for taking on additional risk. Asset pricing models serve as indispensable tools for investors, financial institutions, and policymakers to make informed decisions. These models guide decisions related to investment opportunities, portfolio risk management, and policy frameworks that promote market stability and efficiency.

Central to asset pricing is the **Stochastic Discount Factor (SDF)**, which links present asset prices to their future payoffs by adjusting for risk and time. This concept underpins how investors are compensated for risk[2].

While the **SDF** provides a theoretical link between risk and returns, various models have been developed to capture this relationship in practice. The evolution of asset pricing models has sought to improve our understanding of how systematic risk affects asset returns. The **Capital Asset Pricing Model (CAPM)**, introduced by Sharpe 1964 and Linter 1965 has established a foundational framework for studying the relationship between risk and return. **CAPM** states that an asset's excess returns is proportional to its beta, which measures the sensitivity to the overall market. However, **CAPM**'s reliance on a single market risk factor has been challenged by empirical evidence of other risk drivers. The **Arbitrage Pricing Theory (APT)**, proposed by Ross 1976, expanded on this by introducing multiple factors beyond the market risk factor, such as macroeconomic indicators and industry-specific risks, allowing for a more flexible explanation of asset returns. Building on these models, the **Fama-French three-factor model** (Fama and French 1993) introduced two additional factors: size (Small Minus Big, SMB) and value (High Minus Low, HML). These factors account for the differences in returns associated with small-cap stocks and high book-to-market ratio stocks, which are not fully explained by the **CAPM**'s market risk factor. Further refinements came with the **Carhart four-factor model**, which added momentum as a risk factor, reflecting the tendency of stocks with strong past performance to continue outperforming other stocks. The five-factor model introduced by Fama and French 2015 added factors for profitability and investment, capturing the impact of firm performance and capital investment on returns.

Over the years since the inception of the **CAPM**, a large number of studies have introduced risk factors claimed to predict the cross-sectional variation in expected excess returns. This growing collection of risk factors is commonly referred to as the "**factor zoo**", a term coined by Cochrane 2011. Hou et al. 2016 examine the impact of publishing academic papers that introduce risk factors and find that many of these factors exhibit predictive power before publication, primarily due to market mispricing. However,

[1] Excess returns refer to the returns of an asset that exceed the risk-free rate or the expected return from a benchmark. They represent the additional compensation investors receive for taking on risk.

[2] This relationship is formally captured by the Euler equation described in section 2.2

after publication of many proposed risk factors, their predictability often declines, as informed trading increases market efficiency, reducing the effectiveness of these factors. Many risk factors in the **factor zoo**, have been tested to likely have a false significance in asset pricing tests, which makes it necessary to have higher thresholds for significance asset pricing tests. Harvey and Zhu 2016 argue for three main reasons for false significance of asset pricing test: **i)** likelihood of discovering a true factor has diminished **ii)** that there is a limited amount of available data and **iii)** that the cost of acquiring data has decreased, leaving the selection of factors with the highest prior expected returns not being prioritized to the same degree as earlier. Studying the **factor zoo** of pre-specified factors, based on prior knowledge of expected return behavior, requires treating these factors as observable. The excess returns of individual stocks or portfolios, are then regressed on the factors in a linear regression, from which the coefficients and the intercepts are estimated. The downside of using observable risk factors, is that they most likely give a partial picture about the behavior of the expected returns. This is because the observable risk factors are speculated to be proxies for common risk factors. In response to the overwhelming number of proposed factors, various machine learning techniques have been developed. Methods like **LASSO regression** focus on selecting the most significant risk factors by imposing sparsity on the less relevant ones Chinco et al. 2017. Other approaches, such as random forests Kaczmarek et al. 2022, leverage decision trees to enhance predictive performance. A notable study by Gu et al. 2019 provides an extensive evaluation of machine learning models, demonstrating that neural networks can substantially improve the prediction of individual stock and portfolio returns, outperforming traditional asset pricing models by a wide margin. Additionally, Chen and Perez 2021 employ a combination of neural network techniques, including **feed-forward neural networks (FFN)**, **Long Short-Term Memory (LSTM)** networks, and **Generative Adversarial Networks (GANs)**, to capture non-linearities, reveal subtle economic state processes, and identify portfolio strategies that maximize unexplained pricing information. These advanced methods enable the precise identification of key factors driving asset prices.

Dimension reduction is a sub-branch of the asset pricing literature that views risk factors as latent (or unobservable). These latent factors can be estimated using statistical techniques that leverage predictive variation across a broad set of risk factors to explain expected excess returns Kelly, Priutt, and Su 2018, effectively capturing underlying variations.

In this context, risk factors are assumed to be latent, represented as linear or non-linear combinations of many observable risk factors. Connor and Korajczyk 1986 pioneered the use of latent factor models in asset pricing by estimating unobservable factors through techniques such as Principal Components Analysis **(PCA)**. Ludvigson and Ng 2007 expanded this approach by applying dynamic factor models to extract latent factors from extensive macroeconomic data.

The distribution of asset returns is known to vary significantly over time, meaning that static factor models fail to capture important conditional information. A notable advancement in latent factor models, introduced by Kelly, Priutt, and Su 2018, proposed the **Instrumented Principal Components Analysis (IPCA)** as an asset pricing model. This model builds upon the original **IPCA** framework presented in Kelly, Priutt, and Su 2017, which uses time-varying factor loadings. Kelly, Priutt, and Su 2018

demonstrate substantial empirical improvements by incorporating stock-specific covariates into the factor loadings. In their model, the **IPCA** consists of two key functions: **i)** a factor loading function (or beta function) that integrates stock-specific information across the cross-section, **ii)** and a second function estimating latent factors based on many individual stock returns for each time period, reducing them to a lower-dimensional number of latent factors. The inclusion of stock-specific covariates enhances the accuracy of both the factor loadings and the latent factors.

The authors also show that their **IPCA** model explains nearly all the return variation through the conditional factor loadings (conditional betas), leaving anomaly returns almost non-existent. Their results indicate that estimated alphas (anomaly returns) are typically insignificant or low in absolute value. The key innovation of the **IPCA** asset pricing model is its ability to accommodate time-varying factor loadings based on observable asset characteristics. By merging the strengths of both latent factor models and asset characteristic-based asset pricing , the **IPCA** provides a more flexible and accurate framework for explaining asset returns.

This paper builds on the work of Gu et al. 2020, which introduced a novel autoencoder asset pricing model with latent risk factors and compared it to models with both observable and latent factors. The autoencoder model in Gu et al. 2020 utilizes conditional factor loadings that estimate risk exposures through a neural network, effectively capturing non-linear relationships between the asset characteristics of individual stocks.

The simplest form of the Conditional Autoencoder is a generalization of the **IPCA** model proposed by Kelly, Priutt, and Su 2018, when no hidden layers are present in the neural network[3]. This foundational version can be expanded by incorporating hidden layers to model the non-linear nature of risk exposures to the estimated latent factors.

This paper aims to evaluate the predictive power, economic performance, the ability to predict risk versus anomalies, stock characteristic-importance and the robustness of the Conditional Autoencoder model proposed by Gu et al. 2020 through a series of empirical tests. **i)** The model's predictive performance is compared against benchmark models, including **IPCA**, **PCA**, **PLS**, and **Observable factor models**, both at the individual stock and the portfolio levels. Additionally, this paper evaluates: **ii)** the mean-variance efficiency of portfolio formation, **iii)** the model's ability to explain risk or capture anomalies, **iii)** Characteristics importance analysis **iv)** Robustness check examining the prediction performance and characteristics importance with introduced noise characteristics **v)** Prediction and economic performance evaluations in Monte Carlo finite sample simulations at both the individual stock and portfolio levels, considering linear and non-linear data generation processes. Due to computational restrictions, some performance evaluation metrics have been adjusted to ensure feasibility. The methodologies for analyzing feature importances, the introduction of noise characteristics, and the simulation study were revised to enhance the robustness of the results.

This paper is structured in two main parts. **i)** a Methodological part (**Section** 2) that describes the

[3]demonstrated in Appendix A (Proof of Proposition 2 in Gu et al. 2020)

dataset split, the theoretical framework of asset pricing, focusing identification and estimation of the **SDF**, the technical aspect of the Conditional Autoencoder-models, the benchmark-models as well as the performance evaluation metrics employed. The second part, **ii)** is an empirical section (**Section** 3) with empirical analyses. In the methodological part of the paper, **Section 2.1** details the methodology for splitting the dataset and the training scheme, including time-series walk-forward cross-validation with a 20-year training set, 2-year validation set, and 1-year test set for each fold. Regularization techniques such as L_1 regularization and grid search are used for tuning the Conditional Autoencoder models, ensuring optimal out-of-sample performances across 18 non-overlapping test periods.

Section 2.2.1 provides a theoretical description of the **Conditional Autoencoder (CA)** model, which generalizes the **IPCA** model by employing neural networks to estimate non-linear factor loadings and latent risk factors. The CA model uses a beta network and a factor network, processing lagged stock characteristics and characteristic-managed portfolio returns to predict excess stock returns while reducing dimensionality and computational cost. **Section 2.2.2** provides a theoretical description of the **IPCA** model, detailing its use of stock characteristics and returns to estimate conditional factor loadings and latent risk factors. The section explains the model structure, estimation through iterative methods such as Alternating Least Squares, and normalization techniques for unique identification. Additionally, it highlights how **IPCA** captures time-varying risk exposures through re-weighted covariation among characteristic-managed portfolios. **Section 2.2.3** provides a theoretical description of the **Principal Components Regression (PCR)** model, detailing how **PCA** is used to reduce dimensionality by selecting uncorrelated principal components that capture the most variance. These components are then used as predictors in a regression model, addressing multicollinearity and reducing overfitting. **Section 2.2.4** provides a theoretical description of the **Partial Least Squares Regression (PLSR)** model, detailing how it reduces dimensionality by constructing latent factors that maximize predictive power with stock returns. This section explains how **PLS** differs from **PCR** by focusing on components with higher return predictability, even if it sacrifices an exact representation of the original features. **Section 2.2.5** provides a theoretical description of **Observable factor models**, detailing the composition and progression from **CAPM** to more complex **Fama-French** models with up to six factors. This section also explains how the number of factors (K) is varied from one to six to ensure comparability with latent factor models and evaluates the explanatory power of these models in explaining asset returns. **Section 2.3** of the methodological defines the performance evaluation metrics used to assess the predictive accuracy of the models, including Total-R^2 out-of-sample, Predictive R^2 as well various portfolio evaluation metrics, such as the Sharpe Ratio among others. **Section 3.1** describes the data used for the empirical analysis, including monthly stock returns from the CRSP database covering U.S. stock exchanges over the period 1980 to 2020. It details the use of 94 stock characteristics from Compustat and CRSP and explains preprocessing steps, such as lagging characteristics to prevent future bias and rank-normalizing to handle skewed distributions. **Section 3.2** presents the out-of-sample prediction results for individual stocks and portfolios, using key performance metrics such as predictive-R^2 and total-R^2. The section compares the performance of the Autoencoder models against benchmark models, showing that while Autoencoder models typically achieve higher total-

R^2, predictive-R^2 results are modest across all models. **Section 3.3** evaluates the economic significance of model predictions through portfolio forecasting. The performance of both equal-weighted and value-weighted long-short portfolios is analyzed using key metrics such as the Sharpe Ratio, average returns, volatility, Value-at-Risk (VaR), and turnover. The section compares the performance of the Conditional Autoencoder models against benchmark models, showcasing the CA models' superior risk-adjusted returns and economic performance. Section 3.4 investigates whether the autoencoder models capture risk premia effectively or reveal anomalies by analyzing pricing errors (alphas) using characteristic-managed portfolios. The section applies hypothesis testing on alphas to assess if they are statistically significant, indicating potential uncompensated returns. **Section 3.5** examines the importance of stock characteristics in the Conditional Autoencoder and benchmark models using the permutation method, which measures the impact of characteristic shuffling on R^2_{OOS}. The section further delves into analyzing the contributions of the factor- and beta-networks within the Conditional Autoencoder to provide deeper insight into characteristic selection and prediction accuracy. **Section 3.6** assesses the robustness of the models by introducing noise characteristics into the dataset to test their ability to consistently select relevant stock characteristics and maintain stable predictions. The section evaluates the impact of noise on the models' performance, comparing total and predictive R^2 for individual stocks and characteristic-managed portfolios with and without noise. Additionally, a characteristic-importance analysis is performed to observe changes in characteristic selection under noisy conditions. **Section 3.7** conducts a comprehensive simulation study using Monte Carlo experiments to assess the performance of the Conditional Autoencoder models and benchmark models under both linear and non-linear data-generating processes (DGPs). The simulations evaluate model robustness by analyzing Total and Predictive R^2 and Sharpe Ratios for long-short portfolios over 100 repetitions. This approach provides insights into how well the models capture linear and non-linear relationships and their predictive stability across different data scenarios. Section 4 makes coveys a discussion of the paper. Section 5 Concludes the paper with its findings.

2 Methodology

2.1 Splitting of the Dataset for predictions

To make predictions out-of-sample for evaluating the performance of a model, the dataset is split into parts. The IPCA, the PCA and the Observable Factor models (FF) do not use regularization. For these models, the dataset is divided into a training set and a test set. The models are fit on the training set, and predictions are made directly on the test set, ensuring they are purely out-of-sample. For the Autoencoder models, the dataset is divided into a training set, a validation set, and a test set. These models apply L_1 regularization (Lasso) to the loss function to reduce overfitting on the training set, thereby improving out-of-sample prediction performance. During training, the weights and biases of the neural network are updated to minimize the Mean Squared Error (MSE) loss function based on the in-sample data from the training set. The validation set is used to fine-tune the L_1 regularization parameterwoth a grid search

is across a range of L_1 values[4] to optimize the out-of-sample model predictions. After evaluating the MSE loss for each L_1 value in the grid search, the value yielding the highest out-of-sample R^2 on the validation set is selected[5]. The optimal L_1 regularization value is determined, to make out-of-sample predictions on the test set. The data is split using a time-series walk-forward cross-validation scheme, with a 20-year training set, a 2-year validation set, and a 1-year test set. All splits are non-overlapping and maintain the temporal order of the data to avoid any future bias. Once the model has been trained, tuned on the validation set, and made predictions on the test set, the entire process moves forward by one year. This iteration is repeated for each test set period, with each iteration considered a fold, similar to a conventional cross-validation scheme. In each fold, predictions are made using annual estimations for the test set. In total, 18 splits are performed, resulting in an 18-year out-of-sample test period for predictions. Once predictions for all years have been made, the eighteen 1-year prediction test sets are put together from beginning to end, resulting in a consolidated "test set" of predictions.

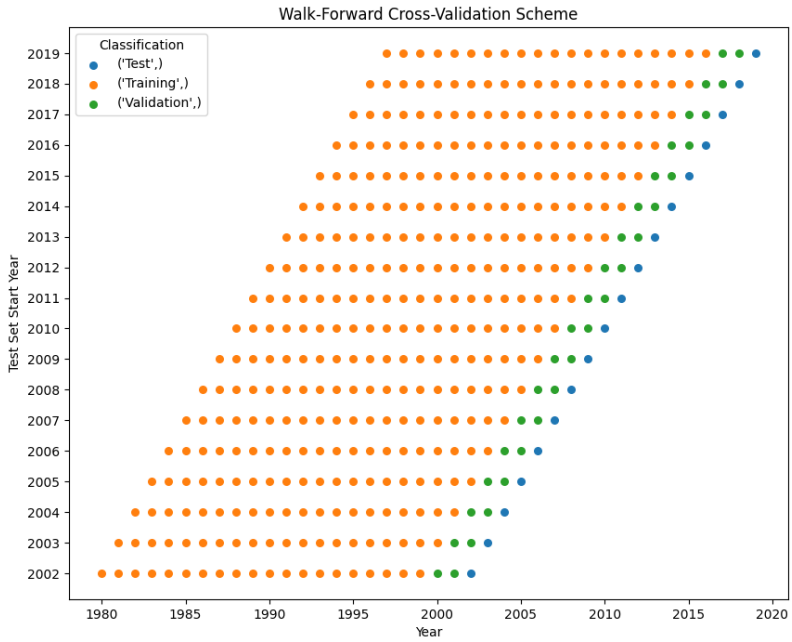


Figure 1: Illustration of the Walking forward cross-validation scheme

Figure 1 illustrates the structure of the walk-forward cross-validation scheme. The **x-axis** represents the time period, while the **y-axis** indicates the start year of the test set for each iteration. Each year of the training set, validation set and test set are marked by a yellow, green and blue dot respectively. As shown in **Figure 1**, the lengths of the time-periods of the training, validation, and test sets are fixed, and they

[4]The L_1 values considered are $L_1 \in 0.00000001, 0.0000001, 0.000001, 0.00001, 0.0001, 0.001, 0.01, 0.1$

[5]The estimation criterion is to minimize the Mean Squared Error (MSE) loss function, while the tuning criterion is to maximize the out-of-sample R^2 on the validation set, by tuning the regularization L_1 penalty values. Once the optimal L_1 regularization value is selected, the next step is to make out-of-sample predictions using this value on the test set.

move forward by one year (the length of the test set) for prediction of one year at a time, consisting of 18 iterations in total (hence the words "walk forward").

2.2 Estimation methods

In asset pricing, a fundamental goal is to relate an asset's expected return to the risk it bears. The **Stochastic Discount Factor (SDF)** framework formalizes this by setting the expected excess return equal to zero when adjusted by the **SDF**. This approach captures the time value of money and risk, bridging the connection between expected returns and the systematic risks investors bear. This framework is central to both theoretical and empirical asset pricing models, as it lays the foundation for understanding how risk and returns are related across different assets. The Euler equation, derived from the no-arbitrage condition, ensuring that assets are uncompensated for risk, and serves as a basis for understanding the relationship between expected excess returns and risks in asset pricing (Kelly, Priutt, and Su 2018). This implies that the expected excess return of any asset is fully compensated for risk and is proportional to its covariance with the **SDF**. The Euler equation is defined as[6]:

$$E_t[m_{t+1}r_{i,t+1}] = 0 \tag{1}$$

Where $E_t[\cdot]$ denotes the conditional expectation given information available at time t, m_{t+1} is the **SDF** and $r_{i,t+1}$ is the excess return of asset $i < t$ time $t+1$. Equation (1) (the Euler equation) suggests that, in the absence of arbitrage opportunities, any asset's expected return should precisely offset the risk it bears. The expected return equation is derived by decomposing the **SDF** into systematic risk factor exposures and their associated prices. A reformulation links expected returns to the asset's exposure to systematic risk, captured by $\beta_{i,t}$, and the price investors set for that risk, λ_t. By reformulating the Euler equation and utilizing properties of covariances and variances, the following is obtained[7]:

$$E_t[r_{i,t+1}] = \beta_{i,t}^T \lambda_t \tag{2}$$

where

$$\beta_{i,t} = \frac{Cov_t(m_{t+1}, r_{t+1})}{Var_t(m_{t+1})}, \quad \lambda_t = -\frac{Var_t(m_{t+1})}{E_t[m_{t+1}]} \tag{3}$$

Where $\beta_{i,t}$ represents asset i's sensitivity to systematic risk at time t, (capturing its exposure to broader economic shocks), and λ_t represents the market price of risk (the premium investors demand for taking on additional systematic risk). Equation (2) links the expected excess return, of asset i at time $t+1$, to its systematic risk exposure, $\beta_{i,t}$ and the corresponding risk price, λ_t, at time t, aligning with the fundamental principles of asset pricing. When the **SDF**, m_{t+1} a linear function of some factors f_{t+1} (representing

[6] Kelly, Priutt, and Su 2018

[7] The derivation of the reformulation of the Euler Equation is described in Appendix A.1

systematic risk), the excess returns can be described with a factor model (Hansen and Richard 1987):

$$r_{i,t+1} = \alpha_{i,t} + \beta^T f_{t+1} + \epsilon_{i,t+1} \tag{4}$$

Where $\beta_{i,t}$ represents the sensitivity of asset i to the systematic risk factors at time $t+1$, $\alpha_{i,t}$ is the intercept term[8] (which is zero under the no-arbitrage condition), and $\epsilon_{i,t}$ is the idiosyncratic error term with $E_t[\epsilon_{i,t+1}] = 0$ and $E_t[\epsilon_{i,t+1} f_{t+1}] = 0$.

This paper focuses on estimating **Stochastic Discount Factors (SDFs)** using latent risk factor models, with particular emphasis on the **Conditional Autoencoder** model proposed by Gu et al. 2020. This model will be compared against several benchmark models, including **Instrumented Principal Components Analysis (IPCA)**, **Principal Components Regression (PCR)**, **Partial Least Squares (PLS)**, and various **Observable Factor models**, to as **Fama&French (FF)** model variations.

The following sections address the theoretical foundations and estimation methods for all models under consideration. First, the theoretical background of the Conditional Autoencoder model is described, followed by a description of estimation for the benchmark models. Since the Conditional Autoencoder model uses neural networks to capture nonlinear patterns in risk-return relationships, a brief introduction to the underlying theory of Neural Networks, backpropagation, and conventional Autoencoders, is provided in the appendices. Specifically, A.2 covers Neural Networks, A.3 discusses the Backpropagation algorithm, and A.4 provides details of the architecture of the Conventional Autoencoder model.

2.2.1 Conditional Autoencoder

The **Conditional Autoencoder** is a generalization of the **IPCA** model, which is more flexible with respect to allowing for non-linearities in its factor loadings. The **Conditional Autoencoder** model proposed by Gu et al. 2020, is structured as a factor model at its highest level. The factor model representation of the **Conditional Autoencoder** consists of latent factor loadings and risk factors, estimated separately by a beta- and a factor Neural-Network, which are multiplied together to forecast the final return. The factor model representation of the **Conditional Autoencoder** model is given by:

$$r_{i,t+1} = \beta_{i,t}^T f_{t+1} + \epsilon_{i,t+1} \tag{5}$$

Where $r_{i,t+1}$ is the scalar return of stock i at time $t+1$, $\beta_{i,t}^T$ is the transposed $1 \times K$ vector of factor loadings for stock i at time t, f_{t+1} is a $K \times 1$ vector of latent risk factors at time $t+1$ and $\epsilon_{i,t+1}$ is the idiosyncratic scalar error term for stock i at time $t+1$. This equation captures how each stock's return is influenced common factors and its specific sensitivities to these factors.

The factor loadings, $\beta_{i,t}$ are computed using a neural network, which reduces the $P \times 1$ dimension

[8]The no-arbitrage assumption, which ensures no unearned profit opportunities, implies that an intercept term in an asset's expected return, ($\alpha_{i,t}$), should be zero. If an intercept were present, it would suggest returns uncompensated for risk, indicating possible mispricing. An intercept-free equation, therefore, represents a 'fairly priced' asset, where returns are entirely explained by risk exposures.

lagged stock characteristics $z_{i,t}$ for each stock i to a $K \times 1$ dimension latent factor loadings in its output. The beta network introduces non-linearly in the factor loadings by having one or more hidden layers. This implies that $K \times 1$ factor loadings computed from the beta network with no hidden layers are linear combinations of $P \times 1$. The latent risk factors for time period $t + 1$ are estimated by a factor network, which takes as input either the $N \times 1$ vector of individual excess returns, r_{t+1}, or the $P \times 1$ vector of characteristic-managed portfolio returns, x_{t+1}, where $P < N$. In this paper, the characteristic-managed portfolio returns x_{t+1} are used as input because their smaller dimensionality $P < N$ reduces the number of parameters (weights and biases) in the Neural Networks, leading to lower computational costs. The factor network consists of an input layer and an output layer, and no hidden layers. This configuration restricts the network to only perform linear combinations of the $P \times 1$ input vector to produce the $K \times 1$ vector of latent risk factors at its output. The beta network is estimating the latent factor loadings, $\beta_{i,t}$ of the model, and is represented by these equations[9]:

$$z_{i,t}^{(0)} = z_{i,t}, \tag{6}$$

$$z_{i,t}^{(l)} = g\left(W^{(l-1)} z_{i,t}^{(l-1)} + b^{(l-1)}\right), \quad l = 1, \ldots, L_\beta, \tag{7}$$

$$\beta_{i,t} = W^{(L_\beta)} z_{i,t}^{(L_\beta)} + b^{(L_\beta)}. \tag{8}$$

Equation (6) defines the input layer of the beta network, where $z_{i,t}$ is a $P \times 1$ vector of P lagged characteristics for stock i at time t. Equation (7) describes the hidden layers for $l = 1, ..., L_\beta$, where $W^{(l-1)}$ is a $d_l \times d_{l-1}$ weight matrix connecting layer $l-1$ to layer l, b^{l-1} is a $d_l \times 1$ bias vector for layer l, $g(\cdot)$ is the activation function, applied element-wise and d_{l-1} and d_l are the numbers of neurons in layers $l-1$ and l, respectively. Equation (8) defines the output layer, where $W^{(L_\beta)}$ is a $K \times d_{L_\beta}$ weight matrix, $b^{(L_\beta)}$ is a $K \times 1$ bias vector and $\beta_{i,t}$ is a $K \times 1$ vector of factor loadings for stock i. The beta network transforms the stock i's characteristics $z_{i,t}$ into factor loadings through a series of linear and non-linear transformations. The hidden layers introduce non-linearity, allowing the model to capture advanced relationships between characteristics and factor loadings.

Before describing the factor network, the input processing step of the characteristic-managed portfolio returns is described. The set of characteristic-managed portfolios for each time period are defined as follows[10]:

$$x_{t+1} = (Z_t^T Z_t)^{-1} Z_t^T r_{t+1} \tag{9}$$

Where x_{t+1} is a $P \times 1$ vector with P characteristic-managed portfolios for each time period, which is dynamically re-weighted each time period. Equation (9) effectively regresses the returns, r_{t+1} in the characteristics, Z_t, providing a summary of returns associated with each characteristic. In x_{t+1}, the jth element, corresponds to a long-short portfolio which is constructed by sorting stocks based on the jth

[9] Gu et al. 2020
[10] Gu et al. 2020

stock characteristic. The characteristic-managed portfolios can be considered a function of individual lagged stock characteristics Z_t, which reduces N individual stock excess returns in the vector r_{t+1} to P units which correspond to the number of stock characteristics in x_{t+1}. The number of N stocks each time period $t+1$, varies and is on average 5000, while the number of stock characteristics, P each time period is 94 for all time periods. Using x_{t+1} as input avoids the need for preprocessing an unbalanced panel dataset of individual stock returns, while reducing computational cost because of less parameters. Additionally, x_{t+1} diversifies most of the idiosyncratic risk away, which reduces signal to noise ratio.Gu et al. 2020 argue furthermore that x_{t+1} links the **Conditional Autoencoder** model to the finance literature on characteristic-managed portfolios which shows that conditional linear factor models of individual returns can be repurposed as static factor analysis on characteristic-managed portfolios. The Factor Network is a simple linear model with no hidden layers, directly mapping the input vector x_{t+1} to the vector f_{t+1} of factors. The network is described by the following two equations [11]:

$$x_{t+1}^{(0)} = x_{t+1}, \tag{10}$$

$$f_{t+1} = \tilde{W}^{(0)} x_{t+1}^{(0)} + \tilde{b}^{(0)} \tag{11}$$

Equation (10) describes the input layer, $x_{t+1}^{(0)}$, of the factor network, which is initialized by the $P \times 1$ vector of characteristic-managed portfolios, x_{t+1}. The input layer passes this vector to the next layer without any transformation. Equation (11) describes the $K \times 1$ output vector of the factor network, f_{t+1}. This output is produced by applying a linear transformation to the input layer, $x_{t+1}^{(0)}$, where $\tilde{W}^{(0)}$ is the weight matrix and $\tilde{b}^{(0)}$ is the bias vector. The weight matrix, $\tilde{W}^{(0)}$, determines how the inputs (characteristic-managed portfolio returns x_{t+1}) are combined to produce the latent risk factors, while the bias vector, $\tilde{b}^{(0)}$, adjusts the output independently of the input.

The $K \times 1$ matrix output of the beta network, $\beta_{i,t}$ is transposed and multiplied with the $K \times 1$ matrix output of the factor network, $\hat{f}_{t+1}$, which results in the predicted excess return of stock i at time period $t+1$:

$$\hat{r}_{i,t+1} = \beta_{i,t}^T \hat{f}_{t+1} \tag{12}$$

where $\hat{r}_{i,t+1}$ is a scalar, and the factor loadings $\beta_{i,t}$ for each stock consist of $K \times 1$ vectors. The consolidated predicted returns of the **Conditional Autoencoder** stacks the predicted returns and factor loadings, such that they are represented in matrix form. The predicted returns in matrix form, consists of a $N \times 1$ vector, $\hat{r}_{t+1}$ while the factor loadings consist of a $N \times K$ matrix, B_t, such that the predicted excess returns in matrix output become:

$$\hat{r}_{t+1} = B_t \hat{f_{t+1}} \tag{13}$$

In this paper, the number of hidden layers in the beta network is treated as a hyperparameter. Consequently, four variations of the Conditional Autoencoder model are employed, each with a different number

[11] Gu et al. 2020

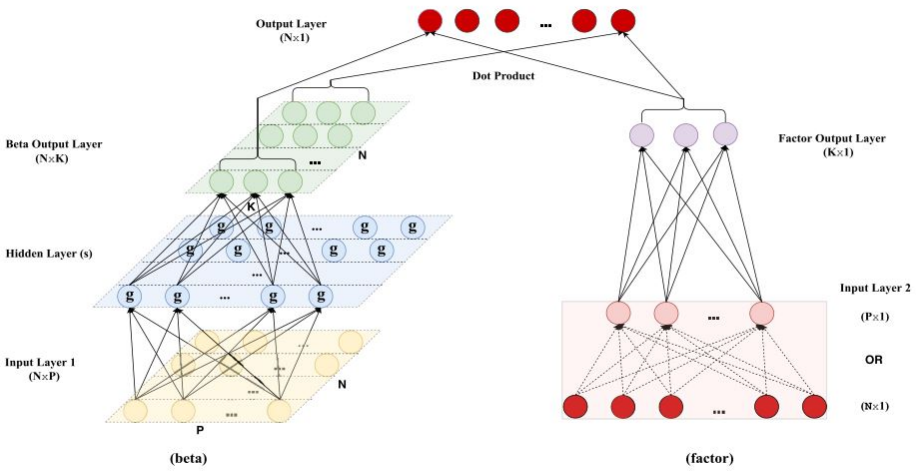


Figure 2: Diagram of the Conditional Autoencoder models' architecture, Gu et al. 2020

of hidden layers: CA_0 has zero hidden layers, CA_1 has one, CA_2 has two, and CA_3 has three. **Figure 2** shows a diagram of the **Conditional Autoencoder** in stacked stock observations for both the beta network **(Left Hand Side)** and factor network **(Right Hand Side)**. The beta network uses an $N \times P$ matrix as input for N stocks with P stock characteristics at time period t. The matrix input is then compressed to lower dimensions through the hidden layers of the beta network, which then is passed to the output $N \times K$ output matrix for N stocks with K latent factors. The factor network can either use a $N \times 1$ or or $P \times 1$ vector of individual stock returns or characteristic-managed portfolio returns, which are linearly combined directly to the corresponding dimension-reduced $K \times 1$ output vector from the output layer at time period $t + 1$. The outputs of the beta network and the factor network are multiplied, resulting in the $N \times 1$ vector of N predicted stock returns for time period $t + 1$. Appendix A.5 Describes the diagram of the **Conditional Autoencoder** model in detail, in terms of matrices, where individual stock observations are stacked.

2.2.2 Instrumented Principal Components Analysis (IPCA)

The **Instrumented Principal Components Analysis (IPCA)**, introduced by Kelly, Priutt, and Su 2017 and adapted for asset pricing by Kelly, Priutt, and Su 2018, is a latent factor model that predicts the cross-section of stock returns using stock characteristics and returns. It estimates latent risk factors and employs conditional factor loadings that adjust over time to asset-specific changes.

The **IPCA** model is specified by[12]:

$$r_{i,t+1} = \alpha_{i,t} + \beta_{i,t} f_{t+1} + \epsilon_{i,t+1} \tag{14}$$

$$\alpha_{i,t} = z_{i,t}^T \Gamma_\alpha + \nu_{\alpha,i,t} \tag{15}$$

[12]Kelly, Priutt, and Su 2018

$$\beta_{i,t} = z_{i,t}^T \Gamma_\beta + \nu_{\beta,i,t} \tag{16}$$

Here, $r_{i,t+1}$ is the scalar return of asset i at time $t+1$. The scalar intercept, $\alpha_{i,t}$, and the $K \times 1$ vector of factor loadings $\beta_{i,t}$ re functions of the $P \times 1$ vector of lagged asset characteristics $z_{i,t}$. Specifically, $\alpha_{i,t}$ is linearly mapped via the $P \times 1$ vector Γ_α and $\beta_{i,t}$ is linearly mapped via the $P \times K$ matrix Γ_β. The $K \times 1$ vector f_{t+1} represents latent factors at time $t+1$. $\epsilon_{i,t+1}$ is the scalar error term. The residual terms $\nu_{\alpha,i,t}$ (a scalar) and $\nu_{\beta,i,t}$ (a $K \times 1$ vector) capture the variation not explained by the systematic components $\alpha_{i,t}$ and $\beta_{i,t}$, respectively. To impose a no-arbitrage condition similar to the Conditional Autoencoder model, $\Gamma_\alpha = 0$ is imposed, implying that expected returns are explained only by $\beta_{i,t}$ and f_{t+1}. Combining residuals into $\epsilon^*_{i,t+1} = \epsilon_{i,t+1} + \nu_{\alpha,i,t} + \nu_{\beta,i,t} f_{t+1}$, the restricted **IPCA** model simplifies to[13]:

$$r_{i,t+1} = z_{i,t}^T \Gamma_\beta f_{t+1} + \epsilon^*_{i,t+1} \tag{17}$$

Here, $z_{i,t}^T \Gamma_\beta$ results in a $1 \times K$ row vector, which when multiplied by the $K \times 1$ vector, f_{t+1}, yields a scalar that models the expected return. Details on the unrestricted model ($\Gamma_\alpha \neq 0$) are provided in Appendix A.6. To estimate the $P \times K$ factor loading matrix Γ_β and $K \times 1$ factors vectors f_{t+1}, the model is expressed in vector form[14]:

$$r_{t+1} = Z_t \Gamma_\beta f_{t+1} + \epsilon^*_{t+1} \tag{18}$$

Where r_{t+1} is the $N \times 1$ vector of asset returns at time $t+1$, where N denotes the number of assets. Z_t is the $N \times P$ matrix of asset characteristics at time t, stacking the $P \times 1$ characteristic vectors, $z_{i,t}$ for all assets $i = 1, ..., N$. The product $Z_t \Gamma_\beta$ results in an $N \times K$ matrix, which when multiplied by f_{t+1}, yields the $N \times 1$ vector $Z_t \Gamma_\beta f_{t+1}$. ϵ^*_{t+1} is the residual vector. The estimation aims to minimize the Sum of Squared Errors (SSE):

$$\min_{\Gamma_\beta, F} \sum_{t=1}^{T-1} \left| r_{t+1} - Z_t \Gamma_\beta f_{t+1} \right|^2 \tag{19}$$

The first-order conditions are derived with respect to f_{t+1} and Γ_β [15][16]:

1. With respect to f_{t+1}:

$$\hat{f}_{t+1} = \left(\Gamma_\beta^T Z_t^T Z_t \Gamma_\beta \right)^{-1} \Gamma_\beta^T Z_t^T r_{t+1}, \quad \forall t \tag{20}$$

   Where Γ_β^T is a $K \times P$ matrix, and Z_t^T is a $P \times N$ matrix. The product $Z_t^T Z_t$ yields a $P \times P$ matrix, and $\Gamma_\beta^T Z_t^T Z_t \Gamma_\beta$ results in a $K \times K$ matrix. The inversion of this $K \times K$ matrix is required to solve for $\hat{f}_{t+1}$.

[13] Kelly, Priutt, and Su 2018
[14] Kelly, Priutt, and Su 2018
[15] The derivations of the first-order conditions for f_{t-1} and Γ_α are described in Appendix A.7 and A.8 respectively
[16] Kelly, Priutt, and Su 2018

2. With respect to Γ_β:

$$\text{Vec}(\Gamma'_\beta) = \left(\sum_{t=1}^{T} Z_t^T Z_t \otimes f_{t+1} f_{t+1}^T\right)^{-1} \left(\sum_{t=1}^{T} \left(Z_t^T r_{t+1} \otimes f_{t+1}^T\right)\right) \tag{21}$$

Where $\text{Vec}(\Gamma'_\beta)$ denotes the vectorization of Γ_β^T, resulting in a $KP \times 1$ vector. The Kronecker product, $Z_t^T Z_t \otimes f_{t+1} f_{t+1}^T$ produces a $KP \times KP$ matrix, which must be inverted to solve for $\text{Vec}(\Gamma'_\beta)$

Since closed-form solutions are unavailable, the Alternating Least Squares (ALS) method is employed to iteratively estimate Γ_β and f_{t+1} (see Appendix A.9). However, Γ_β and f_{t+1} are not unique because any pair $(\Gamma_\beta R^{-1}, R f_{t+1})$, where R is a non-singular $K \times K$ rotation matrix, is also a solution to the minimization problem. To uniquely identify the estimators without imposing economic restrictions, the factor loading matrix is normalized by imposing $\Gamma_\beta^\top \Gamma_\beta = I_K$, where I_K is the $K \times K$ identity matrix. Additionally, it is assumed that the unconditional covariance matrix of f_{t+1} is diagonal with descending diagonal entries and set the expectation $E[f_{t+1}] \leq 0$. Intuitively, IPCA captures common variation among characteristic-managed portfolios. It estimates K latent factors as linear combinations of these portfolios - the factors can be thought of as "portfolios of portfolios." Unlike Principal Components Analysis (PCA), which extracts factors based solely on maximizing explained variance, IPCA re-weights covariation among characteristic-managed portfolios across time and assets. This allows IPCA to adjust for time-varying properties and better capture dynamic risk exposures[17].

2.2.3 Principal Components - Analysis (PCA) and Regression (PCR)

Principal Components Regression (PCR) is a technique that employs **Principal Components Analysis (PCA)** as a preprocessing step to reduce the dimensionality of a dataset to a few factors before performing regression. PCA transforms an original $N \times P$ dataset matrix X with N stocks and P stock characteristics at each time period, into a smaller dataset of uncorrelated variables called **Principal Components (PCs)**. This results in a new $N \times K$ dataset Y with N stocks and K PCs (where $K < P$) at each time period. The transformation is given by:

$$Y = XG \tag{22}$$

Where, G is a $P \times K$ matrix, whose columns are eigenvectors corresponding to the K largest eigenvalues of the covariance matrix Σ_X of the dataset matrix[18] X.The matrix G serves as the transformation matrix that projects the original data X onto the new axes defined by the principal components. By multiplying X with G, we obtain Y, which is the representation of the data in the reduced K-dimensional space spanned by the principal components. Each column in G corresponds to a principal component and captures a

[17] Kelly, Priutt, and Su 2018

[18] These eigenvectors represent principal directions in the feature space along which the data exhibits the greatest variance.This ensures that each principal component captures distinct information, reducing redundancy and multicollinearity in the dataset.

unique direction of maximum variance in the data, while being orthogonal to the other components. The $P \times P$ covariance matrix Σ_X of X becomes diagonal as a result of the transformation, given by[19]:

$$\Sigma_Y = G^T \Sigma_X G = D \tag{23}$$

Where D is a diagonal matrix of eigenvalues of Σ_X in each of its columns, representing the variance captured by each PC[20]. The top K PCs, corresponding to the largest eigenvalues, are selected, and capture the most largest variance in the data.[21]. PCA simplifies the dataset by these constraints[22]:

1. **Ensuring Orthogonality**: the PCs are orthogonal (uncorrelated), so each captures distinct aspects of the original data.

2. **Capturing Maximum Variance**: The variance is concentrated in the first few PCs, meaning that the initial components capture most of the variation, while subsequent components contribute less and can often be discarded as noise.

The optimization in PCA aims to find the PCs that maximize variance under the constraints of orthogonality and unit length[23]:

$$g_j = \arg\max_g \text{Var}(Xg) \quad \text{s.t.} \quad g^T g = 1, \quad \text{and} \quad \text{Cov}(Xg, Xg_l) = 0, \quad l = 1, 2, \ldots, j-1 \tag{24}$$

In PCR, these selected PCs are used as predictors in a regression model to forecast excess returns. The regression is defined as[24]:

$$r_{i,t+1} = Y_{i,t}\gamma_K + \epsilon_{i,t+1} \tag{25}$$

Where $r_{i,t+1}$ is the excess return of stock i at time $t+1$, $Y_{i,t}$ represents the K PCs for stock i at time t, γ_K is a $K \times 1$ vector of coefficients and $\epsilon_{i,t+1}$ is the error term. This approach reduces the risk of overfitting and avoids multicollinearity by limiting the number of predictors and ensuring they are uncorrelated. PCA inherently regularizes the regression by discarding PCs with lower variance, which are assumed to contain more noise than signal. By focusing on the principal components that capture the most variance, PCR effectively reduces dimensionality while retaining the most variation of the data, leading to more robust and efficient predictive models[25]. Detailed mathematical derivations and the process of PCA, including the diagonalization of covariance matrices and the singular value decomposition (SVD), are provided in Appendix A.12 and A.13.

[19] Coqueret and Guida 2020
[20] SVD and diagonalization are described in A.12
[21] Principal Components Analysis is described in detail in AppendixA.13
[22] Coqueret and Guida 2020
[23] Gu et al. 2019
[24] Gu et al. 2019
[25] Coqueret and Guida 2020

2.2.4 Partial Least Squares

The Partial Least Squares **(PLS)** regression is a latent factor regression model, where many predictors can be used as input, which then are reduced to a few latent factors, such that a regression can be run using the latent factors on the individual stock returns. The **PLS** algorithm implements dimension reduction by using the covariation between the features (stock characteristics) and the target variable (excess returns). For every feature (stock characteristic) j, a univariate return prediction coefficient is estimated using OLS. The estimated coefficient, ω_j represents the partial sensitivity of the returns to their corresponding jth feature (stock characteristic). Subsequently, all features are averaged to a single aggregate component (latent factor) with corresponding weights being proportional to ω_j. The largest weights are put on the strongest predicting features, while the lowest weights are placed on the weakest predicting features. Hence, the **PLS** implements dimension reduction with prediction of the stock returns as its primary goal. If more than one latent factors are formed, the target variable (excess returns) and the features (stock characteristics are orthogonalized with respect to all previously formed latent factors. The process is repeated on the orthoginalized dataset until the desired number of **PLS** latent factors have been formed. The orthogonalization ensures that the latent factors are not correlated (covarying) with each other[26] [27]. Appendix A.13 is referred to for an explanation with greater detail.

The **PLS** algorithm begins from the linear return model:

$$r_{i,t+1} = z_{i,t}^T \theta + \epsilon_{i,t+1} \tag{26}$$

Where $r_{i,t+1}$ and $\epsilon_{i,t+1}$ are the excess returns and the error terms of stock i at time period $t+1$, $z_{i,t}$ is a vector of P stock characteristics at for stock i at time period t. The vectorized version of the linear regression model $r_{i,t+1} = z_{i,t}^T \theta \epsilon_{i,t+1}$ is reorganized by stacking it:

$$R = Z\theta + E \tag{27}$$

Where R is the $NT \times 1$ vector of $r_{i,t+1}$, Z is the $NT \times P$ matrix of stacked features $z_{i,t}$ and E is a $NT \times 1$ vector of residuals. θ is the $P \times 1$ vector of coefficients. Like the PCR model, PLS reduces the dimensionality of Z by condensing the set of features (stock charactreristics) from dimension P to a smaller number of K linear combinations of features (latent factors). Therefore, the forecasting model becomes[28]:

$$R = (Z\Omega_K)\theta_K + \tilde{E} \tag{28}$$

where Ω_K is a $P \times K$ matrix with columns $\omega_1, \omega_2, ..., \omega_K$, where each ω_j is a set of weights that are linear combinations for constructing the jth latent factor. Therefore, $Z\Omega_K$ is the version of the original set of features, with reduced dimensions. θ_K is the $K \times 1$ dimension-reduced vector of coefficients. PLS

[26] Kelly and Priutt 2015
[27] Gu et al. 2019
[28] Gu et al. 2019

algorithm procedure consists of the following steps[29]:

The PLS objective forms K latent factors, consisting of linear combinations of Z that have maximal predictive association with the forecast target variable. The weights used to construct the jth PLS latent factor solves:

$$w_j = \arg\max_{w} \text{Cov}^2(R, Z\hat{\varphi}) \quad \text{s.t.} \quad \hat{\varphi}_j^T \hat{\varphi}_j = 1, \quad \text{Cov}(Z\hat{\varphi}, Z\hat{\varphi}_l) = 0, \quad l = 1, 2, \ldots, j-1 \tag{29}$$

Compared to PCR, PLS emphasizes a large variation between latent factors and the target variable, while the PCR is more precise with its estimates of the latent factors. The dimension reduction of the predictors acts as an inherent regularization step, retaining the most relevant variation with the excess returns. The maximization problem can be solved efficiently with a number of algorithms, such as the SIMPLS algorithm [30]. Appendix A.13 describes PLS more in detail, as the Three-Pass regression Filter method, developed by Kelly and Priutt 2015, where a special case of the Three-Pass regression Filter is equivalent to the PLS regression, which is used in this paper.

2.2.5 Observable Factor Models

To ensure comparability between observable factor models and the latent factor models used in this paper, the number of factors is varied from **K=1** to **K=6**. Following the methodology of Gu et al. 2020, each factor model specification is estimated sequentially by adding factors to the model. The first model, with **K=1**, is the **CAPM** model, which includes the market excess return ($Mkt - r_f$)[31]. The second model, with **K=2**, adds the Small Minus Big **(SMB)** factor, which measures the size effect by subtracting the returns of large-cap stocks from small-cap stocks, reflecting the typical out performance of small-cap firms[32]. For **K=3**, the model includes the High Minus Low **(HML)** factor, representing the difference in returns between value stocks (high book-to-market ratios) and growth stocks (low book-to-market ratios). The **HML** factor captures the value effect, where value stocks have historically outperformed growth stocks[33]. The fourth model, with **K=4**, adds the Up Minus Down **(UMD)** factor, which is the difference between stocks that have performed well and those that have performed poorly over a given time period. The **UMD** factor captures the momentum effect, where stocks with high past returns tend to continue performing well, while stocks with poor past returns tend to under perform [34]. The fifth model, with **K=5**, extends the Fama&French three-factor model ($Mkt - r_f$, SMB, HML) with the Conservative Minus Aggressive **(CMA)** and Robust Minus Weak **(RMW)** factors. The **CMA** factor represents the difference in returns between firms following conservative and aggressive investment strategies, capturing the investment effect where conservative firms tend to have higher average returns. The **RMW** factor measures

[29] Gu et al. 2019
[30] Jong 1993 is referenced to for further theoretical knowledge
[31] Sharpe 1964
[32] Fama and French 1993
[33] Fama and French 1993
[34] Carhart 1997

the profitability effect, capturing the differential returns between firms with high and low operating profitability. It is calculated as the average returns of firms with high operating profitability minus the average returns of firms with low operating profitability, using the ratio of operating income to book equity for a given period[35]. The sixth model, with **K=6**, adds the **UMD** momentum factor to the Fama%French five-factor model, further refining the analysis by including the momentum effect alongside the other factors. In summary, the sequence of observable factor models enables a comprehensive comparison with the latent factor models, ensuring that the number of factors in both models is controlled and comparable. **Table 3** illustrates the composition of the observable factors in the Fama&French models according to the number K, which we set from one to six to match the number of latent factors created by compressing the dimensions in the latent factor models.

Table 1: Composition of Observable Factors in Fama-French Models

K	Factors	Model Name
1	$Mkt - r_f$	CAPM
2	$Mkt - r_f$, SMB	CAPM + (Small-Minus-Big)
3	$Mkt - r_f$, SMB, HML	Fama-French 3-factor (FF3)
4	$Mkt - r_f$, SMB, HML, UMD	Carhart (FF4)
5	$Mkt - r_f$, SMB, HML, RMW, CMA	Fama-French 5-factor (FF5)
6	$Mkt - r_f$, SMB, HML, RMW, CMA, UMD	Fama-French 5-factor (FF5)+(Momentum)

The explanatory power of traditional asset pricing models and machine learning-latent factor models are analyzed by comparing their ability in explaining asset returns. By setting the number of factors K from one to six, a natural control over the factors is exerted in both traditional and latent factor models.

2.3 Performance Evaluation Metrics:

This section evaluates the out-of-sample performance of the models, focusing on how well they generalize to new, unseen data at both the individual stock and portfolio levels. Additionally, the portfolio performance is measured, using economic terms to assess the practical implications of the models.

Forecast Performance Evaluation Metrics: Two prediction evaluation metrics are used, inspired by the methodology from Kelly, Priutt, and Su 2018 and Gu et al. 2020.

The Total-R^2 out-of-sample (denoted as R^2_{Total})[36], represents the fraction of variance in excess returns explained by both the dynamic behavior of conditional factor loadings and the contemporaneous risk factors. In this paper, R^2_{Total} is calculated using returns at time $t+1$, with the lagged factor loadings. R^2_{Total} provides insight into how well the conemporaneous systematic risk factors of a model explain the realized returns of individual stocks:

[35]Fama and French 2015

[36]"test" in the definition of R^2_{Total} is an abbreviation for the test period

$$R^2_{Total} = 1 - \frac{\sum_{i,t\in test}(r_{i,t+1} - \hat{\beta}^T_{i,t}\hat{f}_{t+1})^2}{\sum_{i,t\in test} r^2_{i,t+1}} \tag{30}$$

The predictive-R^2 (denoted as R^2_{Pred})[37] measures the model's ability to forecast future excess returns, only on information available up to time t, focusing on the conditional expectation of realized returns at time $t+1$. Unlike the predictive-R^2 defined in Gu et al. 2020, which utilizes the time-series average of estimated factors up to time t (i.e., $\hat{\lambda}_t = \frac{1}{T}\sum_{t=1}^{T}\hat{f}_t$), this adopts a simplified approach by directly using the most recent estimated factors, $\hat{f}_t$, and lagged factor loadings, $\hat{\beta}_{i,t-1}$. This modification reduces the computational complexity by removing the need to compute time-series mean of factors and avoids modifying the model architecture. The predictive R^2 is defined as:

$$R^2_{Pred} = 1 - \frac{\sum_{i,t\in test}(r_{i,t+1} - \hat{\beta}^T_{i,t-1}\hat{f}_t)^2}{\sum_{i,t\in test} r^2_{i,t+1}} \tag{31}$$

Where $\hat{r}_{i,t+1} = \hat{\beta}^T_{i,t-1}\hat{f}_t$ are model's predicted return form stock i at time $t+1$ using lagged factor loadings and current facors. By using lagged factor loadings and current estimated factors, this metric focuses on the model's ability to forecast future returns based on the most recent information, without relying on on historical averages. This approach provides a more conservative evaluation of predictive capabilities and enhances responsiveness to recent data. Finally, in this setting, the **CA** models assume a no-arbitrage condition by not including intercept terms in the factor models. For consistency and comparability, the benchmark models are also evaluated under the same no-arbitrage assumption.

Portfolio Performance Evaluation Metrics: The portfolio performance metrics used are the **Sharpe Ratio**, the **Average Return**, the **Standard Deviance**, the **Value-At-Risk** and the **Turnover**.

Sharpe Ratio: The Sharpe Ratio is Defined as:

$$\textbf{Sharpe ratio} : S = \frac{\mu}{\sigma} \tag{32}$$

Where $r_{i,t}$ are the excess returns, and σ is the standard deviance of the excess returns. A higher Sharpe Ratio indicates a more favorable risk-adjusted return, suggesting that the portfolio achieves higher returns for a given level of risk.

The average return: is the arithmetic mean of the excess returns over the observed time periods. It is defined as:

$$\bar{h} = \bar{\mu} = \mathbb{E}[r] = \frac{1}{T}\sum_{i,t=1}^{T} r_{i,t} \tag{33}$$

[37]"test" in the definition of R^2_{Pred} is an abbreviation for the test period

Where the average returns are calculated as the arithmetic average of the excess return, $r_{i,t}$ for stock i at time t for all time periods T. The average return provides a measure of the portfolio's performance over time.

Volatility: measures the dispersion of portfolio returns around the mean, indicating the level of risk. It is calculated as the standard deviation of the excess returns:

$$\sigma_{i,t} = V[r_{i,t}] \approx \sqrt{\frac{1}{T-1}\sum_{i,t=1}^{T}(r_{i,t}-\mu)^2} \tag{34}$$

Where μ is the mean of the return of stock i for all time observed time periods t until T. A higher volatility implies greater uncertainty and risk associated with the portfolio returns.

The variance can be calculated as the standard errors squared :

$$\sigma_{i,t}^2 = Var(r_{i,t}) = \frac{1}{T-1}\sum_{i,t=1}^{T}(r_{i,t}-\mu)^2 \tag{35}$$

Value At Risk: quantifies the potential loss in portfolio value over a specified time period at a give confidence level. It is defined as:

$$VaR_{\alpha}(r_{i,r}) = F_r(1-\alpha) \tag{36}$$

Where F_r is the empirical **cdf** (cumulative distribution function) of the returns and α is the confidence level, which in this paper is set to 95%. $VaR_{0.95}$ represents the maximum expected loss with 95% confidence, meaning that there is a 5% probability that losses will exceed this value.

Turnover: measures the frequency and magnitude of changes in the portfolio's composition, reflecting trading activity. It is defined as:

$$\textbf{Turnover} = \frac{1}{T-1}\sum_{t=2}^{T}\sum_{n=1}^{N}|\omega_{t,n}-\omega_{t-1,n}| \tag{37}$$

Where $\omega_{t,n}$ and $\omega_{t-1,n}$ are the weights in the portfolio at time t and time $t-1$ respectively. The Turnover provides an indication of the trading intensity of the portfolio. High turnover rates suggest frequent and significant rebalancing, which can lead to increased transaction costs and potential tax implications. In contrast, low turnover rates indicate a more stable portfolio composition.

3 Empirical analysis

3.1 data

The dataset used in this paper consists of monthly observations of stock returns from the CRSP database, covering stocks listed the U.S. stock exchanges: NYSE, AMEX, and NASDAQ. To align with asset pricing theory, the risk-free rate, represented by the U.S. Treasury bill interest rate, is subtracted from the monthly stock returns to calculate the excess returns. These excess returns serve as the primary variable for out-of-sample predictions. The dataset is a panel of monthly observations, encompassing time-series data for approximately 4120 stocks per month. However, it is unbalanced, as the number of stocks varies each month, leading to changes in both the dimension and the variability of the returns over time. The independent variables in the models consist of 94 stock characteristics are sourced from the Compustat and CRSP databases[38]. Of these, 61 are updated annually, 13 quarterly, and 20 monthly.

To avoid future bias, the monthly stock characteristics are lagged 1 month, the quarterly stock characteristics are lagged with 4 months, the annual stock characteristics are lagged by 6 months. To address the issue of highly skewed characteristic distributions, the stock characteristics are rank-normalized to the interval [-1, 1]. This transformation mitigates the influence of outliers and ensures consistent scaling across characteristics. While it does not directly capture non-linear relationships, it reduces the need for assumptions about the underlying distributions by making the data distribution-free and more suitable for non-parametric methods. After accounting for missing data, the dataset includes approximately 15000 unique stocks, with an average of 4120 stocks per month over the entire date range. The period of interest for this paper's analysis spans from November 1980 to December 2020.

3.2 Out-of-sample performance evaluation

This section evaluates the **Conditional Autoencoder (CA)** model against the benchmark models (**IPCA**, **PCR**, **PLS** and **FF**) for $K = 1$ to $K = 6$ factors. **Table 2** presents the Predictive-R^2 (R^2_{Pred}) and the Total-R^2_{Total} for the out-of-sample predictions of individual excess returns, r_t.

Total-R^2 for r_{t+1}: The **CA** models, specifically $\boldsymbol{CA_0}$, $\boldsymbol{CA_1}$, and $\boldsymbol{CA_3}$, exhibit the highest R^2_{Total} of 15% for the 3-, 4-, and 6-factor specifications. For the 3-factor specification, the $\boldsymbol{CA_0}$ model achieves the highest R^2_{Total} of 15%. In the 4-factor specification, $\boldsymbol{CA_1}$ performs best with a Total-R^2 of 15%. Finally, for the 6-factor specification, $\boldsymbol{CA_0}$, $\boldsymbol{CA_1}$, and $\boldsymbol{CA_3}$ all achieve the highest R^2_{Total} of 15%.

In contrast, the **IPCA**, **PCR**, and **PLS** models demonstrate negligible performance, with slightly positive or negative R^2_{Total}. The poor performance of **IPCA** and **PCR** can likely be attributed to the absence of regularization, which likely reduces their ability to select factors that effectively explain the variation in returns.

[38]For inspection of the stock characteristics, the reader is referred to table A.6 of the Appendix in Gu et al. 2019

The **FF** models achieve the highest R^2_{Total} among the benchmark models of approximately 12.3% to 12.6% for 2 to 6 factors respectively.

In summary, the $\boldsymbol{CA_0}$, $\boldsymbol{CA_1}$, and $\boldsymbol{CA_3}$ models, except, deliver the highest R^2_{Total}, particularly, the 3, 5, and 6 factors. The **IPCA**, **PCR**, and **PLS** models perform the worst, producing negligible results, while the observable factor models perform comparably well, though slightly below the **CA** models.

Predictive-R^2 for r_{t+1}: For R^2_{Pred}, as shown in **Table 2**, all models tend to exhibit lower values by default, as these metrics pertain to forecasting future returns, $\mathbb{E}[r_{t+1}|I_{t+1}]$. The **CA** models consistently demonstrate positive R^2_{Pred} across all factor specifications. Specifically, $\boldsymbol{CA_1}$ and $\boldsymbol{CA_2}$ exhibit the highest R^2_{Pred} values of 4% for the 2-factor and 6-factor specifications, respectively. Additionally, the R^2_{Pred} generally increases as the number of factors increases, though no clear relationship between R^2_{Pred} and the number of latent factors is apparent. Interestingly, the number of hidden layers appears to affect performance, with R^2_{Pred} increasing from 0 to 1 hidden layer and decreasing when additional layers are added, particularly from 2 to 3 layers. The **IPCA**, **PCR**, and **PLS** models consistently underperform, exhibiting slightly negative R^2_{Pred} across all factor specifications, indicating weak predictive capability on unseen data. This overall weak performance is likely attributed to absence of regularization or that the models may suffer from an inherently poor ability to capture and predict future returns, a problem that similarly affects the R^2_{Total}. The observable factor models perform poorly, with negative R^2_{Pred} values for the 4- to 6-factor specifications, approximately -0.4%, -0.01%, and -0.50%, respectively. In contrast, the simpler 1- to 3-factor models show slightly positive R^2_{Pred} values of 0.49%, 0.06%, and 0.05%, corresponding to the **mkt**, **SMB**, and **HML** factor specifications. Overall, the **CA** models, particularly $\boldsymbol{CA_1}$ and $\boldsymbol{CA_2}$, offer the most promising predictive capability for excess returns, reinforcing their potential utility in modeling complex financial datasets. All benchmark models have slightly negative R^2_{Pred}, indicating poor predictive performance. While no clear relationship is observed between R^2_{Pred} and the number of factors, the number of hidden layers does seem to impact performance, highlighting the importance of model architecture in prediction tasks.

The characteristics-managed portfolios, x_{t+1} are formed based on individual predicted stock returns using ex-ante stock characteristics of individual stocks as weighting constituents for each portfolio, which is dynamically rebalanced[39]. **Table 3** presents the Predictive-R^2 (R^2_{Pred}) and the Total (R^2_{Total}) for the out-of-sample predictions at portfolio level for x_{t+1}.

Total-R^2 for x_{t+1}: as shown in **Table 3**, the $\boldsymbol{CA_0}$ model, achieves a R^2_{Total} of 12.04% and 19.52% for the 1-factor and 6-factor specifications, respectively, peaking at 20.30% for the 3-factor specification. The $\boldsymbol{CA_1}$ model, reaches its highest R^2_{Total} of 24.76% with the 6-factor specification, while its lowest is 3.48% for the 2-factor specification, showing that this model benefits significantly from more factors. For the $\boldsymbol{CA_2}$ model, the highest R^2_{Total} is 20.29% for the 3-factor specification, but its performance varies, with a low of 2.38% for the 1-factor specification. The $\boldsymbol{CA_3}$ model, with three hidden layers, achieves

[39]Gu et al. 2020

Table 2: Out-of-sample Predictive Performance for r_{t+1}

	1	2	3	4	5	6
FF						
R^2_{Pred}	0.49%	0.06%	0.05%	-0.40%	-0.01%	-0.50%
R^2_{Total}	10.55%	12.31%	12.26%	12.64%	12.30%	12.67%
IPCA						
R^2_{Pred}	-0.18%	-0.17%	-0.11%	-0.06%	-0.04%	-0.03%
R^2_{Total}	-0.16%	-0.15%	-0.04%	0.08%	0.03%	0.03%
PCR						
R^2_{Pred}	-0.07%	-0.04%	-0.07%	-0.03%	-0.03%	-0.03%
R^2_{Total}	-0.07%	-0.05%	-0.08%	-0.04%	-0.04%	-0.04%
PLS						
R^2_{Pred}	-0.04%	-0.00%	-0.04%	-0.04%	-0.05%	-0.04%
R^2_{Total}	-0.03%	0.01%	-0.02%	-0.02%	-0.04%	0.01%
$\boldsymbol{CA_0}$						
R^2_{Pred}	0.00%	1.00%	2.00%	3.00%	2.00%	2.00%
R^2_{Total}	12.00%	12.00%	15.00%	12.00%	15.00%	15.00%
$\boldsymbol{CA_1}$						
R^2_{Pred}	1.00%	4.00%	3.00%	3.00%	0.00%	3.00%
R^2_{Total}	12.00%	12.00%	12.00%	15.00%	14.00%	15.00%
$\boldsymbol{CA_2}$						
R^2_{Pred}	2.00%	1.00%	1.00%	1.00%	2.00%	4.00%
R^2_{Total}	14.00%	11.00%	13.00%	13.00%	13.00%	14.00%
$\boldsymbol{CA_3}$						
R^2_{Pred}	0.00%	1.00%	1.00%	3.00%	3.00%	1.00%
$R^2_{Total;OOS}$	13.00%	13.00%	14.00%	12.00%	12.00%	15.00%

Table 3: Out-of-sample Predictive Performance for x_{t+1}

	1	2	3	4	5	6
FF						
R^2_{Pred}	0.10%	0.07%	0.07%	0.10%	0.16%	0.17%
R^2_{Total}	0.10%	0.07%	0.07%	0.10%	0.16%	0.17%
IPCA						
R^2_{Pred}	0.03%	0.02%	-0.08%	-0.06%	-0.04%	-0.02%
$R^2_{Total;OOS}$	-0.02%	-0.03%	-0.01%	0.19%	0.25%	0.24%
PCR						
R^2_{Pred}	0.09%	0.08%	0.07%	0.09%	0.08%	0.08%
R^2_{Total}	0.35%	0.35%	0.33%	0.34%	0.34%	0.34%
PLS						
R^2_{Pred}	-0.9%	-0.08%	-0.09%	-0.10%	-0.12%	-0.12%
R^2_{Total}	0.05%	0.08%	0.11%	0.13%	0.16%	0.20%
$\boldsymbol{CA_0}$						
R^2_{Pred}	-0.48%	-0.31%	-0.89%	-2.87%	-0.85%	-1.14%
R^2_{Total}	12.04%	13.75%	20.30%	5.34%	17.64%	19.52%
$\boldsymbol{CA_1}$						
R^2_{Pred}	-0.28%	-0.39%	-11.17%	-1.73%	0.73%	0.96%
R^2_{Total}	15.63%	3.48%	15.14%	18.21%	19.66%	24.76%
$\boldsymbol{CA_2}$						
R^2_{Pred}	0.91%	-1.08%	1.90%	-2.07%	0.21%	-0.11%
R^2_{Total}	2.38%	18.55%	20.29%	15.69%	8.80%	24.56%
$\boldsymbol{CA_3}$						
R^2_{Pred}	-0.60%	0.06%	0.46%	-3.27%	-0.51%	-0.33%
R^2_{Total}	10.41%	7.94%	12.39%	13.17%	15.67%	12.98%

its best R^2_{Total} of 15.67% for the 5-factor specification, while its lowest is 7.94% for the 2-factor case. This model shows more consistency but does not outperform the simpler models. Across the **CA** models, higher factor specifications generally improve performance, with the $\boldsymbol{CA_1}$ model performing best at 6 factors. However, models like $\boldsymbol{CA_2}$ and $\boldsymbol{CA_3}$ exhibit greater variability across factor configurations, suggesting that added complexity does not always result in better outcomes.

The **IPCA** model shows negative R^2_{Total} for the 1- and 2-factor specifications but improves with more factors, peaking at 0.25% for the 5-factor specification. Its performance remains relatively low overall. The **PCR** model performs consistently across all factor levels, with R^2_{Total} ranging from 0.33% to 0.35%. It shows stable, though modest, predictive power across specifications. The **PLS** model starts at 0.05% for the 1-factor specification and steadily improves, reaching 0.20% for the 6-factor case. Although its performance increases with more factors, the R^2_{Total} remains moderate. Across the **IPCA**, **PCR**, and **PLS**, the R^2_{Total} is relatively low, with **PCR** offering the most consistent performance. The **FF** models show low R^2_{Total} across all factor specifications, starting at 0.10% for both the 1- and 4-factor specifications. The R^2_{Total} increases slightly with more factors, peaking at 0.17% for the 6-factor specification. However, the overall performance remains consistently low across the different factor configurations. The R^2_{Total} for the **FF** model remains modest, similar to the **IPCA**, **PCR**, and **PLS** models, with no significant improvements as the number of factors increases.

The **CA** models consistently outperform the benchmark models in R^2_{Total}. The $\boldsymbol{CA_1}$ model performs best, reaching 24.76% with the 6-factor specification, while other **CA** models also show strong results, especially with more factors. However, increasing complexity does not always improve performance, as seen with $\boldsymbol{CA_2}$ and $\boldsymbol{CA_3}$.

In contrast, the benchmark models (**IPCA**, **PCR**, **PLS** , and **FF**) exhibit much lower R^2_{Total} values, with the **PCR** model being the most stable but still far behind the**CA** models. Overall, the **CA** models lead in explanatory power across all factor specifications.

Predictive-R^2 for x_{t+1}: In **Figure 3**, the $\boldsymbol{CA_0}$ model shows negative R^2_{Pred} values across all factor specifications, with the lowest value of -2.87% for the 4-factor specification. The best result is -0.31% for the 2-factor specification, indicating poor out-of-sample predictive performance. For the $\boldsymbol{CA_1}$ model, R^2_{Pred} also starts negative, reaching a low of -11.17% for the 3-factor specification. However, its performance improves with higher factors, peaking at 0.96% for the 6-factor specification. The $\boldsymbol{CA_2}$ model achieves positive R^2_{Pred} for some factor specifications, with the best result being 1.90% for the 3-factor case. However, it also shows variability, with negative values for certain specifications, such as -2.07% for the 4-factor case. The $\boldsymbol{CA_3}$ model similarly displays inconsistent predictive performance, with the highest R^2_{Pred} being 0.46% for the 3-factor specification, and the lowest value of -3.27% for the 4-factor case. Across the **CA** models, R^2_{Pred} values are generally low and negative for most factor specifications, though some improvement is seen with more factors, particularly in $\boldsymbol{CA_1}$ and $\boldsymbol{CA_2}$. The **IPCA** model shows weak predictive performance across all factor specifications, with negative R^2_{Pred} for most configurations. The worst result is -0.08% for the 3-factor specification, and its highest value is 0.03% for the

1-factor case. The **PCR** model performs slightly better, with positive R^2_{Pred} across all specifications. It starts at 0.09% for the 1-factor setup and remains fairly consistent, reaching a maximum of 0.09% for the 4-factor case. However, the overall improvement with more factors is marginal. The **PLS** model exhibits negative R^2_{Pred} values across all factor levels, starting at -0.9% for the 1-factor case and slightly improving to -0.12% for the 6-factor specification. This suggests poor predictive performance throughout. Across the **IPCA**, **PCR**, and **PLS** models, R^2_{Pred} remains generally low, with **PCR** showing the most stable, though modest, positive values. Both **IPCA** and **PLS** struggle to achieve positive R^2_{Pred}, indicating weak out-of-sample performance. The Observable factor models **(FF)** models exhibit consistently low R^2_{Pred} across all factor specifications. It starts at 0.10% for the 1-factor case and remains relatively flat, peaking at 0.17% for the 6-factor specification. The changes in R^2_{Pred} are minimal as the number of factors increases, showing limited improvement in out-of-sample performance.

All models show low R^2_{Pred} values. The **CA** models have relatively high R^2_{Pred}, with modest improvements in $\boldsymbol{CA_1}$ and CA_2 as more factors are added. Benchmark models (**IPCA**, **PCR**, and **PLS**) perform poorly, with **PCR** showing slight, stable positive values, while **IPCA** and **PLS** struggle with negative R^2_{Pred}. The observable factor models **(FF)** remain consistently low, peaking at just 0.17%.

3.3 Portfolio forecast performance

The R^2 value measures the model's ability to explain the variation in returns, but its implications for economic performance, such as mean-variance efficiency, are more nuanced. Total-R^2 provides insight into how well systematic risk factors explain realized returns, while Predictive-R^2 evaluates the model's ability to forecast future returns. However, mean-variance efficiency, often measured by metrics like Sharpe Ratios, requires an assessment beyond the explained variance, as it captures the risk-return trade-off. Thus, R^2 alone does not fully capture the economic performance in terms of risk-adjusted returns.

To assess the economic performance of the out-of-sample predictions, a portfolio-based evaluation offers a more meaningful approach. As suggested by Gu et al. 2020, forward returns are used to construct long-short portfolios. For the Long-Short portfolios, a long position is taken in the top decile of the excess returns, while a short position is taken in the bottom decile. Two long-short portfolios are constructed: one weighted by the market capitalization (value-weighted) and another where each stock is assigned equal weight (equal-weighted). In the value-weighted portfolio, stocks are weighted according to their market size, while in the equal-weighted portfolio, each stock receives the same weight. Historically, equal-weighted portfolios are known to be challenging to outperform. The methodology for calculating the long-short portfolio returns follows Gu et al. 2019, where the portfolio returns at time $t+1$ are computed using the predicted returns for period $t+1$ and the portfolio weights from period t:

$$\hat{r}^p_{t+1} = \sum_{i=1}^{n} \omega^p_{i,t} \times \hat{r}_{i,t+1} \tag{38}$$

Where $\omega^p_{i,t}$ are the weights for stock i at time period t, $\hat{r}_{i,t+1}$ are the predicted returns for stock i at time

period $t+1$ and $\hat{r}^p_{t+1}$ are the computed portfolio returns. The calculation of the weights for the equal- and value-weighed portfolio returns are described in more detail in Appendix A.16.

Table 4 presents the portfolio forecasting performance of both equal-weighted and value-weighted long-short portfolios for the models **CA** and its benchmark models (**IPCA**, **PCR**, **PLS**, and **FF**). The left-hand side of the **table 4** displays the performance metrics for the equal-weighted portfolios, while the right-hand side corresponds to the value-weighted portfolios. Each sub-table lists the number of latent factors (**K**) as rows and the performance evaluation metrics—**Sharpe Ratio (SR)**, **Average Return (AvgRet)**, **Standard Deviation (std)**, **Value at Risk (VaR)**, and **Turnover (Turn)**—as columns.

In **Table 4**, the **Conditional Autoencoder (CA) models** consistently outperforms the benchmark models across all key evaluation metrics in both equal-weighted and value-weighted portfolios. Among the **CA** models, the $\boldsymbol{CA_2}$ model stands out, achieving the highest **Sharpe Ratio** of **1.66** in the equal-weighted portfolio with a 6-factor specification. This model also delivers an **average return** of **4%**, while maintaining a **standard deviation** of **0.07**, indicating strong risk-adjusted returns. In the value-weighted portfolio, the $\boldsymbol{CA_2}$ model continues to perform exceptionally well, attaining a **Sharpe Ratio** of **1.47** and an average return of **3%**.

The $\boldsymbol{CA_1}$ also exhibits strong performance. In the equal-weighted portfolio, it achieves its highest **Sharpe Ratio** of **1.55** with 2 latent factors, along with an average return of **3%** and a standard deviation of **0.07**. This indicates that even with fewer factors, the $\boldsymbol{CA_1}$ model can deliver high risk-adjusted returns. In the value-weighted portfolio, the $\boldsymbol{CA_1}$ model reaches a **Sharpe Ratio** of **1.50** with 2 factors, matching its equal-weighted counterpart in terms of risk-adjusted performance. The $\boldsymbol{CA_3}$ model also contributes positively to portfolio performance. In the equal-weighted portfolio, it attains a **Sharpe Ratio** of **1.47** with 5 factors, along with an average return of **3%** and a standard deviation of **0.07**. In the value-weighted portfolio, the $\boldsymbol{CA_3}$ model achieves a **Sharpe Ratio** of **1.14** with 5 factors, indicating robust performance, though slightly lower than its equal-weighted counterpart. For the $\boldsymbol{CA_0}$, the best performance in the equal-weighted portfolio occurs with 4 factors, yielding a **Sharpe Ratio** of **1.40**, an average return of **3%**, a VaR of **-0.04**, and a turnover of **0.61**. In the value-weighted portfolio, its highest **Sharpe Ratio** is **1.37** with 4 factors, similar to the equal-weighted case. Across all **CA models**, the **Value at Risk (VaR)** remains consistently low, typically ranging from **-0.04** to **-0.08**, indicating controlled downside risk. The **turnover** for the **CA** models is efficient, varying between **0.54** and **0.76** across different factor specifications, suggesting reasonable trading activity.

In contrast, the benchmark models underperform compared to the **CA** models. The **IPCA model** shows limited improvement with more latent factors; its **Sharpe Ratio** peaks at only **0.97** with 5 factors in the equal-weighted portfolio and reaches **0.38** for the value-weighted portfolio. Additionally, the **IPCA** model experiences higher VaR values, reaching up to **-0.17** and **-0.18** with fewer factors for both the equal- and the value-weighted portfolio, indicating higher downside risk.

The **PCR model** exhibits modest performance, with its maximum **Sharpe Ratio** of **0.81** and **0.31** achieved in the equal- and value-weighted portfolios respectively. However, its average returns are lower, around **1%** and **0.31%** respectively for the equal- and value-weighed portfolios respectively. It maintains

Table 4: Equal- and value-weighted long-short portfolios for Autoencoder models

Equal Weighted L-S portfolio					
K	**SR**	**AvgRet**	**std**	**VaR**	**Turn**
CA_0					
1	0.37	1.0%	0.05	-0.06	0.67
2	0.82	2.0%	0.07	-0.07	0.63
3	0.84	1.0%	0.06	-0.05	0.65
4	1.40	3.0%	0.07	-0.04	0.61
5	1.10	2.0%	0.06	-0.06	0.64
6	1.30	2.0%	0.06	-0.05	0.60
CA_1					
1	0.41	1.0%	0.07	-0.10	0.67
2	1.55	3.0%	0.07	-0.05	0.56
3	1.33	3.0%	0.07	-0.06	0.61
4	1.15	2.0%	0.07	-0.07	0.59
5	1.23	2.0%	0.06	-0.05	0.60
6	1.53	3.0%	0.07	-0.06	0.56
CA_2					
1	0.95	2.0%	0.07	-0.06	0.62
2	0.93	2.0%	0.06	-0.06	0.65
3	1.05	2.0%	0.07	-0.06	0.61
4	1.46	2.0%	0.06	-0.04	0.56
5	1.27	2.0%	0.07	-0.05	0.61
6	1.66	4.0%	0.07	-0.06	0.54
CA_3					
1	0.40	1.0%	0.05	-0.08	0.64
2	0.81	1.0%	0.05	-0.05	0.64
3	0.74	1.0%	0.05	-0.05	0.65
4	1.16	2.0%	0.07	-0.06	0.58
5	1.47	3.0%	0.07	-0.07	0.57
6	0.91	2.0%	0.06	-0.06	0.63
$IPCA$					
1	-0.19	-1.0%	0.11	-0.16	0.65
2	-0.26	-1.0%	0.11	-0.17	0.66
3	0.32	1.0%	0.07	-0.09	0.65
4	0.83	1.0%	0.05	-0.07	0.64
5	0.97	1.0%	0.04	-0.05	0.66
6	0.94	1.0%	0.04	-0.05	0.66
PCR					
1	-0.03	-0.08%	0.08	-0.11	0.63
2	0.38	1.0%	0.05	-0.08	0.67
3	0.00	1.0%	0.04	-0.06	0.66
4	0.61	1.0%	0.04	-0.04	0.58
5	0.81	1.0%	0.03	-0.05	0.60
6	0.81	1.0%	0.03	-0.05	0.61
PLS					
1	0.67	1.0%	0.04	-0.06	0.61
2	0.99	1.0%	0.04	-0.05	0.60
3	0.85	1.0%	0.04	-0.05	0.61
4	0.86	1.0%	0.04	-0.05	0.61
5	0.91	1.0%	0.04	-0.05	0.60
6	0.96	1.0%	0.04	-0.05	0.59
FF					
1	-0.07	-0.03%	0.01	-0.02	0.71
2	-0.07	-0.03%	0.01	-0.02	0.71
3	-0.08	-0.03%	0.01	-0.02	0.70
4	-0.06	-0.02%	0.01	-0.02	0.70
5	-0.07	-0.03%	0.01	-0.02	0.70
6	-0.08	-0.03%	0.01	-0.02	0.71

Value Weighted L-S portfolio					
K	**SR**	**AvgRet**	**std**	**VaR**	**Turn**
CA_0					
1	0.21	0.0%	0.05	-0.07	0.66
2	0.88	2.0%	0.07	-0.08	0.67
3	0.64	1.0%	0.06	-0.06	0.70
4	1.37	3.0%	0.07	-0.05	0.63
5	0.71	1.0%	0.07	-0.07	0.68
6	1.02	2.0%	0.06	-0.06	0.68
CA_1					
1	0.47	1.0%	0.08	-0.11	0.66
2	1.50	3.0%	0.07	-0.06	0.62
3	1.00	2.0%	0.07	-0.06	0.66
4	0.83	2.0%	0.07	-0.07	0.63
5	0.87	1.0%	0.06	-0.06	0.68
6	1.40	3.0%	0.07	-0.05	0.60
CA_2					
1	0.72	2.0%	0.08	-0.08	0.62
2	0.76	1.0%	0.06	-0.07	0.70
3	0.85	2.0%	0.07	-0.07	0.66
4	0.76	1.0%	0.06	-0.06	0.69
5	1.21	2.0%	0.06	-0.05	0.64
6	1.47	3.0%	0.07	-0.05	0.57
CA_3					
1	0.43	1.0%	0.05	-0.08	0.76
2	0.62	1.0%	0.06	-0.08	0.70
3	0.52	1.0%	0.05	-0.06	0.64
4	1.07	2.0%	0.08	-0.06	0.65
5	1.14	3.0%	0.08	-0.08	0.59
6	0.76	1.0%	0.06	-0.05	0.67
$IPCA$					
1	-0.19	-1.0%	0.11	-0.19	0.69
2	-0.40	-1.0%	0.10	-0.18	0.67
3	0.11	2.0%	0.08	-0.11	0.70
4	0.38	1.0%	0.05	-0.07	0.69
5	0.32	5.0%	0.05	-0.08	0.73
6	0.36	1.0%	0.05	-0.08	0.71
PCR					
1	-0.10	-0.22%	0.08	-0.13	0.66
2	0.20	0.30%	0.05	-0.07	0.67
3	-0.27	-0.30%	0.05	-0.08	0.66
4	0.09	0.10%	0.04	-0.06	0.65
5	0.27	0.31%	0.04	-0.06	0.67
6	0.31	0.35%	0.04	-0.05	0.68
PLS					
1	0.21	0.26%	0.04	-0.07	0.66
2	0.33	0.42%	0.04	-0.07	0.72
3	0.31	0.43%	0.05	-0.07	0.68
4	0.28	0.36%	0.045	-0.07	0.71
5	0.23	0.30%	0.05	-0.07	0.68
6	0.29	0.37%	0.05	-0.08	0.71
FF					
1	0.10	-0.07%	0.02	-0.04	0.66
2	0.10	-0.07%	0.02	-0.04	0.66
3	-0.09	-0.07%	0.02	-0.04	0.66
4	-0.09	-0.07%	0.02	-0.04	0.66
5	-0.10	-0.07%	0.02	-0.04	0.66
6	-0.09	-0.06%	0.02	-0.04	0.66

a higher turnover, up to **0.67** and **0.68**, for the equal- and value-weighed portfolios respectively, implying frequent rebalancing and therefore trading. The **PLS model** provides a relatively stable performance, with **Sharpe Ratio** ranging from **0.67** to **0.96** and **0.21** to **0.33** for the equal- and value-weighted portfolios respectively. Its average returns are consistently between **1%** and **0.31%**, as well as **0.26%** and **0.42%** for the equal- and value-weighed portfolios respectively. It maintains low VaR values of **-0.05** to **-0.06** and **-0.07** to **-0.08** for the equal- and value-weighted portfolios. However, it does not reach the higher risk-adjusted returns achieved by the **CA** models.

The **Observable Factor Model (FF)** records negative **Sharpe Ratio** and average returns close to zero for the equal-weighted portfolios. The **Sharpe Ratio** for the value-weighted portfolios are slightly positive for the 1 and 2 factor specifications but remain low at **0.10**, indicating poor performance. The average returns for both equal- and value-weighted portfolios are minimal, at **-0.03%** and **-0.07%**, respectively. Its **Value-At-Risk (VaR)** remains low at **-0.02** and **-0.04**, reflecting the low volatility associated with its minimal returns. In summary, the **CA models**, particularly $\boldsymbol{CA_1}$, $\boldsymbol{CA_2}$, and $\boldsymbol{CA_3}$, lead in all performance metrics. They achieve high Sharpe Ratios, strong average returns, and maintain low **VaR** with moderate turnover. The superior performance of the **CA** models underscores their effectiveness in capturing latent factors that contribute to enhanced portfolio forecasting, outperforming traditional benchmarks like **IPCA**, **PCR**, **PLS**, and **FF**.

3.4 Risk compensation versus anomalies

The **CA** model inherently assumes no-arbitrage and, as a result, does not include an intercept term (alpha). The intercept term in asset pricing models is interpreted as uncompensated returns or omitted risk factors that the model cannot account for. The no-arbitrage assumption therefore implies that any return predictability is only due to risk exposures and not due to mispricing or omitted risk factors.

To investigate whether there is any validity of the no-arbitrage condition for the **CA** models, intercept terms are calculated from the prediction errors of the models. Because the individual stock returns tend to have high idiosyncratic volatility, portfolio returns are used instead from which the errors of the prediction errors are calculated. The specific process that is undertaken, follows these steps:

1. 94 portfolios are formed based on the 94 different asset characteristics in the dataset. The portfolio returns are denoted by x_{t+1}[40].
2. out-of-sample predictions of x_{t+1} are made on the test set
3. The unconditional prediction errors of the expected portfolio returns are calculated by the following definigtion:

$$\alpha_i : \mathbb{E}(u_{i,t+1}) = \mathbb{E}((Z_t^T Z_t)^{-1} Z_t r_{t+1}) - \mathbb{E}((Z_t^T Z_t)^{-1} Z_t \hat{r}_{t+1}) = \mathbb{E}(x_{i,t+1}) - \mathbb{E}(\hat{x}_{i,t+1}) \tag{39}$$

[40]The calculation of the characteristics-managed portfolios is described in Section 2.2.1

Where $E(x_{i,t+1})$ is the expected excess return of portfolio i at time $t+1$. $\mathbb{E}(\hat{x}_{i,t+1})$ are the expected predicted excess returns of portfolio i at time $t+1$. The expectations are equivalent to the time-series average.

4. The significance of these alphas is tested using a t-test. Following Harvey and Zhu 2016, the critical value is set to 3.0, to ensure a higher threshold for significance, especially when dealing with systematic risk factors.

The **CA** models with 5 latent factors are compared against **PCR**, **PLS**, and **FF** models. When evaluating these models, the intercept terms (alphas) of the prediction errors for 94 characteristic-managed portfolios are analyzed. If these alphas are found to be small or statistically insignificant, it suggests that the CA models effectively capture the variance in portfolio returns through their latent factors, obeying to the no-arbitrage condition. in Contrast, significant alphas would indicate that the models may be missing relevant risk factors or are capturing market anomalies.

The **CA** models with 5 latent factors are compared against the **PCR**, **PLS** and the **FF** models. If the intercepts terms of the prediction errors of the 94 portfolio returns either are small or statistically insignificant, it suggests that the models effectively capture the variance of the returns through its factors, obeying the no-arbitrage condition. If otherwise, significant alphas would indicate that the models may be missing relevant risk factors or are capturing market anomalies.

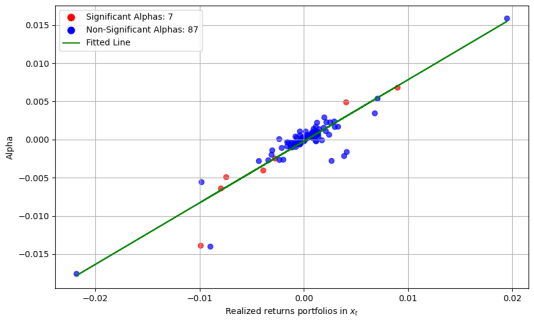
Figure 3 presents scatter plots of the estimated pricing errors (alphas) for each characteristic-managed portfolio, α_i, plotted on the y-axis, and the actual expected characteristic-managed portfolio excess returns, $\mathbb{E}(x_{i,t+1})$, on the x-axis. The subplots display results for the various factor models using five factors: the **CA** models, **PCR**, **PLS**, and **FF** (Fama-French). Alphas for the **IPCA** model are omitted as they require a different significance test than the t-test applied[41][42]. In the $\boldsymbol{CA_0}$ (panel **(a)**) model, there are 7 significant alphas, with 87 alphas found to be insignificant. In contrast, $\boldsymbol{CA_1}$ (panel **(b)**) has 3 significant alphas and 91 that are insignificant, showing a tighter distribution of alphas around zero. $\boldsymbol{CA_2}$ (panel **(c)**) shows 11 significant alphas and 83 insignificant alphas, indicating a slightly larger dispersion. $\boldsymbol{CA_3}$ (panel **(d)** reveals 19 significant alphas with 75 insignificant alphas, with more extreme values compared to the other autoencoder models. The **FF** (panel **(e)**) model produces 8 significant alphas and 86 insignificant ones, and both the **PCR** (panel **(f)**) and **PLS** (panel **(g)**) models show 7 and 5 significant alphas, respectively, with most of their alphas being insignificant. For the **CA** models, the alphas are generally centered around zero, with a positive slope, indicating that portfolios with higher realized excess returns tend to have slightly positive pricing errors. The most extreme outliers for expected realized returns range from approximately -0.02 to 0.02, while the most extreme alphas lie between -0.015 and 0.015. Given that most alphas are insignificant and their magnitudes are small, this suggests that the **CA** models are pricing returns well, with small errors. The complexity of the **CA** models, particularly the number of hidden layers, appears to influence the number of significant alphas. In this case, $\boldsymbol{CA_1}$ seems to achieve the

[41] Kelly, Priutt, and Su 2018
[42] Kelly, Priutt, and Su 2017

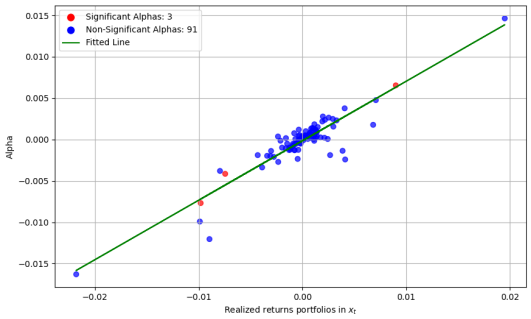
best balance between model complexity and pricing accuracy, as indicated by its relatively few significant alphas and the overall clustering of alphas near zero. The benchmark models (**PCR**, **PLS** and **FF**) show similar trends, with alphas and realized excess returns tightly centered around zero. However, the **FF** model has the narrowest span of alphas and realized returns, suggesting that it produces the smallest pricing errors. The **PLS** model, on the other hand, has the fewest significant alphas, indicating potentially more accurate pricing of returns compared to the other models. Overall, both the **CA** and benchmark models exhibit accurate pricing, with small and consistent pricing errors relative to the realized excess returns.

Table 5 presents statistics for alphas of characteristic-managed portfolios across various models, including **CA** models and benchmark models (**PCR**, **PLS** and **FF**). These statistics help evaluate how well each model captures or misprices returns.

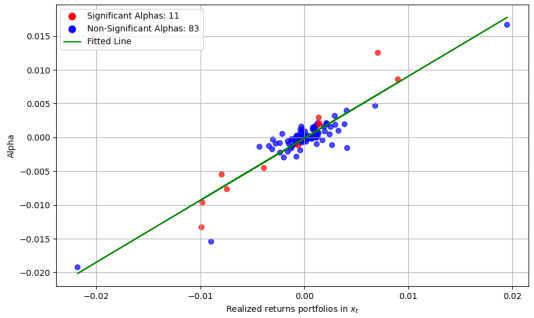
The **CA** models generally report a higher number of significant alphas compared to the benchmark models, indicating some mispricing or failure to fully capture risk exposures through the risk factors. For example, the $\boldsymbol{CA_3}$ model, captures 22 significant alphas for $K = 4$ through the risk factors with an average t-statistic of 1.86. On the other hand, the $\boldsymbol{CA_2}$ model with $K = 4$ captures only 1 significant alpha. Overall, the level of average alphas for all **CA** models seem to be small, indicating that the models are effective at explaining the variation of the excess returns of the portfolios through their latent risk factors. The observable factor models report 8 significant alphas across all factor counts $K = 1$ through $K = 6$, with a low average alpha of 0.0011 and stable t-statistics around 1.35. Low average alphas indicate that the **FF** models are good at explaining the variation in excess returns through their risk factors. This is also expected, as the risk factors themselves are characteristic-sorted portfolios. Similarly, the **PCR** models show performance comparable to the **FF** models, with 8 significant alphas when $K = 1$ to $K = 3$ and slightly lower numbers (7 significant alphas) at higher factor counts. The average alpha is 0.0015 across the board, and the t-critical values hover around 1.35 to 1.40. The proportion of significant t-statistics is also low, varying between 0.07 and 0.09, indicating that the model is relatively good at explaining the portfolio excess returns through its latent risk factors. The **PLS** models 3 to 7 across significant alphas across different all latent factors. The average alpha is 0.0014, slightly lower than the **PCR** model, and the t-critical values are also lower, between 1.21 and 1.35. The proportion of significant t-statistics is notably small, between 0.03 and 0.07. The low low number of significant alphas along with a low average alpha value indicates that the model is relatively effective at explaining the excess returns through its latent risk factors. To conclude this section, the **CA** models show some mispricing, as evidenced by the higher number of significant alphas, especially in models like $\boldsymbol{CA_3}$. However, the small average alphas indicate that these models are still relatively good at capturing returns through their latent risk factors. In contrast, the **benchmark models** (**PCR**, **PLS** and **FF**) generally show fewer significant alphas and lower average alphas, suggesting they better explain returns with less mispricing, particularly the**FF**and **PCR** models. The **PLS** model, while slightly less effective, still captures most of the risk exposures.



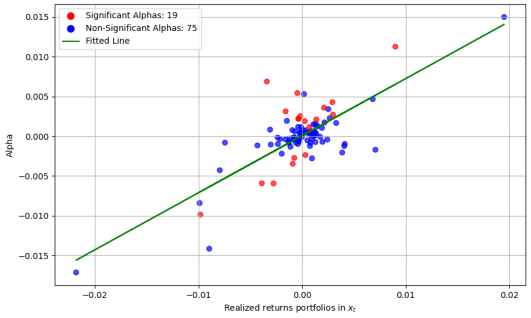
(a) CA_0



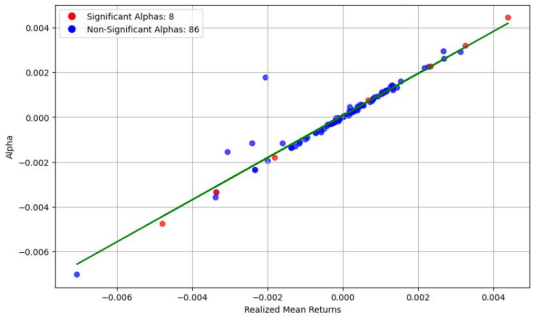
(b) CA_1



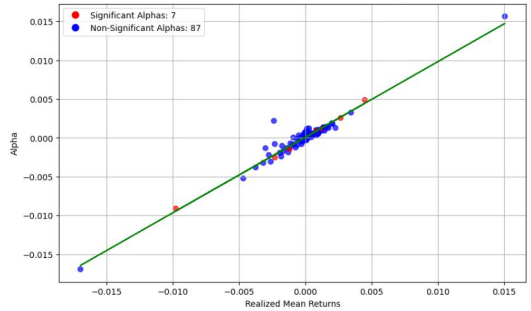
(c) CA_2



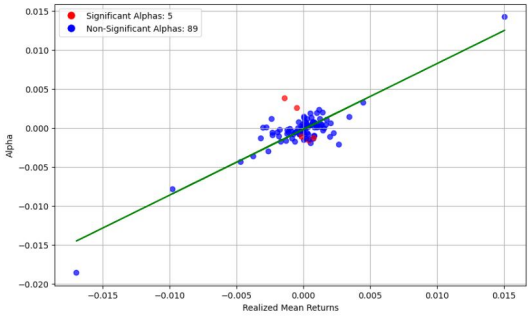
(d) CA_3



(e) FF



(f) PCR



(g) PLS

Figure 3: Scatterplots of the pricing errors (alphas) of the portfolio returns, α_i vs. the actual expected realized returns, $\mathbb{E}(x_{i,t+1})$ with 5 factors. The blue dots show the insignificant alphas (pricing errors) of the characteristics-managed portfolios with a t-statistic with a critical value of less than $|3.0|$.

Table 5: Statistics for alphas for PCR, PLS, FF, and CA Models

K	1	2	3	4	5	6
			FF			
Significant Alpha Counts	8	8	8	8	8	8
Avg alpha	0.001111	0.0011	0.0011	0.0011	0.0011	0.0011
Average T-Critical Alpha	1.36	1.35	1.35	1.35	1.35	1.35
Proportion of Significant T-Statistics	0.09	0.09	0.09	0.09	0.09	0.09
			PCR			
Significant Alpha Counts	8	8	8	7	7	7
Avg alpha	0.0015	0.0015	0.0015	0.0015	0.0015	0.0015
Average T-Critical Alpha	1.35	1.40	1.38	1.38	1.37	1.37
Proportion of Significant T-Statistics	0.09	0.09	0.09	0.07	0.07	0.07
			PLS			
Significant Alpha Counts	4	3	4	5	5	7
Avg alpha	0.0014	0.0014	0.0014	0.0014	0.0014	0.0014
Average T-Critical Alpha	1.30	1.21	1.25	1.28	1.33	1.35
Proportion of Significant T-Statistics	0.04	0.03	0.04	0.05	0.05	0.07
			CA_0			
Significant Alpha Counts	9	7	8	11	7	5
Avg alpha	0.0022	0.0020	0.0021	0.0022	0.0019	0.0020
Average T-Critical Alpha	1.49	1.31	1.37	1.51	1.26	1.30
Proportion of Significant T-Statistics	0.096	0.07	0.09	0.12	0.07	0.05
			CA_1			
Significant Alpha Counts	7	8	5	5	3	5
Avg alpha	0.0022	0.0022	0.0020	0.0019	0.0018	0.0019
Average T-Critical Alpha	1.47	1.47	1.33	1.27	1.21	1.15
Proportion of Significant T-Statistics	0.075	0.085	0.053	0.053	0.032	0.053
			CA_2			
Significant Alpha Counts	9	5	8	1	11	4
Avg alpha	0.0023	0.0020	0.0023	0.0016	0.0022	0.0019
Average T-Critical Alpha	1.47	1.34	1.51	1.05	1.42	1.25
Proportion of Significant T-Statistics	0.096	0.05	0.085	0.011	0.12	0.04
			CA_3			
Significant Alpha Counts	10	9	10	22	19	12
Avg alpha	0.0022	0.0022	0.0022	0.0024	0.0022	0.0021
Average T-Critical Alpha	1.54	1.50	1.50	1.86	1.75	1.75
Proportion of Significant T-Statistics	0.11	0.10	0.11	0.23	0.20	0.13

3.5 Characteristic importance

As the models in this paper are using 94 stock characteristics as input, it is of interest to examine which stock characteristics are ranked of importance, ranked from highest to lowest. The methodology for measuring the importance of the stock characteristics, is the permutation method. It iteratively shuffles each of the the 94 stock characteristics at a tíme and measures the change in R^2_{Total} for each iteration, holdig the other stock characteristics constant. A reduction of R^2_{Total} is translated into the given stock characteristic being of larger importance, while an increase of R^2_{Total}, translates into the stock characteristic being of smaller importance. In this section the characteristic importance is measured for the **CA** against the latent factor benchmark models. Subsequently, the characteristic-importance is measured for the factor-network and the beta-network separately of the **CA** model, for a more nuanced insight to the contribution of the predictions of the whole model.

Figure 4 presents a comparative analysis of the **CA** models ($\boldsymbol{CA_0}$, $\boldsymbol{CA_1}$, $\boldsymbol{CA_2}$, and $\boldsymbol{CA_3}$) (panels **(a)**, **(b)**, **(c)**, and **(d)**) with their benchmark models, **PCR**, **IPCA**, and **PLS** (panels **(e)**, **(f)**, and **(g)**) with a 5-factor specification. Each panel displays the top 20 most important stock characteristics in terms of their contribution to explaining excess returns, represented by horizontal bar plots.

The top 5 most important characteristics for each **CA** model reveal interesting patterns. For $\boldsymbol{CA_0}$, these are **roeq** (Return on Equity), **gma** (Gross Profitability), **sp** (Sales to Price), **std_turn** (Volatility of Share Turnover), and **rd_mve** (R&D to Market Capitalization). In $\boldsymbol{CA_1}$, **roeq** remains important, while **sp**, **std_turn**, **rd_sale** (R&D to Sales), and **gma** also appear prominently. For $\boldsymbol{CA_2}$, **sp**, **std_turn**, and **roeq** again feature, with the addition of **rd_mve** and **roaq** (Return on Assets). Lastly, in $\boldsymbol{CA_3}$, **sp**, **gma**, **retvol** (Return Volatility), **std_turn**, and **rd_mve** dominate the rankings. Notably, **roeq** is the most important characteristic in $\boldsymbol{CA_0}$ and $\boldsymbol{CA_1}$, while **sp** is the top factor in $\boldsymbol{CA_2}$ and $\boldsymbol{CA_3}$[43].

The benchmark models display different patterns. In the **PCR** model, **chempia** (Industry-Adjusted Change in Employees), **lgr** (Long-Term Debt Growth), **hire** (Employee Growth Rate), **invest** (Capital Expenditures and Inventory), and **chinv** (Change in Inventory) are the most important. The **IPCA** model prioritizes **ill** (Illiquidity), **mvel1** (Size), **std_turn**, **turn** (Share Turnover), and **dolvol** (Dollar Trading Volume). Finally, the **PLS** model ranks **turn**, **retvol**, **baspread** (Bid-Ask Spread), **maxret** (Maximum Daily Return), and **age** (Years since First Compustat Coverage) as its top 5. From these rankings, it is evident that the $\boldsymbol{CA_0}$ model stands out by favoring a smaller number of key characteristics, potentially reflecting the effectiveness of its lasso regularization. In contrast, the other **CA** models exhibit more balanced importance across a wider set of characteristics. Despite these variations, certain stock characteristics consistently rank highly across models. **roeq** (Return on Equity) appears among the top 5 in $\boldsymbol{CA_0}$, $\boldsymbol{CA_1}$, and $\boldsymbol{CA_2}$. **gma** (Gross Profitability) is prominent in $\boldsymbol{CA_0}$, $\boldsymbol{CA_1}$, and $\boldsymbol{CA_3}$. **sp** (Sales to Price) is consistently important across all **CA** models and is particularly notable as the top factor in $\boldsymbol{CA_2}$ and $\boldsymbol{CA_3}$. **std_turn** (Volatility of Share Turnover) also ranks in the top 5 for all four **CA** models, while **rd_mve** (R&D to Market Capitalization) appears prominently in $\boldsymbol{CA_0}$, $\boldsymbol{CA_2}$, and $\boldsymbol{CA_3}$. While

[43]For further information regarding the codes of the 94 stock characteristics, look at Table A.6 in Gu et al. 2019

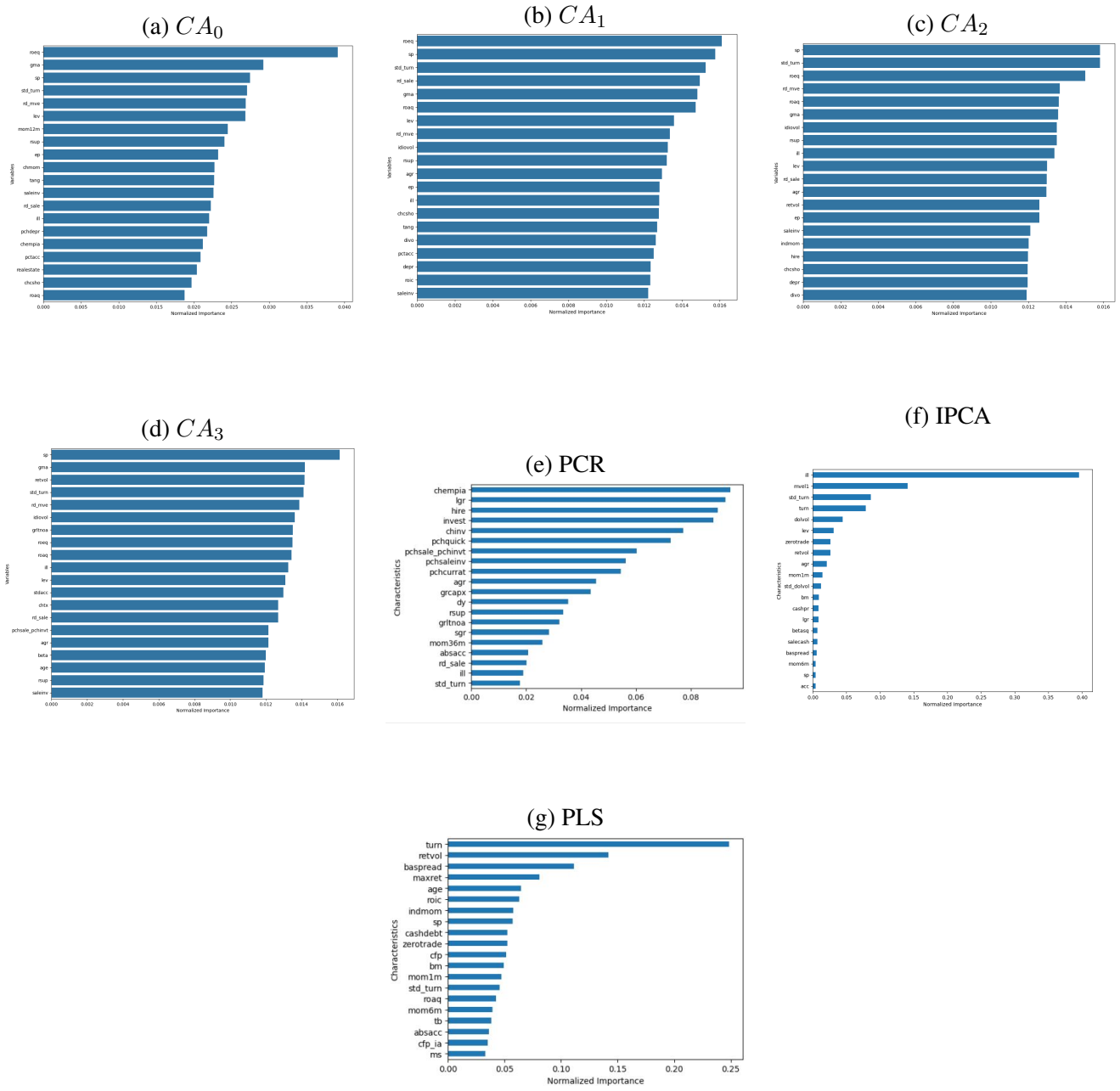


Figure 4: Top 20 of the most important stock characteristics for models normalized to 1 and ranked after importance for CA_0, CA_1, CA_2, and CA_3 with five latent factors.

the benchmark models and the **CA** models diverge in their characteristic rankings, they share a focus on a small set of critical stock characteristics that contribute significantly to explaining variations in excess returns. Next, the characteristic-importance for factor networks of the **CA** models are compared against each other.

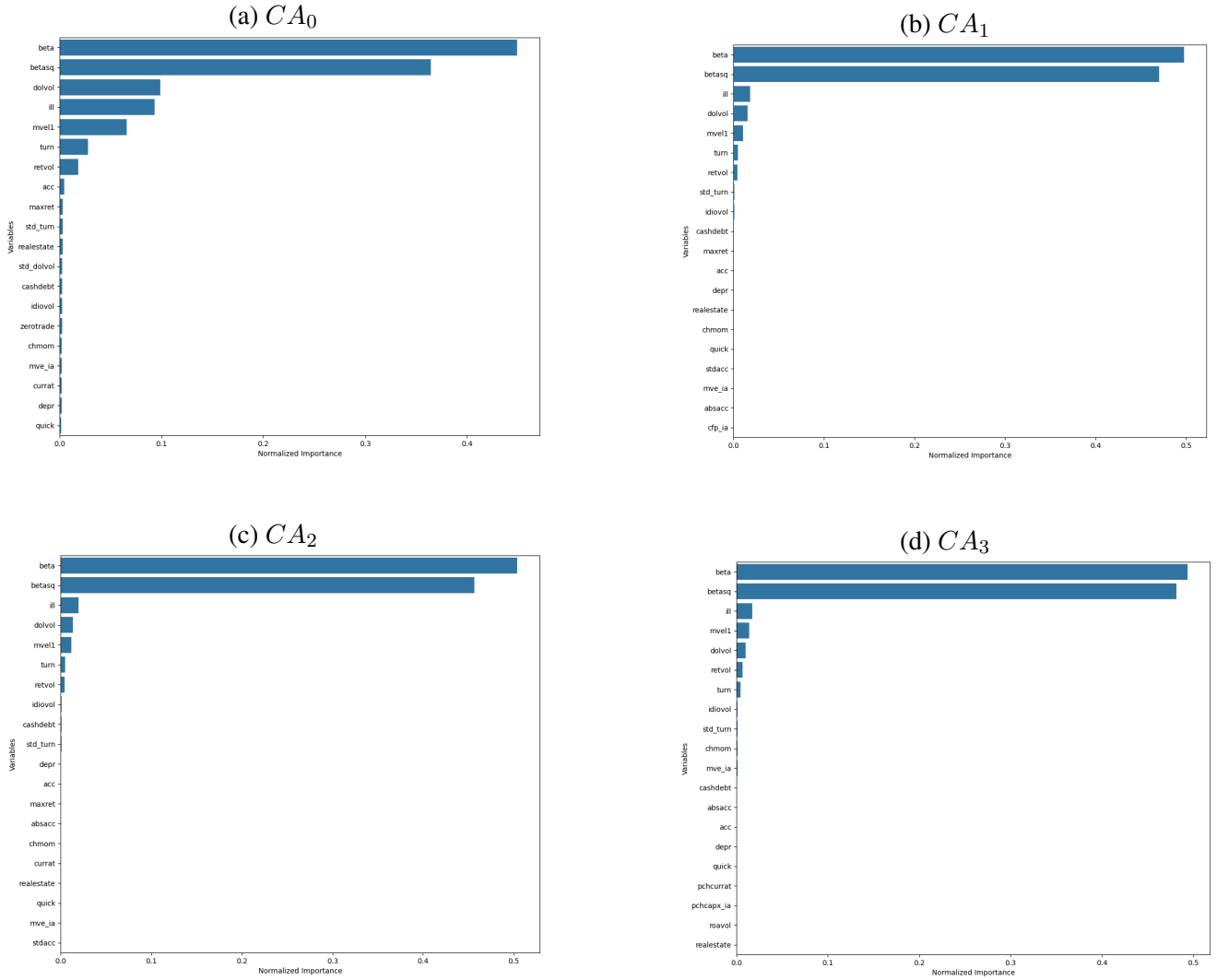


Figure 5: Top 20 of the most important stock characteristics for the factor network of the models normalized to 1 and ranked after importance for CA_0,CA_1,CA_2 and CA_3 with five latent factors

Figure 5 presents characteristic importance plots of the 20 most important stock characteristics for the factor networks of all **CA** models with 5 factors. It is clear that each model emphasizes around seven characteristics, while the importance of the remaining characteristics decreases sharply. This suggests that the lasso regularization effectively penalizes the loss function, driving irrelevant characteristics to zero, and selects only those relevant for predictions. The stock characteristics are then averaged for all folds, resulting in average importances over the whole prediction time period. The seven most important

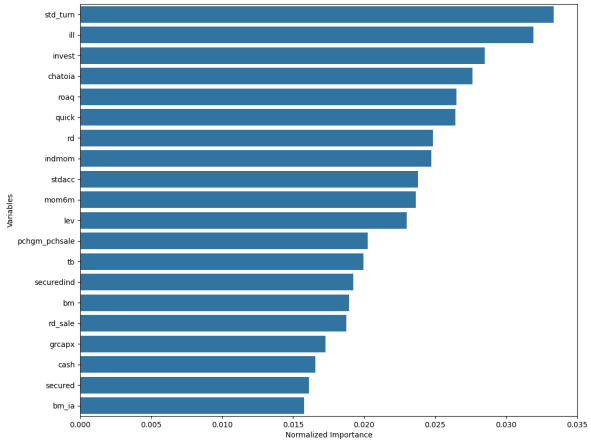
stock characteristics for the CA_0 model are, in order: **beta** (Beta market correlation), **betasq** (Beta market correlation squared), **dolvol** (Dollar trading volume), **ill** (Illiquidity) (Size), **turn** (Share Turnover), and **retvol** (Return Volatility). For the CA_1 and CA_2 models, the ranking remains consistent: **beta**, **betasq**, **ill**, **dolvol**, **mvel1**, **turn**, and **retvol**. In the CA_3 model, the order of the seven most important characteristics follows a similar pattern: **beta**, **betasq**, **ill**, **mvel1**, **dolvol**, **retvol**, and **turn**. Across all models, **beta** and **betasq** emerge as the most significant characteristics, reducing the R^2_{Total} of the factor networks by approximately 0.50% and 0.475%, respectively. Other characteristics, including **dolvol**, **ill**, **mvel**, **turn**, and **retvol**, have moderate importance for the CA_0 model, with contributions ranging from 0.01 to 0.1. However, in the CA_1, CA_2, and CA_3 models, these characteristics hold much less importance, with values between 0.01 and 0.02 at most. Crucially, the factor networks across all models demonstrate a similar ranking of the most important characteristics, indicating consistency in the selection of characteristics that drive predictions in the **CA** models. Finally, the characteristic importances for the beta network for each of the **CA** models, are computed.

Figure 6 presents the characteristic importance for the beta networks of the CA_0, CA_1, CA_2, and CA_3 models. For the CA_0 model, five of the most important stock characteristics are **std_turn** (Volatility of share turnover), **ill** (Illiquidity), **invest** (Capital expenditures and inventory), **charoia** (Industry-adjusted change in asset turnover), and **roaq** (Return on assets). The characteristic importance for these variables ranges from approximately 0.027 to 0.37. For both the CA_1 and CA_2 models, the five most important characteristics are **std_turn**, **ill**, **quick** (Quick ratio), **roaq**, and **charoia**. For the CA_3 model, the five key characteristics are **std_turn**, **sp** (Sales to price), **pchsale_pchinv** (Percentage change in sales minus percentage change in inventory) (Sales to cash), and **grltnoa** (Growth in long-term net operating assets). Across all models, there is agreement on the most important characteristic: **std_turn** (Volatility of share turnover). Furthermore, CA_0, CA_1, and CA_2 consistently identify **std_turn** and **ill** as the two most important characteristics. The CA_3 model, while still highlighting **std_turn**, introduces a different set of characteristics into its top rankings after this factor. The disparity in the importance of the most and least influential characteristics, as shown in **Figure 6**, suggests two possible explanations: either the lasso regularization has not penalized irrelevant stock characteristics enough to clearly isolate a few dominant ones, leaving more characteristics in contention, or the beta-network captures non-linear relationships better than the benchmark models (due to hidden layers in CA_1, CA_2 and CA_3) among stock characteristics, resulting in a broader set of characteristics being important compared to the linear factor networks.

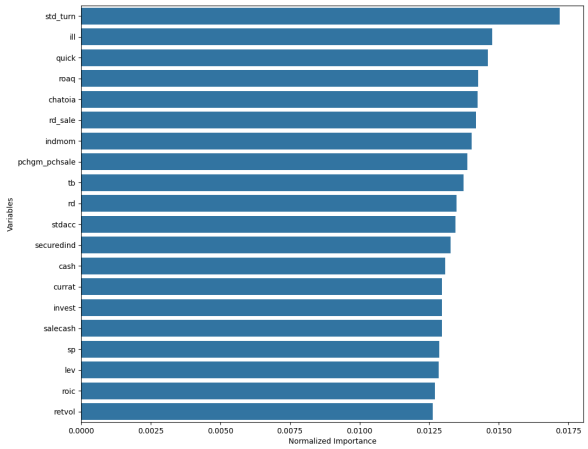
3.6 Robustness Check

This section evaluates the models' reliability and robustness by assessing their ability to consistently select the most relevant stock characteristics for predicting excess returns, when irrelevant noise variables are introduced. This section also examines whether the models maintain stable predictions of excess returns in the presence of these noise characteristics. The introduced noise consists of simulated irrelevant

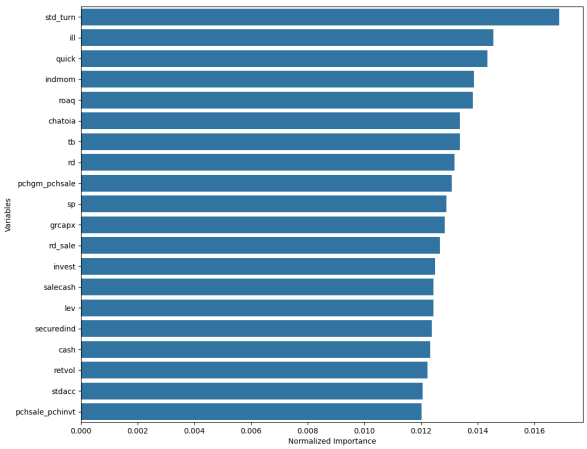
(a) CA_0



(b) CA_1



(c) CA_2



(d) CA_3

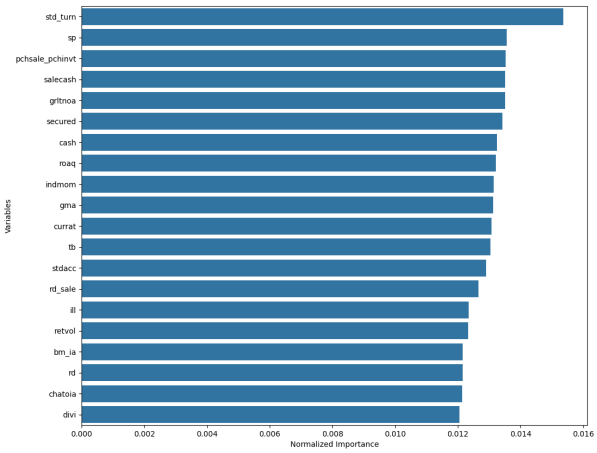


Figure 6: Top 20 of the most important stock characterics for the beta network of the models, normalized to 1 and ranked after importance for CA_0,CA_1,CA_2 and CA_3 with five latent factors

stock characteristics, challenging the variation input characteristics. This tests whether the characteristic selection remains consistent and reliable, while also evaluating the stability of the predicted excess returns. A data-generating process is employed to construct five noise stock characteristics with mean and standard deviation of all original 94 stock characteristics, following the approach of Gu et al. 2019. The noise characteristics are then incorporated into the characteristic space for the robustness check. The next stage involves making predictions on the test set, at the individual stock level and at the portfolio level, followed by a characteristic-importance analysis, that examine whether the introduction of noise affects the stability of the predictions and the selection of stock characteristics respectively. The out-of-sample predictions on the test set compare the models' performance with and without the introduced noise characteristics. The performance evaluation metrics used are the Total- and Predictive-R^2 (R^2_{Total} and R^2_{Pred}) for the excess returns of individual stocks, $r_{i,t+1}$, and the characteristic-managed portfolios, x_{t+1}. The methodology of the characteristics importance with- and without the introduced noise characteristics is the same as in section 3.5. These two analyses help providing insight into the models' predictive stability and selection of characteristics, when noise is introduced.

Table 6: Total R^2 for the individual excess returns of stocks and characteristic-managed portfolios x_t

CA models	R^2 for r_t	R^2 for x_t	R^2 for r_t w/o noise	R^2 for x_t w/o noise
$\boldsymbol{CA_0}$	9.25%	0.89%	15.00%	17.64%
$\boldsymbol{CA_1}$	8.77%	2.64%	14.00%	19.66%
$\boldsymbol{CA_2}$	8.90%	-0.68%	13.00%	8.80%
$\boldsymbol{CA_3}$	9.44%	9.09%	12.00%	15.67%
IPCA	-0.11%	-0.25%	–0.00%	0.00%
PCR	-0.01%	-0.15%	–0.00%	0.00%
PLS	0.00%	0.00%	-0.01%	-0.15%

Table 6 presents the Total out-of-sample R^2_{Total} for individual excess returns ($r_{t'1}$) and characteristic-managed portfolios (x_{t+1}), with and without the introduced noise characteristics, for both the models and the benchmark models (**IPCA**, **PCR**, and **PLS**). For individual stock returns, the R^2_{Total} for the **CA** models ranges from 8.77 to 9.44 with noise. Specifically, $\boldsymbol{CA_0}$ shows the highest performance at 9.25, while $\boldsymbol{CA_1}$, $\boldsymbol{CA_2}$, and $\boldsymbol{CA_3}$ follow closely. When noise is removed, theR^2_{Total} values increase, with $\boldsymbol{CA_0}$ reaching 15.00 and $\boldsymbol{CA_1}$ through CA_3 also improving, with $\boldsymbol{CA_1}$ reaching 14.00, $\boldsymbol{CA_2}$ at 13.00, and $\boldsymbol{CA_3}$ at 12.00. For characteristic-managed portfolios (x_t), the R^2_{Total} varies significantly across the **CA** models, ranging from -0.68 to 9.09 with noise. $\boldsymbol{CA_3}$ has the highest value at 9.09, while $\boldsymbol{CA_2}$ shows a negative R^2_{Total} of -0.68. When noise is removed, all **CA** models show substantial improvement, with $\boldsymbol{CA_1}$ achieving the highest R^2_{Total} at 19.66, followed by $\boldsymbol{CA_0}$ at 17.64, $\boldsymbol{CA_3}$ at 15.67, and $\boldsymbol{CA_2}$ at 8.80. In contrast, the benchmark models (**IPCA**, **PCR**, and **PLS**) exhibit negative or near-zero R^2_{Total} for both r_{t+1} and x_t, indicating that these models explain little to no variation when noise is present. The removal of

noise does not significantly improve their performance, as their R^2 values remain negative or close to zero. Overall, the removal of noise enhances the R^2_{Total} for both r_t and x_t across all **CA** models, demonstrating the robustness of **CA** models, particularly in the presence of noise. In comparison, the benchmark models show negligible improvement or no change, underscoring the superior performance of the **CA** models in capturing the variance of individual stock returns and characteristic-managed portfolios.

Table 7: Predictive R^2 for the individual excess returns r_{t+1} and characteristic-managed portfolios x_{t+1}

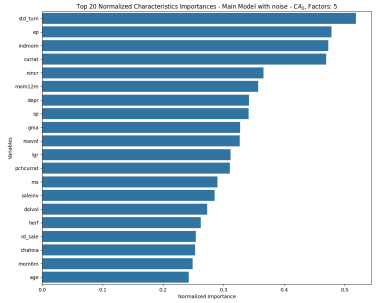
CA models	R^2 for r_t	R^2 for x_t	R^2 for r_t w/o noise	R^2 for x_t w/o noise
$\boldsymbol{CA_0}$	-4.39%	-4.37%	2.00%	-0.85%
$\boldsymbol{CA_1}$	-8.85%	-3.98%	0.00%	0.73%
$\boldsymbol{CA_2}$	-3.73%	-0.76%	2.00%	0.21%
$\boldsymbol{CA_3}$	-3.63%	-0.15%	3.00%	-0.51%
IPCA	-0.013%	-0.16%	–0.00%	–0.00%
PCR	-0.012%	-0.14%	–0.00%	–0.00%
PLS	-0.013%	-0.16%	–0.00%	–0.00%

Table 7 presents the Predictive out-of-ample R^2_{Pred} for individual excess returns of stocks (r_{t+1}) and characteristic-managed portfolios (x_{t+1}), both with and without the introduction of noise characteristics, for the **CA** models ($\boldsymbol{CA_0}$ to $\boldsymbol{CA_3}$) and benchmark models (**IPCA**, **PCR**, and **PLS**).

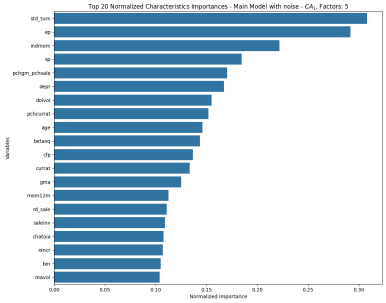
The **CA** models show negative R^2_{Pred} values when noise is introduced, with r_t ranging from -8.85 to -3.63 and x_t ranging from -4.37 to -0.15. When noise is removed, these values generally improve, particularly for r_t, where the R^2_{Pred} increases to positive values, ranging from 0.00 to 3.00. For x_t, while there is some improvement, the R^2_{Pred} remains close to zero or negative in most cases. In contrast, the **IPCA**, **PCR**, and **PLS** models consistently display very low or negative R^2_{Pred} values, both with and without noise. Their R^2_{Pred} values hover near zero, indicating minimal predictive power for both r_{t+1} and x_{t+1}, regardless of the noise presence. Summing up, the table highlights that the introduction of noise significantly affects the predictive power of the **CA** models, but they still show some improvement when the noise is removed, especially for individual stock returns. The benchmark models (**IPCA**, **PCR**, and **PLS**), while less affected by noise, exhibit minimal predictive accuracy overall.

The characteristic importances of the **(CA)** models are evaluated against the benchmark models (**IPCA**, **PCR**, and **PLS**) to assess how these models select stock characteristics when predicting excess returns. The **CA** models, particularly $\boldsymbol{CA_0}$, appear to apply a stricter selection of stock characteristics, especially when noise characteristics are introduced (as seen in Section 3.5). The five most influential stock characteristics for the $\boldsymbol{CA_0}$ model (panel **(a)**) are **std_turn** (Volatility of share turnover), **ep** (Earnings to price), **indmom** (Industry momentum), **currat** (Current Ratio), and **nincr** (Number of earnings increases). In the $\boldsymbol{CA_1}$ model (panel **(b)**), the key characteristics remain similar, with the inclusion of **sp** (Sales to price) and **pchgm_pchsale** (Difference between % change in gross margin and % change

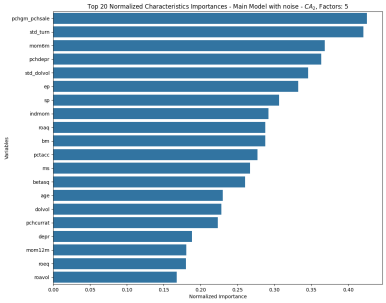
(a) CA_0



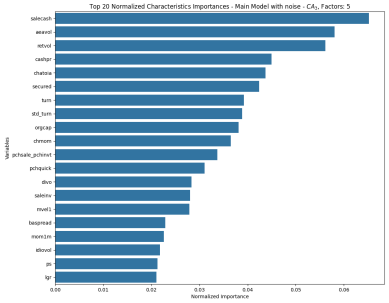
(b) CA_1



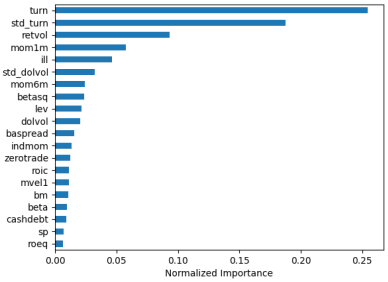
(c) CA_2



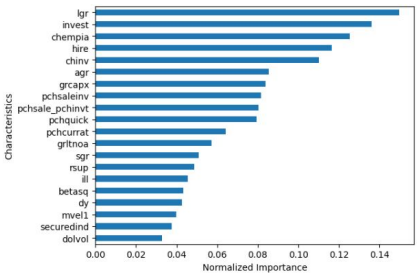
(d) CA_3



(e) IPCA



(f) PCR



(g) PLS

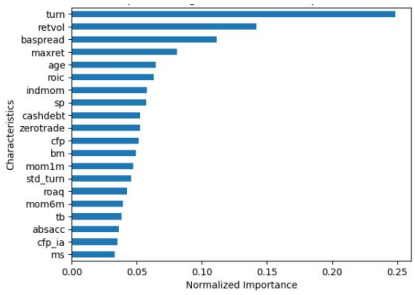


Figure 7: Top 20 most important stock characteristics when 5 noise characteristics are introduced to the set of characteristics. The Characteristics importance for the Conditional Autoencoder models (panels **(a)**, **(b)**, **(c)** and **(d)**) and the benchmark models (**IPCA**, **PCR**, and **PLS**) (panels **(e)**, **(f)** and **(g)**) is evaluated, where the characteristic importances are normalized to 1 and ranked after importance with 5 latent factors

in sales), alongside **std_turn**, **ep**, and **indmom**. This trend of incorporating turnover-related and momentum characteristics continues in the CA_2 model (panel **(c)**), where **pchgm_pchsale**, **std_turn**, and **mom6m** (6-month momentum) emerge as significant, in addition to **pchdepr** (% change in depreciation) and **std_dolvol** (Volatility of liquidity). For the CA_3 model (panel **(c)**, there is a shift in the importance of liquidity and volatility-related characteristics, with the most influential features being **salecach** (Sales to cash), **aeavol** (Abnormal earnings announcement volume), **retvol** (Return Volatility), **cashpr** (Cash productivity), and **chatoia** (Industry-Adjusted change in asset turnover). The benchmark models, particularly **IPCA** and **PLS** (panel **(e)** and **(f)**), exhibit stricter selection patterns, where the importance from top to bottom sharply decreases, with **IPCA** favoring liquidity and volatility measures such as **turn** (Share turnover), **std_turn**, **retvol**, and **std_dolvol**, alongside momentum-related variables like **mom1m** (1-month momentum) and **ill** (Illiquidity). On the other hand, **PCR** (panel **(g)** exhibits a broader selection pattern, akin to the **CA** models. the most influential characteristics for the **PCR** model are **lgr** (Growth in long-term debt), **invest** (Capital expenditures and inventory), and **hire** (Employee growth rate). Concluding this section, across both the Conditional Autoencoder **(CA)** and benchmark models, certain stock characteristics, such as **std_turn** (Volatility of Share Turnover) and **retvol** (Return Volatility), consistently emerge as important for predicting excess returns, even when noise characteristics are introduced. The **CA** models tend to select a broader range of characteristics, such as **sp** (Sales to Price) and **rd_mve** (R&D to Market Capitalization), which appear across multiple models. In contrast, benchmark models like **IPCA** and **PLS** exhibit stricter selection patterns, with importance sharply decreasing from top to bottom, favoring liquidity and volatility measures like **turn**, **std_turn**, and **retvol**. **PCR** follows a broader selection pattern similar to the **CA** models **turn** and **retvol** being the two most important stock characteristics. While both **CA** and benchmark models overlap on key characteristics, the **CA** models capture a more diverse set, suggesting they may offer advantages in identifying complex stock return drivers.

3.7 Simulation Study

Because all models are trained on the same dataset, unintentional randomness can influence their performance. to mitigate this issue, a Monte Carlo Study inspired by Gu et al. 2020 is conducted. The purpose of the Monte Carlo Study, is to evaluate the Conditional Autoencoder (**CA**) models and the latent factor benchmark models (**IPCA**, **PCR** and **PLS**) over a finite number of repetitions. Parameters are chosen to enable a reliable performance evaluation across many repetitions with finite samples, averaging the performance over all iterations.

Monte Carlo Simulations, are useful for testing the robustness of models by repeatedly simulating datasets. As the number of simulations increases, the mean converges to the approximated "true mean" the model's performance. This principle is based on the Law of Large Numbers **(LLN)** and the Central Limit Theorem **(CLT)**[44].

In this study, a factor model, is used as a data-generating process **(DGP)** for the simulated datasets.

[44]The Law of Large Numbers (LLN and the Central Limit Theoem (CLT) are described in Appendix A.17

Both linear and non-linear variations of the **DGP** models are employed to analyze the performance of the **CA** compared to its benchmark models in different simulated dataset scenarios. The **DGP** is a factor model with five systematic risk factors and conditional factor loadings, that simulate the returns of artificial stocks, expanding upon the three-factor model **DGP** used in Gu et al. 2020.

By comparing the two **DGP** variations, this study aims to evaluate how well the **CA** and the benchmark models capture linear versus non-linear relationships in the simulated data. Unlike the benchmark models, the **CA** model incorporates hidden layers in its beta network,[45] enabling it to approximate non-linear relationships more effectively. The general **DGP** model specification is given by the following equations:

$$r_{i,t+1} = \beta_{i,t} f_{t+1} + \psi_{i,t+1}, \quad \beta_{i,t} = \gamma(c_{i,t}, \phi) \quad f_{t+1} = Ax_{t+1} + \eta_{t+1} \tag{40}$$

The first identity describes a conditional factor model, with a conditional β for stock i at time t, a vector of factors, f_{t+1} and an error term, ψ_{t+1} added. The second identity describes the time-varying betas which are a function of the product of a subspace of the simulated conditional stock characteristics of individual stocks, $c_{i,t}$ and a parameter ϕ, which scales certain stock characteristics within the function for β.

The third equation describes the behavior of the factors of the **DGP**. The factors, f_{t+1} are determined by a matrix, A of coefficients multiplied by the $C \times 1$ vector of x_{t+1} of factor components with an error term, η_{t+1} added. The error term, ψ of the factor model is distributed by $\phi_{i,t+1} \sim t_5(0, 0.1^2)$ [46]. The factor components, x_{t+1} are $x_{t+1} \sim \mathcal{N}(0, 0.01^2 \times \mathbb{I}_{L_x})$ [47] and the error term that describes the behavior of the factors is $\eta \sim \mathcal{N}(0, 0.01^2 \times \mathbb{I}_5)$[48]. The lagged stock characteristics for individual stocks i, $c_{i,t}$ are given by the following process :

$$c_{ij,t} = \frac{2}{n+1}\textbf{rank}(\hat{c})_{ij,t} - 1 \quad \hat{c}_{ij,t} = \theta_j \hat{c}_{ij,t-1} + \nu_{ij,t} \tag{41}$$

The process for the simulated stock characteristics follows a conditional process, where $\theta_j \sim \mathcal{U}(0.9, 1)$[50] and $\nu ij, t \sim \mathcal{N}(0, 1)$[51] distributed, such that the stock characteristics are persistent and are cross-sectionally rank-normalized into the interval $[-1, 1]$. There are two scenarios set up for the conditional **DGP**. The first scenario considers a linear factor model, where the vector of betas is a linear function that determines the returns $r_{i,t+1}$:

$$\Gamma(c_{i,t})_{Linear} = \gamma(c_{i,1t}, c_{i,2t}, c_{i,3t}, c_{i,4t}, c_{i,5t}) \quad \text{where} \quad \gamma_0 = (1.2, 1, 0.8, 0.6, 0.4) \tag{42}$$

The second scenario assumes a factor model with a vector of non-linear betas, where the stock character-

[45] For CA_1, CA_2 and CA_3 have 1, 2 and 3 hidden layers respectively in their Beta Networks.

[46] This is a student t-distribution with 5 degrees of freedom, a mean of 0 and a variance of 0.01

[47] This represents a multivariate normal distribution with a mean of zero and a covariance matrix of $\mathbb{I}_{L_x} \times L_x$, where $\mathbb{I}_{L_x}$ is the identity matrix of dimension L_x

[48] [49]

[50] This denotes a uniform distribution $\mathcal{U}(0.9, 1)$, where the variable θ_j is uniformly distributed over the interval $[0.9, 1]$.

[51] $\mathcal{N}(0, 1)$ denotes a normal distribution with mean zero and variance 1

istics are transformed non-linearly:

$$\Gamma(c_{i,t})_{Non-linear} = \gamma(c_{i,1t}^3, c_{i,2t} \times c_{i,1t}, c_{i,3t}^2, c_{i,3t} \times c_{i,4t}, c_{i,4t}^4) \quad \text{where} \quad \gamma_0 = (1.2, 1, 2, 1.6, 0.6) \quad (43)$$

Each simulated dataset, is split into a 60% training set (108 time periods), a 20% validation set, and a 20% test set (36 time periods each). All Autoencoder models are trained on the training set, tuned on the validation set, and making predictions on the test set. The parameters for the assumed distribution of both the linear and the non-linear **DGP** are calibrated, such that the **DGP** model has approximately 45% and 5% in total- and predictive-R^2. Each simulation is repeated 100 times in a Monte Carlo experiment to assess the performance of the Autoencoder model on finite samples through repeated trials. Firstly, forecasts are made for at the individual stock-level, then at the portfolios, both measured by Total-R^2 and Predictive-R^2. Lastly, the mean-variance efficiency is measured using the Sharpe Ratio for equal-weighed long-short portfolios constructed with the same methodology as in Section 3.3.

3.7.1 Monte Carlo Performance Evaluation of r_{t+1}

Table 8 presents the Total and Predictive Out-of-Sample (R^2_{Total} and R^2_{Pred}) values for stock returns ($r_{i,t}$) across ghd **(CA)** and benchmark models (**IPCA**, **PCR**, and **PLS**), tested on both linear and non-linear **DGP**s. The table evaluates how well these models explain the total variation in returns and their predictive accuracy of the returns.

Panel **(a)**, focuses on the Linear **DGP**, where the **CA** models clearly outperform the benchmark models in terms of R^2_{Total}. The $\boldsymbol{CA_2}$ model with 4 factors achieves the highest Total R^2_{Total} of 30.3%, while the $\boldsymbol{CA_2}$ model with 1 factor has the lowest R^2_{Total} at 16.0%. In contrast, the benchmark models show significantly lower R^2_{Total}, with the best performing models being **IPCA** with 5 factors (2.24%) and **PLS** with 2 factors (1.89%), while **PCR** lags further behind (0.3% with 6 factors). This indicates that the **CA** models are much better at capturing the overall variation in stock returns for the linear **DGP**.

Considering the R^2_{Pred} in panel **(a)**, the performance shifts slightly. The $\boldsymbol{CA_0}$ model with 5 factors achieves the highest R^2_{Pred} among the **CA** models at 1.75%, but the $\boldsymbol{CA_3}$ model with 1 factor exhibits the lowest R^2_{Pred} at -0.47%. Interestingly, the benchmark models, particularly **IPCA**, outperform the **CA** models in predictive accuracy. The **IPCA** model reaches a R^2_{Pred} of 2.78% with 4 factors, while **PLS** achieves a peak of 1.99%. This suggests that while **CA** models capture total variation more effectively, **IPCA** excels in predictive performance under the linear **DGP**.

Panel **(b)**, presents results for the Non-Linear DGP, where the **CA** models continue to out-perform in terms of R^2_{Total}. The $\boldsymbol{CA_0}$ and $\boldsymbol{CA_1}$ models, with 5 and 4 factors respectively, achieve R^2_{Total} values of 10.4%, while $\boldsymbol{CA_3}$ with 6 factors shows the lowest value at 7.2%. In contrast, the benchmark models, particularly **IPCA** and **PLS**, perform much worse, with **IPCA**'s best R^2_{Total} reaching only 1.17% and **PCA** even showing a negative R^2_{Total} (-0.01% with 1 factor). This indicates that **CA** models are substantially better at capturing the total variation in returns even under non-linear relationships, although the effectiveness of deeper models like $\boldsymbol{CA_3}$ declines.

	Linear DGP r_{t+1} (a)					
K	1	2	3	4	5	6
			R^2_{Total}			
$\boldsymbol{CA_0}$	20.8%	21.1%	20.6%	20.8%	21.3%	21.4%
$\boldsymbol{CA_1}$	28.8%	28.2%	29.3%	29.3%	28.7%	28.9%
$\boldsymbol{CA_2}$	26.8%	26.0%	26.0%	30.3%	24.0%	25.1%
$\boldsymbol{CA_3}$	17.0%	16.0%	22.0%	27.0%	20.0%	23.1%
IPCA	1.31%	1.59%	1.93%	1.84%	2.24%	2.19%
PCR	0.01%	0.04%	0.1%	0.2%	0.2%	0.3%
PLS	1.57%	1.89%	1.83%	1.74%	1.52%	1.03%
			R^2_{Pred}			
$\boldsymbol{CA_0}$	1.38%	1.25%	1.72%	1.43%	1.75%	1.19%
$\boldsymbol{CA_1}$	1.04%	-0.00%	0.88%	0.94%	1.30%	1.05%
$\boldsymbol{CA_2}$	0.76%	0.53%	0.00%	0.15%	0.10%	0.73%
$\boldsymbol{CA_3}$	-0.47%	0.32%	-0.14%	0.22%	0.39%	0.13%
IPCA	1.04%	2.04%	2.07%	2.78%	2.37%	2.09%
PCR	0.03%	0.06%	0.2%	0.2%	0.2%	0.2%
PLS	2.17%	1.68%	1.15%	1.99%	1.88%	1.99%
	Non-Linear DGP r_{t+1} (b)					
K	1	2	3	4	5	6
			R^2_{Total}			
$\boldsymbol{CA_0}$	9.9%	9.8%	10.3%	10.3%	10.4%	9.7%
$\boldsymbol{CA_1}$	9.7%	10.1%	10.1%	10.4%	10.0%	10.3%
$\boldsymbol{CA_2}$	9.2%	9.2%	8.8%	9.5%	9.3%	9.0%
$\boldsymbol{CA_3}$	7.5%	7.7%	7.7%	7.6%	7.6%	7.2%
IPCA	0.72%	1.04%	1.17%	0.48%	0.65%	0.67%
PCR	-0.01%	0.00%	-0.00%	-0.01%	0.04%	0.00%
PLS	0.38%	0.76%	0.40%	0.51%	0.63%	0.50%
			R^2_{Pred}			
$\boldsymbol{CA_0}$	0.79%	0.67%	0.63%	1.02%	1.29%	0.88%
$\boldsymbol{CA_1}$	-0.09%	-0.08%	-0.04%	-0.45%	-0.53%	-0.07%
$\boldsymbol{CA_2}$	-0.85%	-0.92%	-0.46%	-0.97%	-0.73%	-0.81%
$\boldsymbol{CA_3}$	-0.90%	-1.01%	-1.50%	-1.31%	-1.09%	-0.84%
IPCA	0.66%	1.11%	0.64%	0.84%	0.78%	0.56%
PCR	-0.02%	-0.02%	0.04%	0.01%	0.01%	0.0%
PLS	0.57%	0.33%	0.40%	0.30%	0.38%	0.58%

Table 8: Total and Predictive R^2 of $r_{i,t+1}$ for the Linear and the Non-Linear DGPs

However, when examining R^2_{Pred} in panel **(b)**, the results show a different story. The $\boldsymbol{CA_0}$ model with 3 factors achieves the highest R^2_{Pred} among the **CA** models at 1.02%, but most other **CA** models exhibit negative R^2_{Pred}, indicating poorer performance in predicting future returns under non-linear relationships. For example, the $\boldsymbol{CA_3}$ model with 2 factors shows the lowest R^2_{Pred} at -1.50%. On the other hand, the benchmark models perform relatively better in predictive accuracy. **IPCA** achieves a R^2_{Pred} of 1.11% with 2 factors, outperforming all other models, while **PLS** reaches 0.58% with 6 factors.

The results show that **CA** models outperform the benchmark models in terms of R^2_{Total} for both linear and non-linear DGPs, demonstrating their ability to capture total variation in stock returns. However, for R^2_{Pred}, especially under the non-linear **DGP**, the benchmark models, particularly **IPCA**, show stronger performance in predicting future returns, suggesting that while **CA** models excel at explaining overall variation, their predictive accuracy can be more limited, particularly for non-linear relationships.

3.7.2 Monte Carlo Performance Evaluation of x_{t+1}

Table 9 presents the Total and Predictive R^2 (R^2_{Total} and R^2_{Pred}) values for characteristic-managed portfolios (x_{t+1}) across both linear and non-linear data generating processes (**DGP**s). Panel **(a)** displays the results for the Linear **DGP**, while Panel **(b)** covers the Non-Linear **DGP**. In each panel, the top section shows R^2_{Total}, while the bottom section shows R^2_{Pred}. In Panel **(a)** (Linear **DGP**), the R^2_{Total} for the **CA** models ranges from 38.5% to 61%, with the highest value achieved by the $\boldsymbol{CA_2}$ model with 4 factors (61%). The $\boldsymbol{CA_0}$ model shows the lowest R^2_{Total} at 38.5% for both 1 and 3 factors. The R^2_{Total} generally increases from 38% for the $\boldsymbol{CA_0}$ model to 57% for $\boldsymbol{CA_1}$ before peaking at 61% for $\boldsymbol{CA_2}$, with the $\boldsymbol{CA_3}$ model showing a decrease in performance, ranging between 42% and 55%. By comparison, the benchmark models show significantly worse performance, with R^2_{Total} values around -2%, indicating that these models struggle to capture the variation in characteristic-managed portfolio returns under the linear **DGP**. For R^2_{Pred} in Panel **(a)**, the **CA** models achieve values between 7% and 10.8%. The best-performing **CA** model is $\boldsymbol{CA_1}$ with 5 factors, reaching a R^2_{Pred} of 10.8%, while the worst-performing is $\boldsymbol{CA_0}$ with 2 factors, at 6.8%. Again, the benchmark models underperform, with R^2_{Pred} values around -2%, indicating weak predictive ability. In Panel **(b)** (Non-Linear **DGP**), the R^2_{Total} for the **CA** models ranges from 26.3% to 34%. The best-performing model is $\boldsymbol{CA_1}$ with 6 factors, achieving a R^2_{Total} of 34%, while the $\boldsymbol{CA_3}$ model with 6 factors shows the lowest at 26.3%. Across all **CA** models, R^2_{Total} is generally lower for the non-linear **DGP** compared to the linear **DGP**, though still significantly higher than the benchmark models, which remain around -2%. For R^2_{Pred} in Panel **(b)**, the **CA** models perform similarly to Panel **(a),** with the best result of 7.7% from the $\boldsymbol{CA_1}$ model with 6 factors and the lowest value of 3.1% from the $\boldsymbol{CA_3}$ model with 3 factors. The benchmark models again show R^2_{Pred} values near -2%, highlighting their poor predictive power under the non-linear **DGP**.

Summing up for the portfolio predictions, the results indicate that the **CA** models consistently outperform the benchmark models in both R^2_{Total} and R^2_{Pred}, across both linear and non-linear **DGP**s. While the performance of the **CA** models declines under the non-linear **DGP** compared to the linear **DGP**,

	Linear DGP x_{t+1} (a)					
K	1	2	3	4	5	6
			R^2_{Total}			
$\boldsymbol{CA_0}$	38.5%	39.3%	38.5%	39.2%	39.9%	40.1%
$\boldsymbol{CA_1}$	57.4%	56.7%	57.9%	57.9%	57.3%	57.3%
$\boldsymbol{CA_2}$	55.3%	53.8%	54.0%	61.0%	54.5%	56.1%
$\boldsymbol{CA_3}$	42.4%	42.9%	46.5%	55.7%	51.7%	49.7%
IPCA	-2.17%	-2.24%	-2.22%	-2.19%	-2.20%	-0.02%
PCR	-2.1%	-2.1%	-2.2%	-2.1%	-2.2%	-2.2%
PLS	-2.33%	-2.31%	-2.29%	-2.35%	-2.35%	-2.32%
			R^2_{Pred}			
$\boldsymbol{CA_0}$	7.6%	7.2%	8.3	7.5%	8.6%	6.8%
$\boldsymbol{CA_1}$	10.2%	7.3%	9.8%	9.7%	10.8%	10.1%
$\boldsymbol{CA_2}$	9.6%	8.8%	7.6%	8.0%	8.0%	9.6%
$\boldsymbol{CA_3}$	6.4%	9.4%	7.8%	8.4%	8.5%	8.3%
IPCA	-2.30%	-2.30%	-2.30%	-2.35%	-2.27%	-2.49%
PCR	-2.1%	-2.1%	-2.1%	-2.1%	-2.2%	-2.2%
PLS	2.17%	1.67%	1.15%	1.98%	1.88%	1.99%
	Non-Linear DGP x_{t+1} (b)					
K	1	2	3	4	5	6
			R^2_{Total}			
$\boldsymbol{CA_0}$	28.6%	27.8%	29.5%	29.2%	29.3%	27.8%
$\boldsymbol{CA_1}$	32.3%	33.1%	33.3%	33.6%	32.8%	34.0%
$\boldsymbol{CA_2}$	31.3%	31.5%	31.0%	32.4%	31.4%	30.9%
$\boldsymbol{CA_3}$	27.0%	27.4%	27.6%	27.3%	26.8%	26.3%
IPCA	-2.17%	-2.27%	-2.22%	-2.20%	-2.20%	-2.23%
PCR	-2.1%	-2.1%	-2.1%	-2.1%	-2.1%	-2.2%
PLS	-2.27%	-2.19%	-2.18%	-2.14%	-2.18%	-2.22%
			R^2_{Pred}			
$\boldsymbol{CA_0}$	6.3%	5.8%	5.8%	7.0%	7.6%	6.7%
$\boldsymbol{CA_1}$	7.5%	7.0%	7.2%	6.4%	6.7%	7.7%
$\boldsymbol{CA_2}$	5.3%	5.5%	7.3%	5.5%	6.8%	6.3%
$\boldsymbol{CA_3}$	5.7%	4.6%	3.1%	3.8%	4.1%	5.5%
IPCA	-2.20%	-2.16%	-2.15%	-2.20%	-2.19%	-2.27%
PCR	-2.1%	-2.1%	-2.1%	-2.1%	-2.2%	-2.1%
PLS	-2.19%	-2.24%	-2.18%	-2.18%	-2.19%	-2.24%

Table 9: Total and Predictive OOS R^2 for x_{t+1} of for the linear and non-linear DGPs.

they still demonstrate significantly better results than the benchmark models, especially at the portfolio level. Among the **CA** models, the $\boldsymbol{CA_1}$ model, with one hidden layer, consistently achieves the optimal balance of R^2_{Total} and R^2_{Pred} across all factors (K = 1 to 6) for both **DGP**s. In contrast, the benchmark models struggle to capture meaningful variation in neither the linear nor non-linear scenarios, further emphasizing the superiority of the **CA** models.

3.7.3 Monte Carlo Performance Evaluation of Long-Short portfolios

	Linear DGP (a)					
K	1	2	3	4	5	6
LS Prediction Sharpe Ratio						
$\boldsymbol{CA_0}$	0.51	0.49	0.52	0.51	0.55	0.50
$\boldsymbol{CA_1}$	0.59	0.54	0.58	0.58	0.60	0.59
$\boldsymbol{CA_2}$	0.54	0.55	0.50	0.51	0.51	0.54
$\boldsymbol{CA_3}$	0.43	0.50	0.45	0.47	0.49	0.47
IPCA	0.18	0.27	0.28	0.34	0.29	0.30
PCR	0.03	0.05	0.07	0.08	0.10	0.07
PLS	0.29	0.27	0.24	0.30	0.27	0.28
	Non-Linear DGP (b)					
K	1	2	3	4	5	6
LS Prediction Sharpe Ratio						
$\boldsymbol{CA_0}$	0.38	0.35	0.36	0.39	0.40	0.39
$\boldsymbol{CA_1}$	0.39	0.39	0.39	0.38	0.38	0.38
$\boldsymbol{CA_2}$	0.32	0.32	0.34	0.32	0.33	0.32
$\boldsymbol{CA_3}$	0.29	0.26	0.24	0.27	0.25	0.28
IPCA	0.13	0.17	0.14	0.12	0.13	0.12
PCR	0.02	0.01	0.04	0.04	0.03	0.04
PLS	0.12	0.09	0.12	0.10	0.09	0.13

Table 10: Sharpe Ratios from predictions of forward returns of long-short portfolios based on Monte Carlo Simulations. Panel (a) presents results for the linear DGP, and panel (b) for the non-linear DGP.

Table 10 presents the Sharpe Ratios of predictions of forward returns of Equal-Weighed long-short portfolios formed using the same methodology as in Secton 3.3. **Table 10** is divided into two panels: Panel **(a)** shows Sharpe Ratios for the Linear **DGP**, while Panel **(b)** displays results for the Non-Linear **DGP**.

For the Sharpe Ratios in panel **(a)**, the **CA** models generally fall between 0.50 and 0.60, whereas the benchmark models show lower values, with **IPCA**, **PCA**, and **PLS** reaching a maximum of 0.33, 0.10, and 0.30, respectively. The $\boldsymbol{CA_1}$ model with 5 factors, achieves the highest Sharpe Ratio of 0.60, while the $\boldsymbol{CA_3}$ model with 1 factor has the lowest at 0.47. The $\boldsymbol{CA_1}$ model also achieves the highest level of Sharpe Ratios of around 0.60.

In panel **(b)** with the non-linear **DGP**, the Sharpe Ratios for the **CA** models range from 0.29 to 0.40, while the benchmark models exhibit weaker performances, with **IPCA**, **PCR**, and **PLS** showing ratios between 0.01 and 0.17. The **CA** model with the highest level being the $\boldsymbol{CA_1}$ with a level of around 0.38. The highest Sharpe Ratio of 0.40 is achieved by the $\boldsymbol{CA_0}$ model with 4 factors, while the lowest ratio of 0.29 comes from the $\boldsymbol{CA_3}$ model.

Across both linear and non-linear **DGP**s, the **CA** models consistently outperform the benchmark models for both the linear- and non-linear **DGPs**. Notably, the $\boldsymbol{CA_1}$ model, with 1 hidden layer, stands out as the optimal model across all factors, achieving the highest Sharpe Ratios in most scenarios, in the linear **DGP**. It in forward portfolio return predictions, indicating that this model strikes the right balance between complexity and performance. In the non-linear **DGP**, models with fewer hidden layers, such as $\boldsymbol{CA_0}$ and $\boldsymbol{CA_1}$, also perform better, suggesting that lower complexity is more effective for capturing non-linear relationships. The benchmark models, including **IPCA**, **PCA**, and **PLS**, struggle to generate meaningful Sharpe Ratios across both **DGPs**, further highlighting the robustness of the **CA** models in portfolio return forecasting.

4 Discussion

As this paper focuses on evaluating the robustness of Gu et al. 2020, some points of critique can be raised, because some circumstances of the empirical analyses arguably may lead to other results than their findings. Firstly, the date range of the dataset is shortened due to computational restrictions in Google Colab used not having enough computational power related to training and tuning the models as well as making predictions for the **(CA)** models.

Another point of critique, is the at the preprocessing steps of the data may have been slightly different in this paper than for Gu et al. 2020. A reason for this, is that no detailed description of the preprocessing of the dataset used in their paper was available, which could lead to diverging results.

Furthermore, the choice of training scheme is different than the one in Gu et al. 2020. This paper uses a walk-forward validation scheme, where the training, validation and test set are sliding one year forward for each year of the test set. The validation scheme used in Gu et al. 2020, is a walking-forward validation scheme, where the training period expands for each predicted year, leaving more conditional data for training the models. This can arguably lead to different results. However, there seems to be no established consensus among regarding teg type of validation scheme. The validation scheme is usually determined by a selected time period. The predictive-R^2 as defined by Gu et al. 2020 utilizes the time-series mean of the estimated factors up to time period t, denoted as $\hat{\lambda}_t = \frac{1}{T}\sum_{t=1}^{T} \hat{f}_t$. This factor is then multiplied by the factor loadings at time t, $\beta_{i,t}$, to predict the stock returns at time period $t+1$:

$$\hat{r}_{i,t+1} = \hat{\beta}_{i,t}^{T} \hat{\lambda}_t \tag{44}$$

In contrast, this paper employs a variation of the predictive-R^2 that directly uses the estimated factors

at time t, $\hat{f}_t$, and the factor loadings from time $t-1$, $\hat{\beta}_{i,t-1}$, to predict returns at time $t+1$:

$$\hat{r}_{i,t+1} = \hat{\beta}_{i,t-1}\hat{f}_t \tag{45}$$

This approach simplifies computations by avoiding the need to calculate the time-series mean of factors, which would require modifying the model architecture to store and process historical factor estimates, which is an operation that is computationally intensive and resource consuming. As a consequence of these methodological differences, the predictive-R^2 metrics reported in this paper are not directly comparable to those in Gu et al. 2020. Therefore, any comparisons between the predictive R^2 in this study and that in Gu et al. 2020 should be interpreted with caution, acknowledging that they measure different aspects of the models' predictive capabilities.

The predictive results for the benchmark models may differ significantly due to the absence of explicit regularization in their loss functions. Specifically, the IPCA, PCR, and PLS models do not employ regularization techniques such as Lasso or Ridge penalties. However, as discussed in Sections 2.2.2, 2.2.3 and 2.2.4, these models inherently regularize through dimensionality reduction by selecting a subset of components that capture the most significant variance in the data. This inherent regularization betters their predictive performance and should be considered when interpreting the results. However, inherent regularization may not be enough for these models with respect predictions and characteristics selection, suggesting that these models would have a significantly better performance with explicit regularization in their loss functions.

Methodological differences in assessing characteristic importance also warrant consideration, as they may contribute to discrepancies between this papers findings and those of Gu et al. 2020. In this paper, a permutation characteristic-importance method has ben employed (as detailed in Section 3.5), where each characteristic iteratively is shuffled, thereby randomizing its values while keeping other characteristics constant, and the resulting reduction in the Total-R^2 is measured. Gu et al. 2020 used a characteristic-importance method that omitted each characteristic iteratively, holding the others constant. This subtle but significant difference can lead to different assessments of characteristic importance. Consequently, the characteristic importance rankings in our study differ from those in Gu et al. 2020, and comparisons should be made with this methodological distinction in mind.

Lastly, the Monte Carlo simulation study in this paper is designed with a five factor model Data Generating processes responsible for the variation in excess returns, as opposed to the three factors used in Gu et al. 2020. This design aims to provide a more nuanced and robust examination of the models with both linear and non-linear data-generating processes. The models' abilities to capture complex relationships are better assessed when more factors are present. Furthermore, the simulation methodology in this paper is extended to examine the economic performance of simulated long-short portfolios. The Conditional Autoencoder demonstrates dominant performance in portfolio return prediction within these simulations. This extension provides insights into the models' applicability in portfolio management scenarios.

5 Conclusion

This thesis examines the predictive ability, economic performance, the ability to predict risk versus anomalies, stock characteristic-importance and the robustness, of the Conditional Autoencoder **(CA)** model, as proposed by Gu et al. 2020. The **CA** model offers an alternative to traditional latent factor (and observable) models by conditioning on asset characteristics for its latent risk factors and factor loadings. Unlike linear latent factor models, the **CA** model allows non-linearities in its factor loadings while using linear latent risk factors. At its core, the **CA** model resembles a factor model with conditional factor loadings, but imposes a no-arbitrage restriction, excluding anomaly returns (alphas represented by the intercept term). The prediction performance of the **(CA)** model was evaluated against the benchmark models, **IPCA**, **PCR**, **PLS**, and **FF**. Using both Total- and Predictive R^2 metrics, the models were assessed at the individual stock level and the portfolio level. The findings in this paper demonstrated that the **CA** model consistently outperformed the benchmark models across most factor specifications. For instance, the CA model achieved a Total R^2 of up to**24.76%** at the portfolio level with six factors, significantly higher than the benchmark models, which remained below 1% in most cases. Similarly, the **CA** model demonstrated superior Predictive R^2 values, reaching up to 4% at the individual stock level, while benchmark models often exhibited negative values. However, the overall performance in terms of Total- and Predictive-R^2 was lower in this paper compared to Gu et al. 2020. This discrepancy can be attributed to methodological differences, such as variations in data preprocessing, the use of a different validation scheme, and computational constraints that limited the dataset size and model complexity.

The economic performance of the Conditional Autoencoder (**CA**) models was evaluated against the benchmark models using the financial metrics**Sharpe Ratios**, **Average Returns**, **Value-at-Risk (VaR)**, and **Portfolio Turnover** rates based on forward return predictions. Employing the bottom-up approach proposed by Gu et al. 2019, this paper constructed both equal-weighted and value-weighted long-short portfolios derived from individual stock return predictions.

The empirical analysis revealed that, across both portfolio weighting schemes, the **CA** models consistently outperformed the benchmark models. Specifically, the **CA** models achieved **Sharpe Ratios** as high as **1.66** in equal-weighted portfolios and **1.47** in value-weighted portfolios, significantly exceeding the best-performing benchmark models, which attained their highest **Sharpe Ratios** of **0.97** and **0.38** respectively. Additionally, the the **CA** models delivered higher **Average Returns** up to **4%** compared to **1%** for benchmarks. The **CA** models also delivered lower **VaR** values and maintained moderate turnover rates, indicating superior risk-adjusted performance and efficient trading activity.

This paper evaluated the **CA** model's ability to measure risk compensation and capture anomalies by analyzing the prediction errors (alphas) of characteristic-managed portfolios. Since the **CA** model assumes no-arbitrage and excludes an intercept term, these alphas serve as indicators of the model's pricing accuracy if they're insignificant or small.

A total of 94 characteristic-managed portfolio return forecasts were generated for each model and the alphas were plotted against realized returns. The **CA** models exhibited small pricing errors, with 75 to

91 out of 94 alphas being statistically insignificant at the **5%** significance level. For instance the $\boldsymbol{CA_1}$ (with 1 hidden layer in the beta network) had only 4 significant alphas, indicating a high degree of pricing accuracy. This performance is comparable The **PCR**. **PLS** and **FF** benchmark models, which also showed a high number of statistically insignificant alphas.

The importance of asset characteristics in predicting excess returns was thoroughly evaluated for three variations of the **CA** model with zero to three hidden layers in the factor loading network - for 5-factor model specifications.

These were compared against benchmark latent factor models, **IPCA**, **PCR**, and **PLS**. Our analysis revealed that the characteristic importance in the **CA** models declined less sharply from highest to lowest compared to the benchmark models. This phenomenon may be attributed to insufficient regularization in the **CA** models or their enhanced ability to capture non-linear relationships between stock characteristics.

Specifically, the **CA** models consistently identified **Return on Equity**, **Gross Profitability**, **Sales to Price**, **Volatility of Share Turnover**, and **R&D to Market Capitalization** as the most influential stock characteristics across all model variations. In contrast, these characteristics were generally not among the top factors in the benchmark models. This suggests that the **CA** models are more effective at recognizing key financial indicators that drive stock returns, potentially due to their non-linear modeling capabilities.

Further examination of the **CA** model's internal structures provided additional insights. In the linear **factor network**, the importance sharply declined after the top seven characteristics, with **Market Correlation (Beta)** and **Squared Market Correlation** being consistently the most significant across all models. This highlights the **factor network**'s focus on capturing market-related risk factors. Conversely, in the non-linear **beta network** (for 1 to 3 hidden layers), the decrease in importance among the top 20 characteristics was more gradual. **Volatility of Share Turnover** emerged as the most important characteristic consistently, and **Illiquidity** was significant in all model variations except for the one with three hidden layers, $\boldsymbol{CA_3}$). This indicates that liquidity measures play a crucial role in the **beta network**'s ability to model stock returns.

The disparity between the most important characteristics identified in this study and those reported by Gu et al. 2020 suggests that the selection of significant characteristics by **CA** models can vary based on preprocessing methodologies, model configurations, and the chosen prediction periods. These differences underscore the sensitivity of **CA** models to methodological choices and highlight the importance of standardizing procedures for comparative studies.

The robustness of the **CA** models was assessed by introducing five simulated noise characteristics into the set os asset characteristics. The stability of prediction performance and characteristic selection was evaluated by calculating both the Total R^2 and Predictive R^2 for individual stock returns and portfolio-level predictions. The findings of this paper indicate that the **CA** models continued to outperform the benchmark latent factor models (**IPCA**, **PCR** and **PLS**) - even with the introduction of noise characteristics. For instance the **CA** models achieved a Total R^2 ranging from **8.77%** to **9.44%** for individual stock returns with noise, compared to negative or near-zero values for the benchmark models.

Notably, characteristics such as **Volatility of Share Turnover** and **Return Volatility** remained among

the most important predictors in the **CA** models, even when noise was present. This consistency suggests a degree of stability in characteristic selection, underscoring the robustness of the **CA** models in identifying key drivers of stock returns.

In contrast, the most important stock characteristics identified by Gu et al. 2020 were **Short-term Reversal**, **Stock Momentum**, **Momentum Change**, '**Industry Momentum**', and '**Recent Maximum Return**'. While **Volatility of Share Turnover** was ranked seventh in their study, its prominence in our findings indicates some consistency between the two studies.

The disparity in the top characteristics when noise is introduced, highlights that the **CA** models' selection of important predictors can vary when the market conditions are noisy such as market turmoil, which could, contribute to the model being less stable in its predictions and stock characteristics selection.

A Monte Carlo simulation study, inspired by Gu et al. 2020, was conducted to evaluate the non-linear predictive capabilities of the **CA** models. Utilizing a **Data Generating Process (DGP)** with five systematic risk factors, for both **linear** and **non-linear** scenarios were tested using simulated stock characteristics as inputs. The **CA** models were assessed against the latent factor benchmark models **IPCA**, **PCR**, and **PLS**, both at the individual stock and portfolio return levels.

The results demonstrated that the **CA** models significantly outperformed the benchmark models across both **linear** and **non-linear DGP** settings. Specifically, in the **linear DGP**, the **CA** models achieved a Total-R^2 of up to **61%** at the portfolio level with four factors, whereas the benchmark models exhibited negative Total-R^2 values, indicating poor explanatory power. In the **non-linear DGP**, the **CA** models maintained strong performance with Total-R^2 values reaching **34%**, while the benchmark models continued to show negative or negligible results. This poor performance of the benchmark models was likely due to lack of explicit regularization in the loss functions.

Furthermore, the **CA** models exhibited higher mean-variance efficiency in long-short portfolios. For example, the **CA** models achieved **Sharpe Ratios** up to **0.60** in the **linear DGP** and **0.40** in the **non-linear DGP**, significantly outperforming the benchmark models, which had **Sharpe Ratios** below **0.33** and **0.17**, respectively. This superior performance likely underscores the **CA** models' robustness and adaptability, particularly in capturing complex, **non-linear** relationships in asset returns.

While the **IPCA** model showed some advantages in a Predictive-R^2 of **2.78%** for individual stocks under the linear **DGP**, the **CA** model performed almost as high on **2.37%**. However, the level of the performance of the **CA** model was higher.

These findings have broad implications for asset pricing and portfolio management. The **CA** model's ability to capture both **linear** and **non-linear** relationships and to consistently outperform traditional models suggests it can be a valuable tool for financial practitioners and researchers. By integrating non-linear modeling techniques using architectures that allow for non-linear factor loadings like the **CA** model, there is potential for significantly improved predictive performance and portfolio optimization.

Future research could focus on enhancing the **CA** model's architecture, configurations, to further improve its performance. Investigating the model's applicability across different market conditions, asset classes, and with alternative datasets would also provide deeper insights into its robustness and gener-

alizability. Addressing computational limitations and standardizing methodological approaches across researchers, could facilitate more comprehensive comparisons with existing models.

In conclusion, this thesis demonstrates that the **CA** model offers a strong alternative to traditional factor and conventional latent factor benchmark models in asset pricing. In this paper, the **CA** model excelled particularly at the portfolio level and in mean-variance efficiency, demonstrating robust predictive power and economic performance as well as consistent stock characteristics selection to some degree. Their consistent outperformance across various evaluation metrics and adaptability in non-linear settings affirm their practical advantages. These results contribute to the growing body of evidence supporting advanced machine learning techniques in financial modeling and highlight the potential for future advancements in the field.

A Appendix

A.1 Derivation of the Expected return from the Euler Equation

1. The objective is to derive the expression for the expected excess return $E_t[r_{t+1}]$ in terms of $\beta_{i,t}$ and λ_t:

   $$E_t[m_{t+1} \cdot r_{i,t+1}] = 0$$

   Where m_{t+1} is the Stochastic Discount Factor **(SDF)** at time $t+1$ and $r_{i,t+1}$ is the excess return of asset i at time $t+1$. This rule is used :

   $$E[XY] = E[X] \cdot E[Y] + Cov(X, Y)$$

   which implies that

   $$E_t[m_{t+1}] \cdot E_t[r_{i,t+1}] + Cov(m_{t+1}, r_{i,t+1}) = 0$$

2. The equation is rearranged:

   Subtract $Cov_t(m_{t+1}, r_{i,t+1})$ on both sides:

   $$E_t[m_{t+1}] \cdot E_t[r_{i,t+1}] = -Cov_t(m_{t+1}, r_{i,t+1})$$

3. Isolate $E_t[r_{i,t+1}]$ by dividing with $E_t[m_{t+1}]$ on both sides:

   $$E_t[r_{i,t+1}] = -\frac{Cov_t(m_{t+1}, r_{i,t+1})}{E_t[m_{t+1}]}$$

4. $\beta_{i,t}$ and λ_t are defined:

$$\beta_{i,t} = \frac{Cov_t(m_{t+1}, r_{i,t+1})}{Var_t(m_{t+1})} \quad and \quad \lambda_t = -\frac{Var_t(m_{t+1})}{E_t[m_{t+1}]}$$

Where $\beta_{i,t}$ represents the conditional covariance between the **SDF** and m_{t+1} and asset i's excess return, scaled by the variance of the **SDF** and measures the assets sensitivity to the **SDF**. λ_t represents the negative of the variance of the **SDF**, scaled by its expected value. It can be interpreted as the market price of risk associated with the **SDF**.

5. Express $Cov_t(m_{t+1}, r_{i,t+1})$ in terms of $\beta_{i,t}$. Starting from the definition of $\beta_{i,t}$, $Var_t(m_{t+1})$ is multiplied on both sides, resulting in the $Cov_t(m_{t+1}, r_{i,t+1})$ being isolated:

$$Cov_t(m_{t+1}, r_{i,t+1}) = \beta_{i,t} Var_t(m_{t+1})$$

6. $E_t[m_{t+1}]$ is expressed in terms of λ_t by Multiplying by $E_t[m_{t+1}]$ on both sides of f the definition of λ_t:

$$\lambda_t \cdot E_t[m_{t+1}] = Var_t(m_{t+1})$$

Divide on both sides by λ_t :

$$E_t[m_{t+1}] = \frac{Var_t(m_{t+1})}{\lambda_t}$$

7. Substitute back into the expression for $E_t[r_{i,t+1}]$:

The expressions from steps 5 and 6 are inserted into the equation from step 3 (replace $Cov_t(m_{t+1}, r_{i,t+1})$ with $\beta_{i,t} Var_t(m_{t+1})$. Replace $E_t[m_{t+1}]$ with $-\frac{Var_t(m_{t+1})}{\lambda_t}$), obtaining:

$$E_t[r_{i,t+1}] = -\frac{\beta_{i,t} Var_t(m_{t+1})}{-\frac{Var_t(m_{t+1})}{\lambda}}$$

$$\Leftrightarrow E_t[r_{i,t+1}] = \left(-\frac{\beta_{i,t} Var_t(m_{t+1})}{1}\right) \cdot \left(-\frac{\lambda_t}{Var_t(m_{t+1})}\right) = \beta_{i,t} \cdot \lambda_t$$

$$\Leftrightarrow E_t[r_{i,t+1}] = \beta_{i,t} \cdot \lambda_t$$

A.2 Feed-Forward Neural Networks (FFNs)

Neural Networks have a been present in the literature for a while, and date back to (Rosenblatt 1958). Since then several advances in the field of machine learning, have gained traction in the application of

neural networks. Neural networks belong to the discipline of Machine Learning, which is related to statistics and other disciplines. Machine Learning techniques primarily rely on probabilistic approaches that minimize or optimize certain loss functions with respect to a certain task.

Feed Forward Neural Networks (FNNs) are a fundamental type of artificial neural network, where information flows in one direction from the input layer, through any hidden layers, to the output layer. These networks are widely used in machine learning for tasks such as classification and regression.

We are interested in regression tasks, as we want to predict the excess return of stocks.

A **FFN** consists of multiple layers of interconnected units, or neurons. The basic structure includes an **input layer, one or more hidden layers, and an output layer**. Each layer consists of **Units**. The Units in each layer are connected to units in the subsequent layer through weighted connections.

The Computation in a neural network begins with the input layer, where each input variable is passed directly to the units of the next layer. As data moves through the network, each unit computes a weighted sum of its inputs, applies a non-linear activation function to this sum, and adds a bias term to produce its output. The type of **activation function** used depends on the specific task the neural network is designed to perform[52].

The output of a neural network is computed recursively through a chain of nested functions. Each function corresponds to the interconnected units within each layer, transforming the data as it moves forward. This process, known as ”forward propagation”, is central to how neural networks generate predictions.

In the output layer, the network's final predictions are generated by applying another set of weights and possibly an activation function, depending on the task at hand. For example, in regression task, the output might be a continuous value, such as a stock's predicted excess return (Nagel 2020).

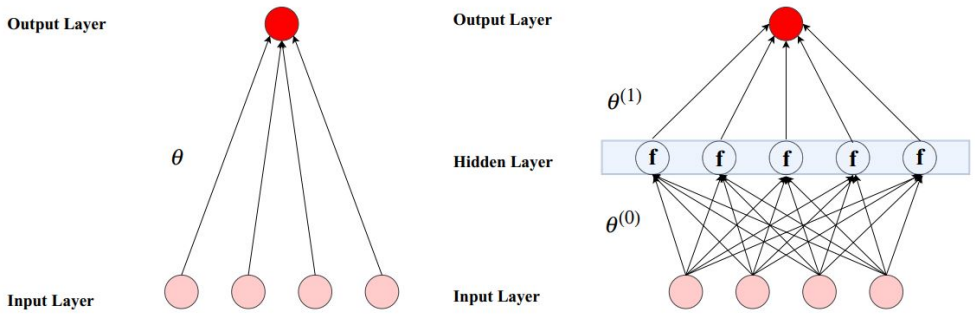


Figure 8: Diagram of two Feed-forward Neural Networks. (LHS) has no hidden layer and (RHS) has one hidden layer (Gu et al. 2019)

Figure 7 shows two diagrams of simple **FFNs**. In both diagrams , the **pink circles** represent the input units of the neural network. These input units directly pass the input data to the network without applying any activation function. The connections between the input units and the subsequent layer are depicted by

[52]Common activation functions include: ReLU, Sigmoid, Tanh and Softmax

arrows, and indicate the flow of information to that layer where the input data is weighted. The **light blue circles** represent the units in the hidden layer. Each of these units applies an activation function to perform a non-linear transformation of the weighted input data. A bias term is then added to this transformed data, which helps the network to model more complex data. The **red circles** represents the output layer unit. This unit receives the weighted sum of the outputs from the prior layer and produces the final output of the network, which is compared against the actual value to assess the network's performance. The **left-hand-side (LHS)** of Figure 7 shows the simplest form of a neural network with four input units (pink circles). These inputs are directly connected to the output unit, where they are weighted using the weight vector θ. The output unit sums all the weighted input information to a final output of the model. Since there are no hidden layers, the network essentially represents a linear model, where the output is a weighted sum of the inputs. The **right-hand side (RHS)** of Figure 7 depicts a slightly more complex network with one hidden layer. In this configuration, each input unit is connected to every unit in the hidden layer, where they are weighted using the weight vector $\theta^{(0)}$. The inputs are transformed non-linearly and biased within each hidden layer unit. The outputs from the hidden layer are then connected and weighted by the weight vector $\theta^{(1)}$ to produce the final sum of weighted output in the output unit. This structure allows the network to capture more complex relationships in the data compared to the simpler linear model shown on the LHS.

A.3 B: Backpropagation

The forward propagation described in Appendix A.2, is the process of computing the final output of a neural network. Once the output has been computed (a forward pass), the final output of the neural network is evaluated with a loss function, that calculates the error of the predicted value to the actual value. The objective of the loss function is to minimize the difference between the actual value and the predicted value. To prevent overfit on the training data, a penalization term is typically added to the loss function. Overfitting occurs when the neural network model learns to make overly precise estimations based on the training data, capturing noise and specific details rather than general patterns. This leads to poor out-of-sample prediction performance.

The regularization term in the loss function penalizes overly complex models by increasing the error, encouraging the neural network to generalize better on unseen data.

Such a loss function can be defined as:

$$O = \sum_{i=1}^{I} loss(y_i, \tilde{y}_i) + penalization \tag{46}$$

where $\tilde{y}_i$ are the estimated values and y_i are the true values.

The loss function sums the errors across all the i observations, subject to a penalization term which makes the loss larger.

To minimize the loss function after a forward pass, the network iteratively updates its weights and

biases. This update process is guided by an algorithm known as **back-propagation**.

Back-propagation uses the **chain rule** to efficiently calculate the gradients of the loss function with respect to each weight and bias in the network. The reason the chain rule is necessary is that the loss function depends on the outputs of the neural network, which in turn depend on the weights and biases through multiple layers of interconnected units.

In other words, the output of the network is a composition of many nested functions (one for each layer), and the chain rule allows us to systematically compute how changes in the weights and biases at each layer affect the final output. By applying the chain rule, we can determine how to adjust the weight and bias to reduce the loss, which leads to a better performance of the neural network.

The back-propagation algorithm uses the following steps for one given sample:

1. **Forward Pass:** A batch of data (a subset of the training data) is used for forward propagation, where the input data points flow from the input layer to the output layer. This process is involves passing the data through each layer of the network, applying weights, biases and activation functions, until the final output is produced.

2. **Error Calculation:** At the final output layer, the error between the predicted values and the actual values is calculated using the loss function. This error represents how far the network's predictions are from the actual targets.

3. **Backward Pass (Gradient Calculation):** Since the neural network is a collection of nested functions, the derivatives of the loss function with respect to the weights in the network are computed, and the weights are updated accordingly.

   To improve the network's performance, adjust the weighs and biases are to reduce the error. this is where the **chain rule** comes into play. The network is a composition of nested functions (one for each layer), and the chain rule allows the calculation of the derivative of the loss function with respect to each weight and bias in the network. This calculation gives the gradients, which indicate the direction and the magnitude of the changes needed in each weight and bias.

4. **Gradient Propagation:** The gradients calculated in the backward pass are then propagated back through the network, layer by layer, starting from the output layer and moving toward the input layer. This backward flow of information adjusts the weights and biases in each layer based on the gradients.

5. **Weight Update:** After the gradients have been propagated through the network, the weights and biases in all layers are updated using an optimization algorithm, such as the **Stochastic Gradient Descent (SGD)**. The goal of this update is to reduce the error of the loss function, thereby improving the predictions for future inputs.

Figure 8 is a visual illustration of the back-propagation algorithm. The forward pass is represented by the **blue arrows** in **Figure 8**, where the data is being processed by the model. The error calculation

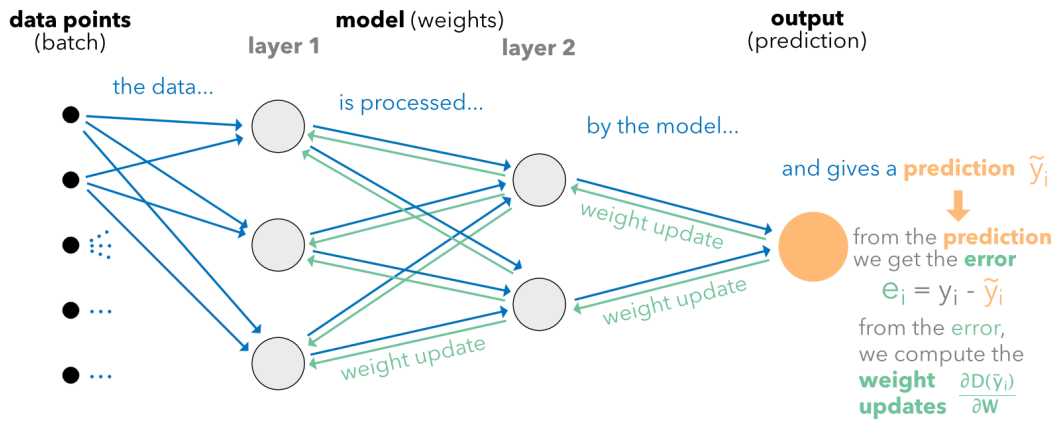


Figure 9: Visual illustration of the Back Propagation algorithm (Coqueret and Guida 2020)

is depicted in **figure 8**, where the error, e_i is calculated by the difference between the actual values y_i and the predicted values $\tilde{y}_i$. In **Figure 8**, the gradient of the loss function is calculated by $\frac{\partial D(\tilde{y}_i)}{\partial W}$. The gradient propagation in figure 2, is depicted by the **green arrows**, which show the calculated gradients being propagated back through the neural network. This process occurs across each layer, moving toward the input layer and involves updating weights and biases accordingly.

For the mathematical description of the back-propagation algorithm, a general loss function is defined as $D(\tilde{y}_i(W)) = loss(y_i, \tilde{y}_i)$, where W denotes all the weights and biases in the neural network.

The iterative process of updating weights and biases within the neural network to minimize the loss function, O, is conducted with the **Gradient Descent Algorithm**. The gradient (or derivative) of the loss function is calculated to determine the direction and magnitude of the adjustment needed.

Gradient Descent is given by :

$$W \to W - \eta \cdot \frac{\partial D(\tilde{y}_i)}{\partial W} \tag{47}$$

Where η is the learning rate, which is a parameter, that determines the size of the weight adjustment.

The following is a mathematical description of back-propagation. Consider a neural network for regression tasks with no activation function in the output unit:

$$\tilde{y}_i = (o_i^{(L)})' w^{(L+1)} + b^{(L+1)} = b^{(L+1)} + \sum_{k=1}^{U_L} w_k^{(L+1)} o_{i,k}^{(L)} \tag{48}$$

Where $\tilde{y}_i$ is the output of the neural network for stock i, which is a linear combination of the outputs from the previous layer, L. The term $o_i^{(L)}$ is the vector of outputs from the previous layer (the last hidden layer L) for stock i. The weights $w_k^{(L+1)}$ represent the connections between the neurons in layer L and the output unit. The term $b^{(L+1)}$ is the bias associated with the output unit.

The output $\tilde{y}_i$ for stock i is computed as the sum of the weighted outputs from all neurons in the last hidden layer L, plus the bias term $b^{(L+1)}$. Specifically, each output $o_{i,k}^{(L)}$ from the k-th neuron in layer L is

multiplied by the corresponding weight $w_k^{(L+1)}$, and then these products are summed along with the bias to produce $\tilde{y}_i$.

The error term of the output layer is given by the derivative of the loss function with respect to the final output of the neural network, $\tilde{y}_i$:

$$\delta_i^{(L+1)} = \frac{\partial D(\tilde{y}_i)}{\partial \tilde{y}_i} = D'(\tilde{y}_i) \tag{49}$$

Where $\delta_i^{(L+1)}$ is the error term, $\frac{\partial D(\tilde{y}_i)}{\partial \tilde{y}_i}$ is the derivative of the loss function with respect to the output of $\tilde{y}_i$ of stock i .

The gradient of the loss function with respect to the weights in the output layer $L+1$ for unit k are:

$$\frac{\partial D(\tilde{y}_i)}{\partial w_k^{(L+1)}} = \delta_i^{(L+1)} \cdot o_{i,k}^{(L)} \tag{50}$$

The gradient of the loss function with respect to the biases in the output layer $L+1$ are:

$$\frac{\partial D(\tilde{y}_i)}{\partial b^{(L+1)}} = \delta_i^{(L+1)} \tag{51}$$

Backpropagation to the Previous Layers: To propagate the error back to the previous layer L, the chain rule is applied. In back-propagation, the goal is to compute how much each weight and bias in the network contributed to the overall error so that these parameters can be adjusted accordingly.

For the k-th unit in layer L, the error term $\delta_{i,k}^{(L)}$ is calculated using the error term from the next layer (layer $L+1$) and the weights connecting the layers. The error term for the k-th unit in layer L for the i-th example is given by:

$$\delta_{i,k}^{(L)} = \frac{\partial D(\tilde{y}_i)}{\partial o_{i,k}^{(L)}} = \delta_i^{(L+1)} \cdot w_k^{(L+1)} \tag{52}$$

where, $\delta_{i,k}^{(L)}$ is the error term of stock i in the k-th unit in layer L, which quantifies the contribution of this unit to the error in the output. The error is propagated backwards through the network by multiplying the error term from the subsequent layer ($\delta_i^{(L+1)}$) by the weight $w_k^{(L+1)}$ that connects the neuron in layer L to the neuron in layer $L+1$. This process allows the network to calculate the impact of each unit in the previous layer on the overall error, facilitating the adjustment of weights and biases to reduce the error in future iterations.

Generalized Backpropagation Through All Hidden Layers: For any hidden layer L with activation function $f^{(L)}$, the error term $\delta_{i,k}^{(L)}$ for the k-th unit for stock i is computed as:

$$\delta_{i,k}^{(L)} = \left(\sum_{m=1}^{U_{L+1}} \delta_{i,m}^{(L+1)} \cdot w_{m,k}^{(L+1)} \right) \cdot f^{(L)'}(v_{i,k}^{(L)}) \tag{53}$$

where $v_{i,k}^{(L)}$ is the pre-activation input:

$$v_{i,k}^{(L)} = \sum_{j=1}^{U_{L-1}} w_{k,j}^{(L)} o_{i,j}^{(L-1)} + b_k^{(L)} \tag{54}$$

The gradients of the loss function with respect to the weights and biases in layer L are:

$$\frac{\partial D(\tilde{y}_i)}{\partial w_{k,j}^{(L)}} = \delta_{i,k}^{(L)} \cdot o_{i,j}^{(L-1)} \tag{55}$$

$$\frac{\partial D(\tilde{y}_i)}{\partial b_k^{(L)}} = \delta_{i,k}^{(L)} \tag{56}$$

These gradients are used to adjust the parameters during training.

Some Gradients can update the weights with extreme differences, which pushes the gradient far away from their optimal value. This means that each update can become too large in order to converge. This is called the "Exploding Gradient Problem". In order to avoid the exploding gradient problem, gradient clipping can be implemented, which specifies the maximum magnitude the gradients can take, which is usually expressed as a norm. If the gradient exceeds the magnitude, it is re-scaled in order to reach the desired threshold. Therefore, the direction is the same, but the adjustment becomes smaller (Coqueret and Guida 2020).

A.4 The conventional Autoencoder model

The Conventional Autoencoder is a type of neural network with a symmetrical architecture, where the input and output dimensions are identical. The hidden layers within the network compress **(encode)** the input signal into a lower-dimensional representation of latent factors. The primary goal of the Autoencoder is to create this set of latent factors, which is used to reconstruct **(decode)** the original data. Thus, the Autoencoder preserves the predictive signal within the data while effectively reducing noise.

Figure 9 illustrates the architecture of a conventional Autoencoder. This model has identical dimensions for the input and output layers, while the hidden layers have reduced dimensionality, which create latent factors. In a financial context, the input layer corresponds to the cross-sectional data of individual stock returns at time period t. The input data is first encoded into a lower-dimensional representation via the hidden layers and is then decoded back to its original dimensionality in the output layer.

Notation and Structure: In the Autoencoder, the hidden layers are sequentially numbered as $l = 1, \ldots, L$, where L represents the total number of hidden layers. Each layer l contains $K^{(l)}$ units. The output of the k-th unit in layer l for data point i is denoted as $o_{i,k}^{(l)}$. The vector of outputs for all units in layer l for data point i is denoted as $o_i^{(l)} = (o_{i,1}^{(l)}, \ldots, o_{i,K^{(l)}}^{(l)})^T$.

The input to the autoencoder is the cross-section of stock returns, represented as $o_i^{(0)} = r_i = (r_{i,t}, r_{i,t+1}, \ldots, r_{i,T})^T$, where $r_{i,t}$ is the return of stock i at time period t, and T is the final time period considered.

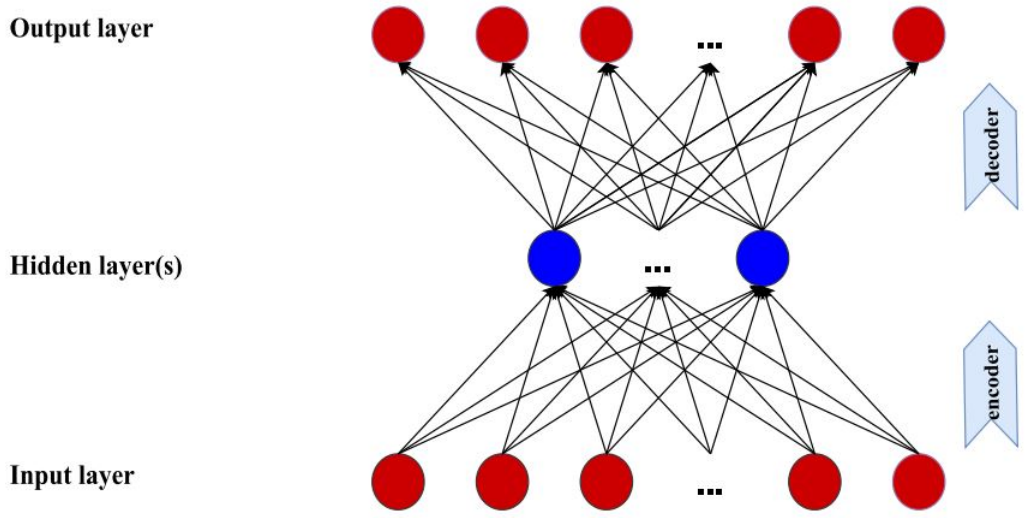


Figure 10: Diagram of a conventional autoencoder neural network (Gu et al. 2020)

Recursive Output Calculation: For any layer $l > 0$, the output of the neural network is recursively defined as:

$$o_i^{(l)} = g(W^{(l-1)} o_i^{(l-1)} + b^{(l-1)}) \tag{57}$$

Where $W^{(l-1)}$ is a $K^{(l)} \times K^{(l-1)}$ matrix containing the weighted parameters connecting layer $l-1$ to layer l, $b^{(l-1)}$ is a $K^{(l)} \times 1$ vector of bias terms for layer l, and $g(\cdot)$ denotes the activation function applied element-wise to the linear combination of inputs. The number of parameters in each hidden layer l is $K^{(l)}(1 + K^{(l-1)})$.

Final Output: The final output of the Autoencoder is given by:

$$G(o_i^{(L)}, b^{(L+1)}, W^{(L+1)}) = W^{(L+1)} o_i^{(L)} + b^{(L+1)} \tag{58}$$

Where $G(o_i^{(L)}, b^{(L+1)}, W^{(L+1)})$ is the reconstructed output vector for data point i, $W^{(L+1)}$ is the matrix of weights connecting the last hidden layer L to the output layer, and $b^{(L+1)}$ is the bias vector for the output layer. The final output, $G(o_i^{(L)}, b^{(L+1)}, W^{(L+1)})$, has the same dimensionality as the input vector r_i, reflecting the Autoencoder's task of approximating the input onto itself.

A.5 description of the diagram of the Conditional Autoencoder model

Figure 10 illustrates a diagram of the structure of the **Conditional Autoencoder (CA)** in matrix form, where the inputs and outputs of all stocks are stacked (leverage efficient matrix operations), such that the model is representative for N stocks. The **CA** model in its general matrix form for all stocks is defined

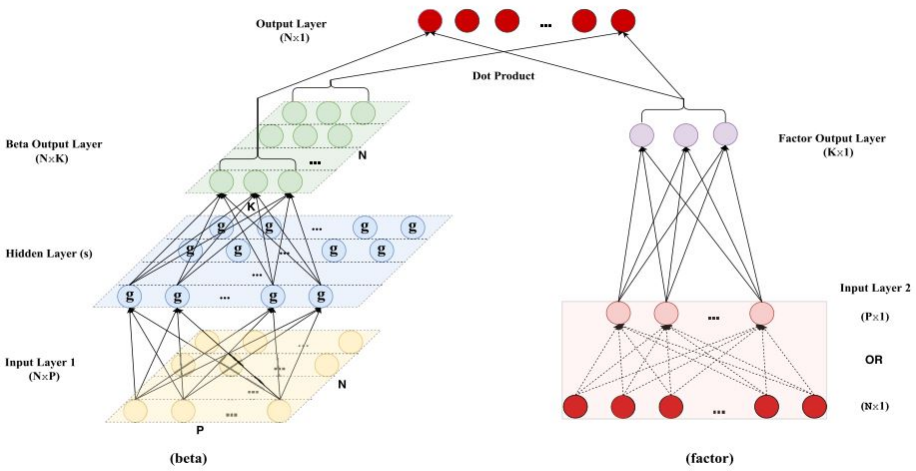


Figure 11: figure from Autoencoder asset pricing models, Gu et al. 2020

by the following factor model equation:

$$r_{t+1} = B_t f_{t+1} + \epsilon_{t+1} \tag{59}$$

Where r_{t+1} is an $N \times 1$ vector of returns for all N stocks at time $t+1$, B_t is an $N \times K$ matrix of factor loadings for all stocks at time t, where each row i of B_t is $\beta_{i,t}$. f_{t+1} is the same $K \times 1$ vector of latent factors and ϵ_{t+1} is an $N \times 1$ vector of error terms for all stocks at time $t+1$. The matrix multiplication, $B_t f_{t+1}$ combines the factor loadings and factor across all stocks. The **left hand side** of the **CA** model **figure 10** depicts the **beta network**, which uses $N \times P$ stock characteristics as input, which consists of an input layer, a hidden layer and and an output layer. The input layer of the beta network in the general matrix form is defined as:

$$Z_t^{(0)} = Z_t \tag{60}$$

Where Z_t is an $N \times P$ matrix of lagged characteristics for all N stocks with each row corresponding to $z_{i,t}^T$ at the individual stock level. The hidden layers of the beta network are defined as:

$$Z_{t-1}^{(l)} = g\left(Z_{t-1}^{(l-1)} W^{(l-1)T} + 1_N b^{(l-1)T}\right) \tag{61}$$

Where $Z_t^{(l)}$ is an $N \times d_l$ matrix, $W(l-1)$ is a $d_{l-1} \times d_l$ weight matrix, transposed to match dimensions. b^{l-1} is a $d_l \times d_l$ bias vector, 1_N is an $N \times 1$ vector of ones (for broadcasting across all stocks and $g(\cdot)$ is an activation function applies element-wise. The output layer of the beta network is defined as:

$$B_t = Z_t^{L_\beta} W^{L_\beta T} + 1_N b^{L_\beta T} \tag{62}$$

Where B_t is the $N \times K$ matrix of factor loadings, $W^{(L_\beta)}$ is a $d_{L_\beta} \times K$ weight matrix, transposed to match dimensions and $b^{(L_\beta)}$ is a $K \times 1$ bias vector.

The **right hand side** side of the diagram in **figure 10** of the **CA** model depicts the **factor network**, which is a linear model that consists of an input layer and an output layer of latent latent risk factors. The input layer of the factor network is defined by the following equations:

$$x_{t+1}^{(0)} = x_{t+1} \tag{63}$$

Where x_{t+1} are is the $P \times 1$ vector of characteristic-managed portfolio returns defined by equation (**). The output layer of the factor network is defined by:

$$f_{t+1} = \tilde{W} x_{t+1}^{(0)} + \tilde{b} \tag{64}$$

Where $\tilde{W}$ is a $K \times P$ weight matrix, $\tilde{b}$ is a $K \times 1$ bias vector and f_{t+1} is a $K \times 1$ vector of latent risk factors. The factor network effectively maps the $P \times 1$ input, x_{t+1}, to the $K \times 1$ output f_{t+1} through linear transformation, thereby reducing computational complexity. The outputs of the beta- and factor networks, B_t and f_{t+1} are multiplied together for prediction of the returns:

$\tilde{r}_{t+1} = B_t f_{t+1}$ Where $\tilde{r}_{t+1}$ is the vector of $N \times 1$ of predicted returns for all stocks at time $t+1$.

A.6 IPCA estimation of the unrestricted model, $\Gamma_\alpha \neq 0$

The unrestricted IPCA : $\Gamma_\alpha \neq 0$ In the unrestricted model, where the $L \times 1$ matrix Γ_α which maps the potential large number of L asset characteristics used as instruments to the dimension reduced intercept in the conditional factor model that the **IPCA** model represents. The intercepts allow for anomalies (alphas). When the **IPCA** model is unrestricted, it allows for the presence of an intercept, which is a function of the asset characteristics that are used as instruments allow the a part of the returns r_{t+1} to be explained by uncompensated risk by the intercept term, α. Because the intercept is allowed to exist, it is now assumed that it is possible for the expected excess returns to have a linear relationship with the asset characteristics may may not be compensated for systematic risk. The unrestricted model is defined as follows:

$$r_{i,t+1} = z_{i,t}^T \Gamma_\alpha + z_{i,t}^T \Gamma_\beta f_{t+1} + \epsilon_{i,t+1}^* \tag{65}$$

where the intercept, $\alpha_{i,t} = z_{i,t}^T \Gamma_\alpha$ is defined as a linear function of stock characteristics and weights, which is a linear combination between stock characteristics used as instruments, $z_{i,t}$ and a $1 \times L$ vector of weights Γ_α for the respective L characteristics. The term $z_{i,t}^T \Gamma_\beta f_{t+1}$ corresponds to the term $\beta_{i,t} f_{t+1}$ for the conditional factor model. Therefore, the notation for the unrestricted **IPCA** model can be written as $r_{i,t+1} = z_{i,t}^T \tilde{\Gamma} \tilde{f_{t+1}} + \epsilon_{i,t+1}^*$, where $\tilde{\Gamma} \equiv [\Gamma_\alpha, \Gamma_\beta]^T$ and $\tilde{f}_t \equiv [1, f_{t+1}^T]^T$. The rest of the equation is the same as the prior defined conditional factor model with an added constant.

The first order condition for the unrestricted model while using the definitions $\tilde{\Gamma}$ and $\tilde{f}_t$ therefore becomes:

$$f_{t+1} = \left(\tilde{\Gamma}^T Z_t^T Z_t \tilde{\Gamma} \right) \tilde{\Gamma}^T Z_t^T \left(r_{t+1} - Z_t \tilde{\Gamma}_\alpha \right) \quad \forall t \tag{66}$$

The first order condition is the same as the one for the restricted **IPCA** model ($\Gamma_\alpha = 0$) (in Section 2.2.2) by taking the first order derivatives of $\tilde{f_{t+1}}$ which become constant. The first order condition for $\tilde{\Gamma}_\alpha$ is found by substituting $\tilde{\Gamma}_\alpha$ and $\tilde{f}_{t+1}$ with $\tilde{\Gamma}$ and $\tilde{f}_{t+1}$ in the first order condition previously defined for the restricted model ($\Gamma_\alpha = 0$). This implies that the first column of the $\tilde{\Gamma}$ matrix is the column corresponding to the Γ_α^T vector and the rest of the L columns belong to the corresponding Γ_β instrument mapping matrix. In order to find a unique solution in the case of this unrestricted **IPCA** model, it becomes necessary to add the restriction of orthogonality between the mapping of instruments for the intercept vector Γ_α and the dimension-reduced mapping from the instruments to the factors, Γ_β, such that $\Gamma_\alpha^T \Gamma_\beta = 0$.

A.7 Derivation of the First Order Condition for the IPCA estimator w.r.t. f_{t+1}

Starting from the objective function:

$$L = \sum_{t=1}^{T-1} |r_{t+1} - Z_t \Gamma_\beta f_{t+1}|^2 \tag{67}$$

The derivative of L w.r.t f_{t+1} is taken of the objective function:

$$\frac{\partial L}{\partial f_{t+1}} = -2\Gamma_\beta^T Z_t^T \left(r_{t+1} - Z_t \Gamma_\beta f_{t+1}\right) \tag{68}$$

Setting the derivative to zero and rearranging:

$$\Gamma_\beta^T Z_t^T Z_t \Gamma_\beta f_{t+1} = \Gamma_\beta^T Z_t^T r_{t+1} \tag{69}$$

Isolating f_{t+1}:

$$f_{t+1} = \left(\Gamma_\beta^T Z_t^T Z_t \Gamma_\beta\right)^{-1} \Gamma_\beta^T Z_t^T r_{t+1} \tag{70}$$

And the First Order Condition for the **IPCA** estimator w.r.t. f_{t+1} is derived.

A.8 Derivation of the First Order Condition for the IPCA estimator w.r.t. Γ_α

The terms for Γ_β are vectorized:

Let $\beta_t = Z_t \Gamma_\beta$ and $\epsilon_t = r_{t+1} - \beta_t f_{t+1}$.

The objective function becomes:

$$L = \sum_{t=1}^{T-1} \left(\epsilon_t^T \epsilon_t\right) \tag{71}$$

The derivative of the objective function w.r.t. Γ_β is taken:

$$\frac{\partial L}{\partial \Gamma_\beta} = -2 \sum_{t=1}^{T-1} Z_t^T \left(r_{t+1} - Z_t \Gamma_\beta f_{t+1} \right) f_{t+1}^T \tag{72}$$

The derivative is set equal to zero:

$$\sum_{t=1}^{T-1} Z_t^T Z_t \Gamma_\beta f_{t+1} f_{t+1}^T = \sum_{t=1}^{T-1} Z_t^T r_{t+1} f_{t+1}^T \tag{73}$$

Vectorizing both sides:

$$\text{Vec} \left(\sum_{t=1}^{T-1} Z_t^T Z_t \Gamma_\beta f_{t+1} f_{t+1}^T \right) = \text{Vec} \left(\sum_{t=1}^{T-1} Z_t^T r_{t+1} f_{t+1}^T \right) \tag{74}$$

Using properties of the Kronecker product[53] :

$$\left(\sum_{t=1}^{T-1} Z_t^T Z_t \otimes f_{t+1} f_{t+1}^T \right) \text{Vec}(\Gamma_\beta) = \sum_{t=1}^{T-1} \left(Z_t^T r_{t+1} \otimes f_{t+1}^T \right) \tag{75}$$

Solving for Vec:

$$\text{Vec}(\Gamma_\beta) = \left(\sum_{t=1}^{T-1} Z_t^T Z_t \otimes f_{t+1} f_{t+1}^T \right)^{-1} \left(\sum_{t=1}^{T-1} \left(Z_t^T r_{t+1} \otimes f_{t+1}^T \right) \right) \tag{76}$$

A.9 Alternating Least Squares (ALS) algorithm

The **Alternating Least Squares**, algorithm works with the following steps[54][55]:

1. An an initial guess for the ALS algorithm is made:

   (a) Calculate Characteristic-managed portfolios:

$$x_{t+1} = Z r_{t+1} \tag{77}$$

   where x_{t+1} is a set of P managed portfolios. The j'th column of the matrix x_{t+1} is a weighted average of stock returns, weighed by the j'th characteristics in Z_t at time period t. Characteristic-

[53] **Definition of the Kronecker Product:** For matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$, the Kronecker product $A \otimes B$ is a $(mp) \times (nq)$ matrix defined as

$$(A \otimes B)_{(i-1)p+k,\,(j-1)q+l} = a_{ij}\, b_{kl},$$

for $i = 1, \ldots, m$, $j = 1, \ldots, n$, $k = 1, \ldots, p$, and $l = 1, \ldots, q$.

[54] Kelly, Priutt, and Su 2018

[55] Kelly, Priutt, and Su 2017

managed portfolios are stacked, such that $X = [x_t^T, ..., x_T^T]$, each column corresponds to a time-series of characteristic-managed portfolios.

(b) second order moments of x_{t+1}

$$X^T X = \sum_t x_{t+1}^T x_{t+1} \tag{78}$$

(c) Make a Singular Value Decomposition (SVD)[56] of the second order moment of the Characteristic-managed portfolios, $X^T X$:

$$X^T X = U \Sigma V^T \tag{79}$$

Where U and V^T are the right and left eigenvector matrices respectively and Σ is the eigenvalue matrix with eigenvalues in its diagonal.

(d) Order the eigenvalues in the eigenvalue matrix Σ with decreasing entries, such that the magnitude of the eigenvalues are decreasing from largest to smallest. Choose the K corresponding eigenvectors (columns) in the left eigenvector matrix U, which are the columns of the initial guess of Γ_β. The columns in Γ_β therefore represent the rotational direction of the coordinate system with the highest variance that represents the variation in the characteristic-managed portfolios. The initial guess, Γ_β corresponds to a static loading matrix estimate by applying the static PCA for the characteristic-managed portfolio returns x_{t+1}. This initial guess is a close approximation of Γ_β that minimizes equation (28), as long as the second order moment of the stock characteristics, $Z_t^T Z_t$ is not too volatile, because the static PCA is applied across all time periods.

2. Use the initial guess of Γ_β as input in equation (20) for estimation of $\hat{f}_{t+1}$
3. use the estimated $\hat{f}_{t+1}$ as input in equation (31) for estimation of Γ_β
4. Repeat steps 2 and 3, such that $\hat{f}_{t+1}$ and Γ_β are iteratively updated in equation (20) and (21) until convergence, defined as the point at which is the absolute change in any element of Γ_β and f_{t+1} (for all t) is smaller than 10^{-6}

A.10 IPCA assumptions for consistency

Kelly, Priutt, and Su 2017 defined the following assumption for consistency of estimation and assumptions for asymptotic normality for the **IPCA** model.

The definitions for assumptions are the following: z, ϵ, ϵ^* and f^0 are random stochastic variables in a panel dataset:

[56] Singular Value Decomposition is described in detail in Appendix A.9

1 Assumptions for consistency

Assumption 1

An instrument is orthogonal to its errors: $\mathbb{E}[z_{i,t}^T \epsilon_{i,t}] = 0$, which comes from the assumption that $\mathbb{E}[z_{i,t}\nu_{\beta,i,t}] = 0$ and that $\mathbb{E}[z_{i,t}^T \nu_{\alpha,i,t} f_{t+1}] = 0$, which amounts to the exclusion of instrumental variables, a common assumption in this type of setting.

Assumption 2 These moments have to exist:

A. $(i)_\, \mathbb{E}\,||f_t^0 f_t^T||$

B. $(ii)_\, \mathbb{E}\,||z_{i,t}\epsilon_{i,t}||^2$

C. $(iii)\, \mathbb{E}\,||z_{i,t}^T z_{i,t}||^2$

D. $(iv)\, \mathbb{E}[||z_{i,t}^T z_{i,t}||^2 ||f_t||^2]$

Assumption 3 The parameter space of Γ is compact and away from rank deficient, $\mathbf{det}\Gamma^T\Gamma > \epsilon$ for $\epsilon > 0$.

Assumptions for asymptotic normality

i. **Assumption 4**

For N and $T \to \infty$,

$$\frac{1}{NT}\sum_{i,t} \text{Vect}(z_{i,t}^T \epsilon_{i,t}^* f_t^{0T}) B \xrightarrow{d} N(0, \Omega^{z\epsilon^* f})$$

For any t as $N \to \infty$,

$$\frac{1}{\sqrt{N}} \mathcal{C}_t' \epsilon_t^* \to^d \mathcal{N}(0, \Omega_t^{z\epsilon^*})$$

$$\lim_{N,T\to\infty} \frac{1}{\sqrt{T}} \sum_t \text{Vecb}(f_t^0 f_t^0 - V^{ff}) \to^d \mathcal{N}(0, \overset{[3]}{\mathbb{V}})$$

where $V^{ff} = \mathbb{V}[f_t^0 f_t^0]$

ii. **Assumption 5** (Bounded dependence)

There exists an $M < \infty$, such that $\forall N, T$ there is

$$\frac{1}{NT}\sum_{i,j,t,s} ||\tau_{i,j,t,s}|| \leq M$$

where $\tau_{i,j,t,s} = \mathbb{E}[z_{i,t}\epsilon_{i,t}\epsilon_{j,s}z_{j,s}]$.

The first assumption in **Assumption 4** is a Central Limit Theorem for panels, which has the asymptotic distribution of the dominant term in the score function, $S(\Gamma^0)$. The second assumption in Assumption 4 is a cross-sectional CLT, which conditions information I on aggregate series and gives factor estimation its asymptotic distribution. The third element in **Assumption 4** is related to orthogonalizing factors as normalization, which introduces

contamination to estimation. The fifth assumption sets constraints on time-series and cross-sectional dependency of $z_{i,t}\epsilon_{i,t}$ as well as asymptotic normality.

Assumption 6

The variation in the cross-sectional second-order moment of $z_{i,t}$ is shut down to retain expressions of the asymptotic variances in specific identification cases. The essence of the asymptotic theory does not depend on Assumption 6. Convergence rates would not change if this were relaxed.

A.11 Principal Components Analysis in details

This Appendix describes the theory behind **Principal Components Regression (PCR)**, which uses **Principal Components Analysis (PCA)** as a preprocessing step for creation of latent factors **(Principal Components)**, which are used as estimated risk factors in a multiple linear regression model **(PCR)** for excess return prediction. **PCA** is primarily used for preprocessing- and estimation-purposes that reduce the dimension from $P \times N$ of the dataset to a smaller dimension $K \times N$, where $P < K$. The latent factors (Principal Components) are constructed such that they obtain the maximum variance, while being orthogonal to each other, implying that each component has its own variation uncorrelated with the components in the set of Principal Components (PC's).

PCA reduces the number of stock characteristics from an $N \times P$ matrix X with N stocks and P characteristics for each time period t, to a reduced $N \times K$ dimension matrix, Y with N stock characteristics and K (PCs):

$$Y = XG \tag{80}$$

Where G is a $P \times P$ matrix with eigenvectors of the covariance matrix of X, Σ_X. K Principal Components are then selected using the diagonalization decomposition method (Appendix A.12) for the covariance matrix Σ_X of X, that decomposes Σ_X into P eigenvectors in a matrix V and P eigenvalues in a matrix, D. Each eigenvector in the matrix V, represents a PC, which corresponds to a direction in the P-dimensional feature space along which the data exhibits the greatest variance[57]. These directions form the axes of a new coordinate system defined by the PCs. The eigenvectors consist of linear combination weights for the original features, which are used to compute the jth PC. The K Principal Components with the most variance are selected, such that G in equation (80), becomes $P \times K$[58], where P denotes the number of Stock Characteristics and K denotes the number of selected PCs (latent factors). In summary, PCA simplifies the dataset from X into Y, reducing complexity and enabling more effective regression. Two constraints are used doing for the PCA.

i) The first constraint is that the columns of Y must be uncorrelated. This makes each factor (PC)

[57]Each eigenvalue in the matrix D , represents the amount of variance in the data along the corresponding eigenvector (PC). The larger the eigenvalue, the more variance is captured by that principal component Baldi and Hornik 1989.

[58]The K PCs are selected by ranking the P eigenvalues in D in descending order, from which the highest K eigenvalues are selected and along with their corresponding Eigenvectors that serve as linear combination weights, w_j.

capture different uncorrelated aspects of the original matrix, X.

ii) The second constraint, is that the variance of the columns in the matrix Y is concentrated among a few columns. A high concentration among a few columns in Y means that a few factors (PCs) will capture most of the variation and therefore explanatory power (signal) in the matrix Y, while the other factors are irrelevant because they consist of noise. There are two conditions imposed for covariance matrix, Σ_Y of the matrix Y:

1. The Covariance of Σ_Y of Y must be diagonal.
2. The second constraint is imposing that when the diagonal elements of Σ_Y are ranked decreasing in magnitude, their values decline (preferably sharply).

The covariance matrix, Σ_Y, of Y is defined as:

$$\Sigma_Y = \frac{1}{I-1} Y^T Y = \frac{1}{I-1} P^T X^T X G = \frac{1}{I-1} G^T \Sigma_X G \tag{81}$$

Where Σ_Y can be decomposed and diagonalized (equation (84) in appendix A.12), such that $\Sigma_Y = QDQ^T$, where it is assumed that Σ_Y is symmetric and there exists an orthogonal matrix, Q and a diagonal matrix, D.

The Diagonalized decomposition of Σ_X in equation (81), gets:

$$\Sigma_Y = \frac{1}{I-1} Y^T Y = \frac{1}{I-1} G^T X^T X G = \frac{1}{I-1} G^T Q D Q^T G \tag{82}$$

From equation (82), orthogonality, $\Sigma_Y = \frac{1}{I-1} D$, is achieved by setting $G = Q$. This means that Σ_Y is a variance matrix for Y, because it only has elements in its diagonal. The columns in Y can be shuffled for decreasing magnitude of variance, such that the diagonal elements of Σ_Y are progressively shrinking. In this manner, the latent factors with the most information are identified in sequence. Y is therefore a linear combination of X with approximately the same information as X. The less important latent factors are discarded, such that a subset of Y, $\tilde{Y}$, is left (Coqueret and Guida 2020).

A.12 Singular Value Decomposition (SVD) and Diagonalization

Singular Value Decomposition: An $I \times K$ matrix X (with $I > K$) has orthonormal columns if, $X^T X = I_K$, where I_K is the $K \times K$ identity matrix. This means that the columns of X are orthogonal to each other and each has unit lenght. If X is a square matrix $(I = K)$ and satisfies $X^T X = X X^T = I$, then X is an orthogonal matrix, implying $X^{-1} = X^T$.

For any $I \times K$ matrix X Singular Value Decomposition (SVD), decomposes the matrix X into three parts:

$$X = U \Sigma Q^T \tag{83}$$

Where the matrix U is an $(I \times I)$ orthogonal matrix whose columns are the **left singular vectors** of X, Σ is a $(I \times K)$ diagonal matrix containing the **singular values** of X on its diagonal (non-negative and arranged in descending order), and Q is a ($K \times K$) orthogonal matrix whose columns are the **right singular vectors** of X. For the diagonal matrix Σ, the non-diagonal elements (singular values) are equal to zero and the diagonal elements are greater than or equal to zero, such that $\Sigma_{i,k} = 0$ for $u \neq k$ and that $\Sigma_{i,i} \geq 0$, when $i, i \geq 0 \in \Sigma_{i,i}$. The diagonal Elements of Σ (Singular Values) are arranged in descending order with respect to their magnitude. The Singular Values in Σ provide insight into the rank and conditioning of X, and the singular vectors in U and V reveal the directions of maximum variance in the data. The SVD of X helps identify the principal directions of variation in the data, where U and V contain the directions of maximum variance, and Σ represents the strength of these directions (Coqueret and Guida 2020).

Diagonalization: When analyzing the covariance matrix, $\Sigma_X = \frac{1}{I-1} X^T X$ of zero mean data X, Σ_X can be diagonalized. There exists an orthoginal matrix V, whose columns are the eigenvectors of Σ_X, and diagonal matrix D, containing the eigenvalues of Σ_X:

$$\Sigma_X = V D V^T \tag{84}$$

Where V is a matrix of eigenvectors (Principal Components) and D is a diagonal matrix of eigenvalues (the variances along the principal components).
In the context of Principal Components Analysis (PCA), the covariance matrix Σ_X is analyzed to identify the Principal Components. The eigenvectors in V correspond to the Principal Components (PCs), and the diagonal elements of D represent the amount of variance explained by each PC. Diagonalization ensures that the covariance matrix Σ_X can be expressed in terms of its eigenvalues and eigenvectors, simplifying the PCA process.

Connection Between SVD and Diagonalization While diagonalization applies specifically to square symmetric matrices, SVD is a more general technique applicable to any matrix. However, when X is symmetric and positive semi-definite (as in covariance matrices), there is a close relationship between SVD and eigenvalue decomposition. The eigenvalues D of Σ_X are equal to the squares of the singular values in Σ (i.e. $D = \Sigma^2$). The eigenvectors V of Σ_X are the same as the right singular vectors V from the SVD of X. This relationship allows the use of SVD as an alternative method for computing the principal components in PCA, providing a robust framework for dimensionality reduction and data analysis.

A.13 Detailed Explanation of Partial Least Squares and Its Equivalence to the Three-Pass Regression Filter

In this appendix, a comprehensive explanation of the **Partial Least Squares (PLS)** regression method and its equivalence to the **Three-Pass Regression Filter (3PRF)** by (Kelly and Priutt 2015) under specific conditions is provided. This detailed exposition includes mathematical formulations, algorithmic steps, and proofs to elucidate how PLS can be viewed as a special case of the 3PRF. Understanding this equivalence offers valuable insights into dimension reduction techniques aimed at maximizing predictive accuracy in regression models with many predictors (Kelly and Priutt 2015).

A.13.1 Partial Least Squares (PLS) Regression

Partial Least Squares regression is a statistical method that models the relationship between a set of predictor variables (features) and a response variable by extracting a set of latent factors (components). These factors are constructed to maximize the covariance between the predictors and the response, thereby directly focusing on predictive power rather than merely explaining variance among the predictors (as in Principal Component Regression) (Kelly and Priutt 2015) (Hastie et al. 2009).

Notation: Let T be the number of observations and P be the number of predictors. Let $\mathbf{X} \in \mathbb{R}^{T \times P}$ be the matrix of predictor variables. Let $\mathbf{y} \in \mathbb{R}^T$ is the response variable. Let $\mathbf{w}_k \in \mathbb{R}^P$ be the weight vector for the k -th latent factor. Let $\mathbf{t}_k = \mathbf{X}\mathbf{w}_k$ is the k -th latent score vector.
The objective is to find weight vectors $\mathbf{w}_k$ such that the covariance between the score vector $\mathbf{t}_k$ and the response $\mathbf{y}$ is maximized, while ensuring that each $\mathbf{t}_k$ is orthogonal to the previous ones.

The **PLS Algorithm** consists of the following steps:

Step 1: Data Preprocessing

Centering (and optionally scaling):
$\mathbf{X}_0 = \mathbf{X} - \mathbf{1}_T \boldsymbol{\mu}_X^\top, \quad \mathbf{y}_0 = \mathbf{y} - \bar{y}\mathbf{1}_T$
where $\boldsymbol{\mu}_X$ is the vector of means of $\mathbf{X}$, $\bar{y}$ is the mean of $\mathbf{y}$, and $\mathbf{1}_T$ is a $T \times 1$ vector of ones.

Step 2: Iterative Extraction of Latent Factors

For $k = 1, \ldots, K$:

(a) Compute the weight vector $\mathbf{w}_k$:

$$\mathbf{w}_k = \frac{\mathbf{X}_{k-1}^\top \mathbf{y}_{k-1}}{\left\| \mathbf{X}_{k-1}^\top \mathbf{y}_{k-1} \right\|}$$

This step finds the direction in $\mathbf{X}_{k-1}$ that has the maximum covariance with $\mathbf{y}_{k-1}$.

(b) Compute the score vector $\mathbf{t}_k$:

$$\mathbf{t}_k = \mathbf{X}_{k-1}\mathbf{w}_k$$

(c) Compute the loading vector $\mathbf{p}_k$:

$$\mathbf{p}_k = \frac{\mathbf{X}_{k-1}^\top \mathbf{t}_k}{\mathbf{t}_k^\top \mathbf{t}_k}$$

(d) Compute the regression coefficient q_k:

$$q_k = \frac{\mathbf{y}_{k-1}^\top \mathbf{t}_k}{\mathbf{t}_k^\top \mathbf{t}_k}$$

(e) Deflate $\mathbf{X}_{k-1}$ and $\mathbf{y}_{k-1}$:

$$\mathbf{X}_k = \mathbf{X}_{k-1} - \mathbf{t}_k \mathbf{p}_k^\top$$

$$\mathbf{y}_k = \mathbf{y}_{k-1} - q_k \mathbf{t}_k$$

This ensures that subsequent latent factors capture new information not explained by previous factors.

Step 3: Construct the Final Model

(a) **Scores Matrix:**

$$\mathbf{T} = [\mathbf{t}_1, \ldots, \mathbf{t}_K]$$

(b) **Regression Coefficients:**

$$\boldsymbol{\beta}_{\text{PLS}} = \mathbf{W}(\mathbf{P}^\top \mathbf{W})^{-1}\mathbf{q}$$

where $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_K]$, $\mathbf{P} = [\mathbf{p}_1, \ldots, \mathbf{p}_K]$, and $\mathbf{q} = [q_1, \ldots, q_K]^\top$.

(c) **Predicted Response:**

$$\hat{\mathbf{y}} = \mathbf{T}\mathbf{q}$$

Interpretation: Each $\mathbf{t}_k$ represents a latent factor that is a linear combination of the original predictors, chosen to maximize its covariance with the response $\mathbf{y}$. The deflation step ensures that each latent factor is orthogonal to the previous ones, preventing multicolinearity. PLS emphasizes the components that are the most relevant for predicting $\mathbf{y}$, rather than just explaining variance in $\mathbf{X}$.

A.13.2 The Three-Pass Regression Filter (3PRF)

The **Three-Pass Regression Filter (3PRF)** is a method designed to extract relevant predictive components from a large set of predictors, particularly in high-dimensional settings where P is large relative to T . The 3PRF focuses on identifying and utilizing factors that are most relevant to the target variable, thus improving predictive accuracy (Kelly and Priutt 2015).

Notation: $\mathbf{x}_t \in \mathbb{R}^P$ is denoted as the vector of predictors at time t. y_{t+1} is the target variable to be forecasted. $\mathbf{Z}_t$ denotes a matrix of proxy variables at time t. let $\hat{\mathbf{\Phi}} \in \mathbb{R}^{P \times L}$ be the matrix of estimated sensitivities from the first pass. Finally, $\hat{\mathbf{F}}_t \in \mathbb{R}^L$ is the vector of estimated factors at time t.

The **3PRF Algorithm** consists of three passes (steps):

1. **First Pass:** Estimating Sensitivity of Predictors to Factors

   For each predictor $i = 1, \ldots, P$:

   (a) Run Time Series Regression:

   $$x_{i,t} = \phi_{0,i} + \mathbf{Z}_t^\top \boldsymbol{\phi}_i + \epsilon_{i,t}$$

   (b) Obtain Estimated Coefficients:

   $$\hat{\boldsymbol{\phi}}_i = [\hat{\phi}_{i1}, \ldots, \hat{\phi}_{iL}]^\top$$

2. **Second Pass:** Estimating the Latent Factors

   For each time t:

   (a) Run Cross-Sectional Regression:

   $$\mathbf{x}_t = \boldsymbol{\phi}_{0,t} + \hat{\mathbf{\Phi}}\mathbf{F}_t + \boldsymbol{\epsilon}_t$$

   (b) Solve for Estimated Factors:

   $$\hat{\mathbf{F}}_t = (\hat{\mathbf{\Phi}}^\top \hat{\mathbf{\Phi}})^{-1} \hat{\mathbf{\Phi}}^\top \mathbf{x}_t$$

3. **Third Pass:** Forecasting the Target Variable

   (a) Run Time Series Regression:

   $$y_{t+1} = \beta_0 + \hat{\mathbf{F}}_t^\top \boldsymbol{\beta} + \eta_{t+1}$$

   (b) Obtain Forecasts:

   $$\hat{y}_{t+1} = \hat{\beta}_0 + \hat{\mathbf{F}}_t^\top \hat{\boldsymbol{\beta}}$$

Interpretation: The first two passes aim to extract factors from the predictors that are most relevant for forecasting y_{t+1}. By focusing on these factors, the 3PRF reduces the dimensionality of the predictor set, mitigating issues related to multicollinearity and overfitting. Similar to PLS, the 3PRF emphasizes components that have predictive power for the target variable.

A.13.3 Equivalence of PLS and 3PRF

The PLS method becomes equivalent to the 3PRF under the following these conditions:

1. Predictors are Demeaned and Variance-Standardized:

$$\tilde{x}_{i,t} = \frac{x_{i,t} - \bar{x}_i}{s_i}, \quad i = 1, \ldots, P$$

   where $\bar{x}_i$ is the mean and s_i is the standard deviation of predictor i.

2. No Intercept Terms in the First Two Passes:

   - The regressions in the first and second passes of the 3PRF are performed without intercepts.

3. Automatic Selection of Proxies:

   - The proxy variables $\mathbf{Z}_t$ in the first pass are chosen to be proportional to the response variable $\mathbf{y}_t$ or residuals from an initial regression.

A.13.4 Mathematical Proof of Equivalence

1. First Pass Equivalence:

   - In PLS:

$$\mathbf{w}_1 = \frac{\mathbf{X}^\top \mathbf{y}}{\|\mathbf{X}^\top \mathbf{y}\|}$$

   - In 3PRF:

     (a) Each predictor $\tilde{x}_{i,t}$ is regressed on y_t without an intercept:

$$\tilde{x}_{i,t} = \phi_i y_t + \epsilon_{i,t}$$

     (b) The estimated coefficient is:

$$\hat{\phi}_i = \frac{\sum_{t=1}^{T} \tilde{x}_{i,t} y_t}{\sum_{t=1}^{T} y_t^2} = \frac{\text{Cov}(\tilde{x}_i, y)}{\text{Var}(y)}$$

     (c) Since predictors are standardized and $\text{Var}(y)$ is common, $\hat{\boldsymbol{\phi}} = [\hat{\phi}_1, \ldots, \hat{\phi}_P]^\top$ is proportional to $\mathbf{X}^\top \mathbf{y}$, matching $\mathbf{w}_1$ up to a scaling factor.

2. Second Pass Equivalence:

   - In 3PRF:

(a) The cross-sectional regression without intercept:

$$\tilde{\mathbf{x}}_t = \hat{\mathbf{\Phi}} F_t + \boldsymbol{\epsilon}_t$$

(b) Solving for F_t:

$$F_t = (\hat{\mathbf{\Phi}}^\top \hat{\mathbf{\Phi}})^{-1} \hat{\mathbf{\Phi}}^\top \tilde{\mathbf{x}}_t$$

- Relation to PLS Score:
  - Since $\hat{\mathbf{\Phi}}$ is proportional to $\mathbf{w}_1$, and $\tilde{\mathbf{x}}_t$ is the standardized predictor vector at time t, we have:

    $$F_t \propto \mathbf{w}_1^\top \tilde{\mathbf{x}}_t = t_{1,t}$$

    where $t_{1,t}$ is the first PLS score at time t.
  - Thus, the estimated factor F_t in 3PRF is proportional to the first PLS score $t_{1,t}$.

3. Third Pass Equivalence:

- In 3PRF:

$$y_{t+1} = \beta F_t + \eta_{t+1}$$

- In PLS:

$$y_t = q_1 t_{1,t} + \tilde{y}_{1,t}$$

- Conclusion:
  - Since $F_t \propto t_{1,t}$, the regression equations are equivalent up to scaling, leading to the same forecasts.

Extension to Multiple Components: The equivalence between PLS and 3PRF extends to multiple components when the deflation steps are handled similarly.

- Deflation in PLS:

$$\mathbf{X}_k = \mathbf{X}_{k-1} - \mathbf{t}_k \mathbf{p}_k^\top$$

- Deflation in 3PRF:
  - After extracting the first factor, residuals can be computed:

    $$\tilde{\mathbf{x}}_t^{(1)} = \tilde{\mathbf{x}}_t - \hat{\mathbf{\Phi}} F_t$$

  - Subsequent factors are extracted using the residuals, analogous to PLS.

- Result:

– The subsequent factors in both methods align, and the forecasts remain equivalent under the specified conditions.

A.14 Walk-Forward validation scheme

Figure 11 illustrates the structure of the walk-forward cross-validation scheme. The **x-axis** represents the time period, while the **y-axis** indicates the start year of the test set for each iteration. Each year of the training set, validation set and test set are marked by a yellow, green and blue dot respectively. As shown in **Figure 11**, the lengths of the time-periods of the training, validation, and test sets are fixed, and they move forward by one year (the length of the test set) for prediction of one year at a time, consisting of 18 iterations in total (hence the words ”walk forward”).

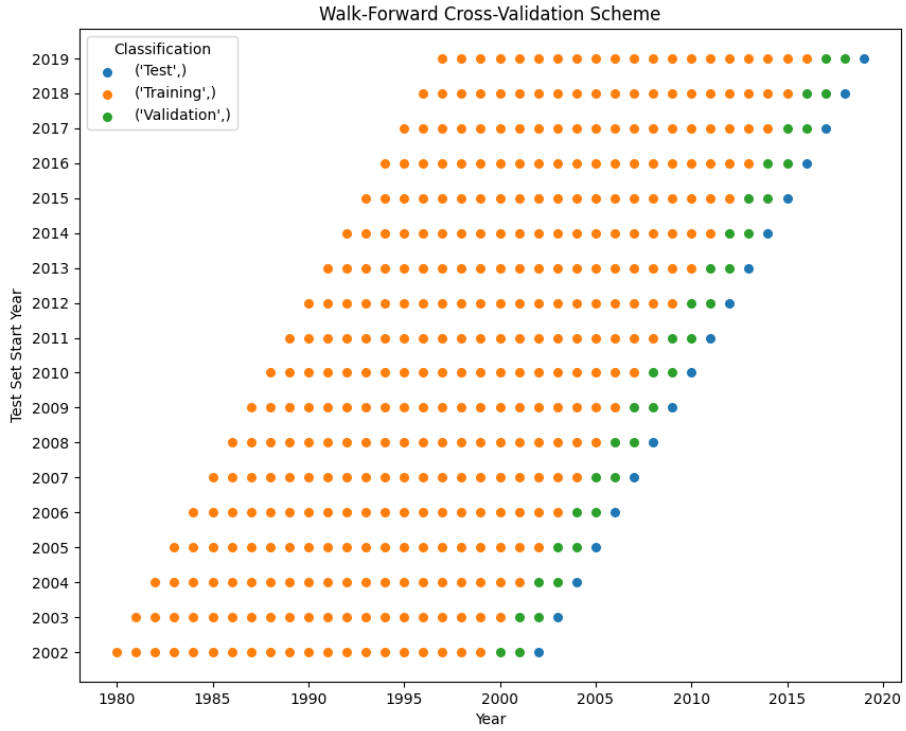


Figure 12: Illustration of the Walking forward cross-validation scheme

A.15 Predictive R^2 by Gu et al. 2020

The the predictive R^2 is defined by Gu et al. 2020 and (Kelly, Priutt, and Su 2018), as the returns in period $t+1$ relative to the factors and the factor loadings in period t. In a no-arbitrage setting, this metric is summarizing the ability of the model to describe risk compensation through exposure to systematic risk. If arbitrage is allowed, it is describing how good the characteristics explain he expected returns through

factor loadings and/or anomaly intercepts. The predictive R^2 is defined as the following:

$$R^2_{Pred} = 1 - \frac{\sum_{i,t\in test}(r_{i,t+1} - \hat{\beta}'_{i,t}\hat{\lambda}_t)^2}{\sum_{i,t\in test} r^2_{i,t+1}} \tag{85}$$

Where $\hat{\lambda}_t$ is the sample average of the estimated risk factors $\hat{f}$ until time period t and $\hat{\beta}_{i,t}$ are the estimated factor loadings at time t. ($test$ denotes the out-of-sample prediction period on the test set.)

A.16 L: Calculation of Portfolio Weights

Value weights: The market capitalization, $MC_{i,t}$, of stock i at time t is calculated as:

$$MC_{i,t} = p_{i,t} \cdot \phi_{i,t} \tag{86}$$

Where p_i and ϕ are the price and the number of shares outstanding for stock i at time t. The total market capitalization of all n stocks in the portfolio at time t is computed by summing the individual market capitalizations:

$$MC_{total,t} = \sum_{i=1}^{n} MC_{i,t} \tag{87}$$

The portfolio weight for stock i at time t, $\omega^p_{i,t}$, is calculated as the share of its share of market capitalization of the total market capitalization of the portfolio :

The market capitalization of stock i at time t as a share of the total market capitalization for all n stocks at time t is the market capitalization:

$$\omega^p_{i,t} = \frac{MC_{i,t}}{MC_{total,t}} \tag{88}$$

Where $MC_{total,t}$ is the total market capitalization of all n stocks at time t, while $\omega^p_{i,t}$ is the calculated weight for for stock i at time t in the value-weighted portfolio.

Equal weights The weights in an equally weighted portfolio are all equal in as a share of the total number of stocks within the portfolio. The formula is defined as :

$$\omega_{i,t} = \frac{1}{n} \tag{89}$$

Where $\omega_{i,t}$ is the equal weight of stock i at time t, while n is the total number of stocks in the portfolio.

A.17 Law of Large Numbers (LLN) and Central Limit Theorem (CLT)

A.17.1 Law of Large Numbers (LLN)

The Law of Large Numbers (LLN) is states that:

For every positive number, ϵ, the probability that the sample average deviates from the population mean by more than ϵ approaches zero as the number of observations approaches infinity:

$$\lim_{n\to\infty} P(|\bar{X}_n - \mu| \geq \epsilon) = 0 \tag{90}$$

Where $\bar{X}_n$ is the sample mean and μ is the population mean.

A.17.2 Central Limit Theorem (CLT)

The Central limit theorem states that the distribution of the sum (or average) of a large number of independent, identically distributed (i.i.d.) random variables, will regardless of the original distribution approximate a normal (Gaussian) distribution, as the number of variables grow. Formally, the CLT is stated as follows :

If $X_1, X_2, ..., X_n$ are $i.i.d.$ random variables with mean μ and finite variance, σ^2 , then the distribution of the sample mean, $\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$ approaches a normal distribution with mean μ and variance $\frac{\sigma^2}{n}$ as n becomes large. This implies that :

$$\frac{\bar{X}_n - \mu}{\frac{\sigma}{\sqrt{n}}} \xrightarrow{d} \mathcal{N}(0,1) \tag{91}$$

References

Baldi, Pierre and Kurt Hornik (1989). "Neural Networks and Principal Components Analysis: Learning From Examples Without Local Minima". In: *Neural Networks* 2, pp. 53–58.

Carhart, Mark M. (1997). "On Persistence in Mutual Fund Performance." In: *The Journal of Finance*.

Chen, Luyang and Katarzyna Perez (2021). "Deep Learning in Asset Pricing". In: *Management science* 70.2, pp. 714–750.

Chinco, Alex, Adam D. Clark-Joseph, and Mao Ye (2017). "Sparse Signals in the Cross-Section of Returns". In: *The Journal of The American Finance Association* 74.1, pp. 449–492.

Cochrane, John H. (2011). "Presidential Address: Discount Rates". In: *The Journal of Finance* 66.4, pp. 1047–1108.

Connor, Gregory and Robert A. Korajczyk (1986). "Performance Measurement with the Arbitrage Pricing Theory: A New Framework for Analysis". In: *Journal of Financial Economics*.

Coqueret, Guillaume and Tony Guida (2020). *Machine Learning for Factor Investing*. Chapman and Hall/CRC; 1st edition. Chap. 15 Unsupervised learning.

Fama, Eugene F. and Kenneth R. French (1993). “Common Risk Factors in the Returns on Stocks and Bonds.” In: *The Journal of Finance*.

— (2015). “A Five-Factor Asset Pricing Model.” In: *The Journal of Finance*.

Gu, Shihao, Bryan Kelly, and Dacheng Xiu (2019). “Empirical Asset Pricing via Machine Learning”. In: *Journal of Financial Economics* 134.3, pp. 501–522.

— (2020). “Autoencoder Asset Pricing Models”. In: *Working Paper*.

Hansen, Lars Peter and Scott F. Richard (1987). “The Role of Conditioning Information in Deducing Testable Restrictions Implied by Dynamic Asset Pricing Models”. In: *Econometrica*.

Harvey, Campbell R. and Yan Liu Heqing Zhu (2016). “... and the Cross-Section of Expected Returns”. In: *The Review of Financial Studies* 29.1, pp. 5–68.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference and Prediction*. Springer; 2nd edition. Chap. 3.5.2 Partial Least Squares.

Hou, Kewei, Chen Xue, and Lu Zhang (2016). “Does Academic Research Destroy Stock Return Predictability?” In: *The Journal of Finance* 71.1, pp. 5–32.

Jong, Sijmen de (1993). “SIMPLS: An alternative approach to partial least squares regression”. In: *Chemometrics and Intelligent Laboratory Systems* 18, pp. 251–263.

Kaczmarek, Tomasz, Markus Pelger, and Jason Zhu (2022). “Building portfolios based on machine learning predictions”. In: *Economic research - Ekonomska istraživanja* 35.1, pp. 19–37.

Kelly, Bryan and Seth Priutt (2015). “The three-pass regression filter: A new approach to forecasting using many predictors”. In: *Journal of Econometrics* 186, pp. 294–316.

Kelly, Bryan, Seth Priutt, and Yinan Su (2017). “Instrumented Principal Components Analysis”. In: *Working Paper*.

— (2018). “Characteristics are Covariances: A Unified Model of Risk and Return”. In: *Journal of Financial Economics* 134.3, pp. 501–524.

Linter, John (1965). “The Valuation of Risk Assets and the Selection of Risky Investments in Stock Portfolios and Capital Budgets”. In: *The Review of Economics and Statistics*.

Ludvigson, Sydney C. and Serena Ng (2007). “The Empirical Risk-Return Relation: A Factor Analysis Approach”. In: *Journal of Financial Economics*.

Nagel, Stefan (2020). *Machine Learning for Algorithmic Trading*. Packt Publishing; 2nd ed. edition. Chap. Autoencoders for Conditional Risk Factors and Asset Pricing.

Rosenblatt, Frank (1958). “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”. In: *Psychological Review*.

Ross, Stephen A. (1976). “The Arbitrage Theory of Capital Asset Pricing Pricing”. In: *Journal of Economic Theory*.

Sharpe, William F. (1964). “Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk.” In: *The Journal of Finance*.